

M.Sc. Engg. (CSE) Thesis

Generating Unique Neighborhood Networks From Seed Graphs

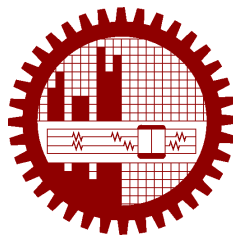
Submitted by

Siam Habib

0422052126

Supervised by

Dr. Sadia Sharmin



Submitted to

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology
Dhaka, Bangladesh

in partial fulfillment of the requirements for the degree of
Master of Science in Computer Science and Engineering

April 2026

Candidate's Declaration

I, do, hereby, certify that the work presented in this thesis, titled, “Generating Unique Neighborhood Networks From Seed Graphs”, is the outcome of the investigation and research carried out by me under the supervision of Dr. Sadia Sharmin, Associate Professor, Department of CSE, BUET.

I also declare that neither this thesis nor any part thereof has been submitted anywhere else for the award of any degree, diploma or other qualifications.

Siam Habib

0422052126

The thesis titled “**Generating Unique Neighborhood Networks From Seed Graphs**”, submitted by Siam Habib, Student ID 0422052126, Session April 2022, to the Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology, has been accepted as satisfactory in partial fulfilment of the requirements for the degree of Master of Science in Computer Science and Engineering and approved as to its style and contents on April 18, 2026.

Board of Examiners

1. _____
Dr. Sadia Sharmin
Associate Professor
Department of CSE, BUET, Dhaka
Chairman
(Supervisor)
2. _____
Dr. Md. Saidur Rahman
Professor
Department of CSE, BUET, Dhaka
Member
(Co-Supervisor)
3. _____
Dr. Tanzima Hashem
Professor and Head
Department of CSE, BUET, Dhaka
Member
(Ex-Officio)
4. _____
Dr. Muhammad Abdullah Adnan
Professor
Department of CSE, BUET, Dhaka
Member
5. _____
Dr. Atif Hasan Rahman
Associate Professor
Department of CSE, BUET, Dhaka
Member
6. _____
Dr. Mohammad Kaykobad
Distinguished Professor
Department of Computer Science and Engineering
BRAC University, Dhaka
Member
(External)

Acknowledgement

First, I would like to express my gratitude to my supervisor and co-supervisor, Dr. Sadia Sharmin and Dr. Saidur Rahman, for their guidance and support during this thesis.

I would also like to acknowledge Dr. Debajyoti Mondal, who not only went through the arduous task of reviewing, revising, and editing my initial drafts to make them presentable, but also was the one who came up with the original idea for the FPT algorithm for the UNN Extension Problem.

I would like to acknowledge Anwarul Azim and Ruhul Azgor from the GDIVL. Awnwar was present in the lab during the day I came up the original problems and was the first to know my initial results. He helped tremendously in proof reading my original drafts. Azgor also helped in vetting my results.

Dhaka
April 18, 2026

Siam Habib
0422052126

Contents

Candidate's Declaration	i
Board of Examiners	ii
Acknowledgement	iii
List of Figures	vii
List of Tables	viii
List of Algorithms	ix
Abstract	x
1 Introduction	1
1.1 Graphs and Notations	1
1.2 Unique Neighborhood Networks	2
1.3 Applications of Unique Neighborhood Networks	3
1.3.1 Topology-Aware Addressing in Networks Without Naming Authorities	3
1.3.2 Peer Authentication	4
1.4 Current Known Methods for Generating UNNs and Their Limitations	5
1.5 The UNN Extension Problem	6
1.6 The UNN Completion Problem	8
1.7 Our Contribution	8
1.8 Thesis Organization	9
2 Preliminaries	10
2.1 Decision And Optimization Problems	10
2.2 Approximating Optimization Problems	11
2.3 Reductions and Approximating Preserving Reductions	11
2.4 Dominating Set and Total Dominating Set	12
2.5 Parameterizing Hard Problems And Fixed Parameter Tractability	13
2.5.1 Fixed Parameter Tractability	14

2.6	FPT reduction and W-Hierarchy	14
2.7	Treewidth and Courcelle's Theorem	16
2.7.1	Monadic Second Order Logic	17
2.7.2	Courcelle's Theorem	19
3	An Approximation Algorithm for UNN Extension Problem	20
3.1	Concept of Suppressors and Suppressed Nodes	20
3.2	Relating Dominating Set to the UNN Extensions	22
3.3	An Approximation Algorithm	23
4	Hardness of Approximation for the UNN Extension Problem	27
4.1	UNNification of a Graph	27
4.2	Properties of $\overline{U_G}$	30
4.3	Blue Dominating Set Problem and its Relation with the Total Dominating Set Problem	31
4.4	Reduction from Total Dominating Set Problem to the Minimum-cost UNN Extension Problem	32
4.5	On Hardness of Decision Variant and Regular Optimization Variant of the UNN Extension Problem	35
5	Hardness of Approximation for the UNN Completion Problem	36
5.1	A Reduction from the UNN Extension Problem to the UNN Completion Problem	36
6	An Approximation Algorithm for the UNN Completion Problem	39
6.1	Notation	39
6.2	Effects of Edge Insertions	39
6.3	An Algorithm for the UNN Completion Problem	43
6.3.1	Recovery Patch	44
6.3.2	Structure of a Recovery Patch	48
6.3.3	Relating Suppression Pairs To Recovery Patches	52
6.3.4	Suppressor Partitioned Graphs	54
6.3.5	Resolving Recovery Tail Suppression Pairs	55
6.3.6	Removing a Single Suppression Pair	59
6.3.7	Removing All Suppression Pairs from a Suppressor Partitioned Graph	60
6.3.8	Putting It All Together	61
7	UNN Extension and Completion Problems from a Parameterized Perspective	64
7.1	Parameterized Hardness of the UNN Extension and Completion Problems	64
7.2	The UNN Extension Problem Parameterized by Treewidth	66

8	Miscellaneous	67
8.1	No Small Forbidden Subgraphs in UNNs	67
8.1.1	Constructing UNN from Arbitrary Graphs	67
8.1.2	Properties of $c^*(n)$	69
8.1.3	Optimality of Construction	71
8.1.4	Size of Forbidden Induced Subgraph	71
8.2	The UNN k -Extension Problem	73
9	Conclusion	74
	References	78
	Index	80

List of Figures

1.1	Examples of UNNs	3
1.2	Examples of Graph Extensions	7
1.3	Example of a UNN Extension and a non-UNN Extension	7
2.1	Example of A Tree Decomposition of A Graph	17
3.1	Example of A Good and A Bad Dominating Set	23
3.2	Reduction from the Minimum Dominating Set Problem to the Minimum UNN Extension Problem	24
3.3	Finding A Good Dominating Set From A Dominating Set In The Complement Graph of A UNN (Case 3)	25
4.1	UNNification of A Graph	28
4.2	UNNification Step By Step	29
4.3	Reduction from Total Dominating Set Problem in General Graphs to Minimum Dominating Set Problem in Complement Graphs of UNNs	33
5.1	Reduction from the UNN Extension Problem to the UNN Completion Problem	38
8.1	Construction of $UNN(G, c)$ step by step	68

List of Tables

2.1 Abbreviation Of Optimization Problem Names	11
--	----

List of Algorithms

1	Approximation Algorithm for the UNN Extension Problem	23
2	Algorithm for solving Total Dominating set problem using a black box solution to the UNN Extension Problem	32
3	Approximation Algorithm for the UNN Extension Problem	37
4	Remove Suppression Pair from a Graph (version 1)	43
5	Find a UNN Completion of a Graph (version 1)	43
6	Find Recovery Patch in a non-UNN from Suppression Pair	47
7	Make Graph Suppressor Partitioned	55
8	Remove Suppression Pairs from Tail Vertices	58
9	Remove Suppression Pair from a Graph (final version)	59
10	Remove All Suppression Pair from a Suppressor Partitioned Graph	60
11	Find a UNN Completion of a Graph	61
12	Algorithm for solving the Set Cover problem parameterized by set cover size using black box for the total dominating set problem parameterized by total domination number	65

Abstract

Unique Neighborhood Networks (UNNs) are a class of graphs introduced by Rass [1], motivated by applications in computer networking and cryptography. A graph is a UNN if no vertex has its open neighborhood contained in the open neighborhood of another vertex. This property enables vertex identification using local neighborhood structure and has been proposed for applications such as topology-aware addressing and peer authentication in distributed networks.

In practical settings, UNN-based systems may need to evolve over time as nodes join or network links fail. Motivated by this, we study the problem of generating and maintaining UNNs under structural changes. In particular, we consider two fundamental problems: the *UNN Extension Problem*, which asks how to add a single vertex to a UNN while preserving the UNN property using a minimum number of incident edge insertions, and the *UNN Completion Problem*, which asks how to transform an arbitrary graph into a UNN using the minimum number of edge insertions.

We study the computational complexity of both problems. We show that both the UNN Extension and UNN Completion problems are NP-complete. We further establish that neither problem admits a polynomial-time approximation algorithm with a factor better than $\mathcal{O}(\log n)$ unless $\mathcal{P} = \mathcal{NP}$. On the algorithmic side, we present an $\mathcal{O}(\log n)$ -approximation algorithm for the UNN Extension Problem and a polynomial-time approximation algorithm with linear factor for the UNN Completion Problem. We also study their parameterized complexity, showing that both problems are W[2]-hard when parameterized by solution size, while the UNN Extension Problem becomes tractable when parameterized by treewidth.

Overall, this thesis develops the algorithmic foundations of UNN generation and maintenance under structural constraints, providing tight complexity bounds along with approximation and parameterized results for both problems.

Chapter 1

Introduction

Unique Neighborhood Networks (UNNs) are a class of graphs first introduced by Rass [1]. They have attracted significant interest due to their applications in computer networking and cryptography. Implementing any UNN-based protocol requires the ability to efficiently generate UNNs with specific desirable properties. Although previous work has explored methods for UNN generation [1], these approaches are not directly applicable to real-world protocol construction. In this thesis, we address the problem of generating UNNs in a manner suitable for practical protocol design.

This chapter establishes the fundamental terms and notations used throughout the thesis, formally defines UNNs, and surveys existing generation techniques along with their limitations. It concludes with an overview of our contributions and the organization of the thesis.

1.1 Graphs and Notations

A *graph* G is an ordered pair of sets $(V(G), E(G))$, where $V(G)$ is the *vertex set* and $E(G)$ is the *edge set*. Members of $V(G)$ are called *vertices*, and members of $E(G)$ are called *edges*. Each edge corresponds to either an ordered or unordered pair of (not necessarily distinct) vertices.

If edges correspond to unordered pairs, G is called an *undirected graph*. If edges correspond to ordered pairs, G is called a *directed graph* or *digraph*. In an undirected graph, if an edge e connects vertices u and v , we say that u and v are *adjacent*, and that e is *incident* to u and v (or that e *joins* u and v).

If two distinct edges $e, e' \in E(G)$ connect the same pair of vertices, the graph contains *multi-edges* and is called a *multi-graph*. An edge connecting a vertex to itself is called a *self-loop*. A simple graph is an undirected graph with no multi-edges or self-loops. In this thesis, unless stated otherwise, all graphs are assumed to be simple.

In a simple graph, an edge joining vertices u and v can be represented as (u, v) or (v, u) ; both notations correspond to the same edge.

The *neighborhood* (or *open neighborhood*) of a vertex $v \in V(G)$ is the set of vertices adjacent to v , denoted by $N_G(v)$. Vertices in $N_G(v)$ are called the *neighbors* of v . A vertex with no neighbors is called *isolated*. The *closed neighborhood* of v is $N_G[v] = N_G(v) \cup v$.

The *complement* of a simple graph G is denoted $\overline{G}$. It has the same vertex set as G , and two vertices u, v are adjacent in $\overline{G}$ if and only if $u \neq v$ and u, v are not adjacent in G .

1.2 Unique Neighborhood Networks

Despite the name, a *unique neighborhood network* (UNN) is not a graph in which each vertex has a unique neighborhood. Graphs with that property are known as *weak UNNs* [1], or alternatively as *point-determining graphs* [2] or *mating graphs* [3].

A graph G is called a *unique neighborhood network* [1] if the (open) neighborhood of no vertex is a subset of the neighborhood of another vertex.

Some examples of UNNs include:

- Any cycle graph other than C_4 ,
- Any wheel graph other than W_4 ,
- Any complete graph,
- A single isolated vertex, and
- Path graphs of length 1.

Examples of graphs that are **not** UNNs include:

- Any complete bipartite graph $K_{m,n}$ other than $K_{1,1}$,
- Path graphs of length greater than 1,
- Forests except those where all connected components are of size two, and
- Graphs with more than one isolated vertex.

Figure 1.1 illustrates examples of a UNN and a non-UNN graph. In Figure 1.1a, we show a graph G with eight vertices that is a UNN. In this graph, $N_G(u) \not\subseteq N_G(v)$ for any two vertices $u, v \in V(G)$. Conversely, the graph H in Figure 1.1b is **not** a UNN, since $N_H(a) \subseteq N_H(c)$.

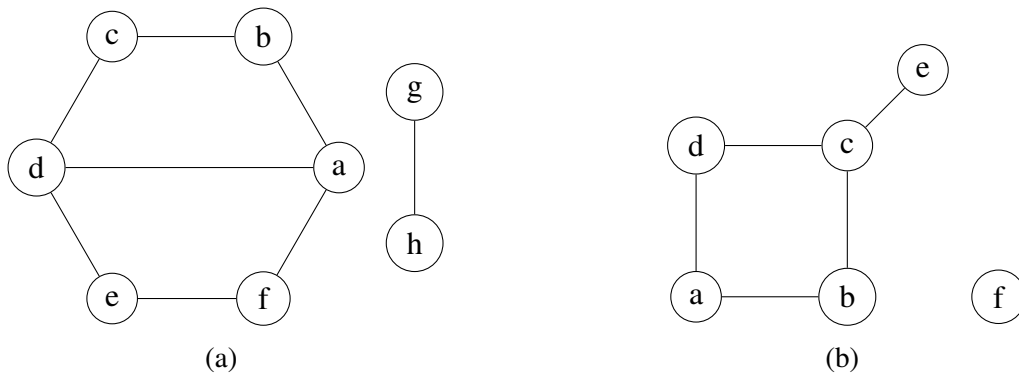


Figure 1.1: Examples of (a) a UNN with eight vertices (b) a graph with six vertices that is not a UNN

1.3 Applications of Unique Neighborhood Networks

UNNs have potential applications in computer networking and cryptography. In this section, we discuss some of their applications.

1.3.1 Topology-Aware Addressing in Networks Without Naming Authorities

In communication networks, each node is typically assigned a unique address to distinguish it from others. This process usually requires a central naming authority. For example, in the Internet, ICANN acts as a central authority, assigning IP address blocks to Regional Internet Registries, which then distribute unique public IP addresses to nodes in their respective regions. For large-scale networks, establishing and maintaining such a central authority can be challenging. UNNs provide an alternative mechanism for node identification that does not rely on a central naming authority.

The key idea is that distinguishing nodes does not necessarily require globally unique identifiers; any property that is unique to each vertex suffices. In a UNN, the neighborhood of each vertex is unique. Therefore, the set of neighbors of a vertex can serve as a distinguishing feature, allowing the vertex to identify itself using its local embedding within the network.

The requirement that the network be a UNN is not restrictive. Many large real-world networks naturally satisfy this property. For instance, in large social networks, it is highly unlikely for two vertices to share exactly the same set of neighbors. Indeed, it has been shown that the probability of a large random network being a UNN approaches one as the network size increases [1], for several stochastic models including the Erdős–Rényi (more precisely, the Erdős–Rényi–Gilbert) model [4, 5] and the random geometric graph model [6].

Consequently, UNN-based topology-aware addressing is a practical and feasible approach for many real-world networks.

1.3.2 Peer Authentication

Usually, secure communication between two nodes in a network is enabled via tools of public key cryptography [7]. Formally, we define a communication protocol to be secure if it has the following properties:

- **Secrecy:** If node u sends a message to node v , no other nodes other than u and v should know the contents of the message (at least not in any significant way).
- **Non-repudiation:** If v receives a valid message that claims to be from u , then v can be certain that the message originated from u .
- **Liveness:** If u sends a message to v , v must receive it after some finite time.

Current known methods of public key cryptography offer computational security in the presence of probabilistic polynomial time adversaries [7] under the assumption that trapdoor functions [8] exist. However, one may consider relaxing these assumptions. UNNs provide us with a method for secure communication in a network without the requirement of trapdoor functions in the presence of threshold adversaries.

Threshold cryptography already provides us with methods to send secret messages over a public channel with information-theoretic guarantees. For example, Shamir's Secret Sharing [9] uses information-theoretic properties of polynomial interpolation to send secret messages over a network in the presence of eavesdropping adversaries as long as the network has enough connectivity and the number of adversaries is limited.

However, an issue with Shamir's Secret Sharing is that it fails in the presence of active adversaries who can flood the receiver with false messages, making it difficult for them to figure out which messages can actually combine to make a secret.

Peer authentication is a method of multi-path authentication that solves exactly this problem. Peer authentication solves the problem of non-repudiation without the requirement of any shared secrets between the sender and the receiver using the properties of UNNs. The idea is that if the underlying network is a UNN, every node is uniquely identified by its neighbors. And instead of requiring each pair of nodes to have a shared secret, each node only has one key, which it shares with its neighbors.

This way, upon receiving a message, the receiver can look up the neighbors of the neighbors of the supposed sender from a well-known directory, and query them on whether the hash of the message is actually signed by the sender. Since the hash does not leak any information about the original message, this prevents any leakage of information while validating the original sender.

The idea of the protocol is as follows:

- Bob initially has k neighbors, which is known to all other nodes. Furthermore, the keys of the neighbors $s_1, s_2, \dots, s_k$ are known to Bob.

- When Bob wants to send the message m to Alice, Bob generates k message authentication codes $\text{sig}_1, \text{sig}_2, \dots, \text{sig}_k$ using the k shared keys for the hash value of the message $h(m)$ and then sends d copies of the message $(m, \text{sig}_1, \text{sig}_2, \dots, \text{sig}_k)$ to Alice using d node disjoint paths.
- Upon receiving the message, Alice looks up Bob's neighbors and queries each of them if $(h(m), \text{sig}_i)$ is a valid MAC. This query must be done over a public broadcasting protocol since it does not require any secrecy.
- If all of Bob's neighbors agree that the MACs are valid, then Alice accepts that the message came from Bob, and Bob can not claim otherwise.

The identity of the sender Bob can be verified by the protocol only if Bob is aware of the collection of keys $s_1, s_2, \dots, s_k$. Thus, for this protocol to work, the graph must be a UNN. Furthermore, the network requires d vertex connectivity and a secure communication channel between two neighboring nodes. Then, it will be possible to have authentication in the presence of $\lfloor \frac{d}{2} \rfloor$ adversaries.

This protocol, when used in unison with a threshold-based protocol, achieves non-repudiation along with secrecy and liveness only using cryptographic hash functions and private key cryptography, side-stepping the necessity of trapdoor functions and public key cryptography.

1.4 Current Known Methods for Generating UNNs and Their Limitations

The generation methods for UNNs have not been studied directly, but rather it has been shown that random graphs generated under the Erdős–Rényi model [4, 5] and the random geometric graph model [6] are almost always UNNs if the graphs are large enough [10]. This provides us with a natural method for generating UNNs via the generation of large random graphs.

However, for practical applications, such graphs are often not suitable because of their limitations. One obvious drawback is that such generation models are only usable when we are generating large graphs. For graphs of small sizes, the probability that a random graph is a UNN may be small. But the issue of most concern is that generating UNNs with no other constraints upon them just does not properly model real-world scenarios. In practical applications, we will not be generating an arbitrary static graph, but rather the graph will evolve with time with insertions and deletions of vertices (to model scenarios of nodes either joining or leaving the protocol) and deletions of edges (to model scenarios like a link being severed due to a fault or failure), and we will be required to patch the graph to restore its UNN property after every change with minimal cost.

Thus, we are not interested in generating arbitrary UNNs, but rather UNNs that are easy to extend (to model vertex additions) or easy to repair (to model vertex and edge deletions), which the currently known generation methods cannot do.

In this thesis, we try to overcome these limitations by proposing two new combinatorial optimization problems: the *UNN Extension Problem* and the *UNN Completion Problem*, which better model the problem of generating UNNs for practical applications.

1.5 The UNN Extension Problem

The UNN Extension Problem is aimed at modeling the problem of extending an existing graph by a single vertex. Consider the scenario where a new node wants to join a protocol based on some underlying UNN. Then we would have to accommodate adding a new vertex to the UNN. To preserve the UNN property, we would need to add zero or more edges between the new vertex and the previously existing vertices. Since edges of the underlying graph may represent some expensive resource like physical links, links in a quantum network, or private keys, addition of edges may be expensive, and therefore we would want to use the minimum number of edge insertions as possible. The UNN extension problem aims to minimize the number of such edge insertions.

Formally, we call a graph H a k -extension of a graph G if G is an induced subgraph of H and $|V(H)| = |V(G)| + k$. We can think of a k -extension of G as a graph resulting from G by adding k new vertices and inserting zero or more edges incident on the newly added vertices. The *cost of a k -extension H of a graph G* is defined as the number of edge insertions used to generate H from G , i.e., $|E(H)| - |E(G)|$.

In the case where k is one, we simply call it an *extension*, the newly added vertex the *extension vertex*, and the neighborhood of the extension vertex as the *extension neighborhood*.

In Figures 1.2a and 1.2b we show a graph G and an extension (or 1-extension) H of G with the extension vertex v (in green). The extension neighborhood of H is $N_H(v) = \{a, c, e\}$ and is shown in blue. The cost of the extension is $|N_H(v)| = 3$. In Figure 1.2c, we show a graph H' that is not an extension of G . This is because G is not an induced subgraph of H' since (b, d) is adjacent in H' but not in G . Figure 1.2d shows an example of a 2-extension of G of cost four.

We call an extension H of a UNN G a *UNN extension* if H is also a UNN. It is trivial to see that every UNN has a UNN extension.

In Figure 1.3 we show a UNN G and two extensions G_1 and G_2 of G in Figures 1.3b and 1.3c respectively. G_1 is a UNN extension of G whereas G_2 is not a UNN extension since G_2 is not a UNN ($N_{G_2}(v) \not\subseteq N_{G_2}(a)$).

Given a UNN G , the UNN extension problem seeks a UNN extension with minimum cost (i.e., the smallest extension neighborhood).

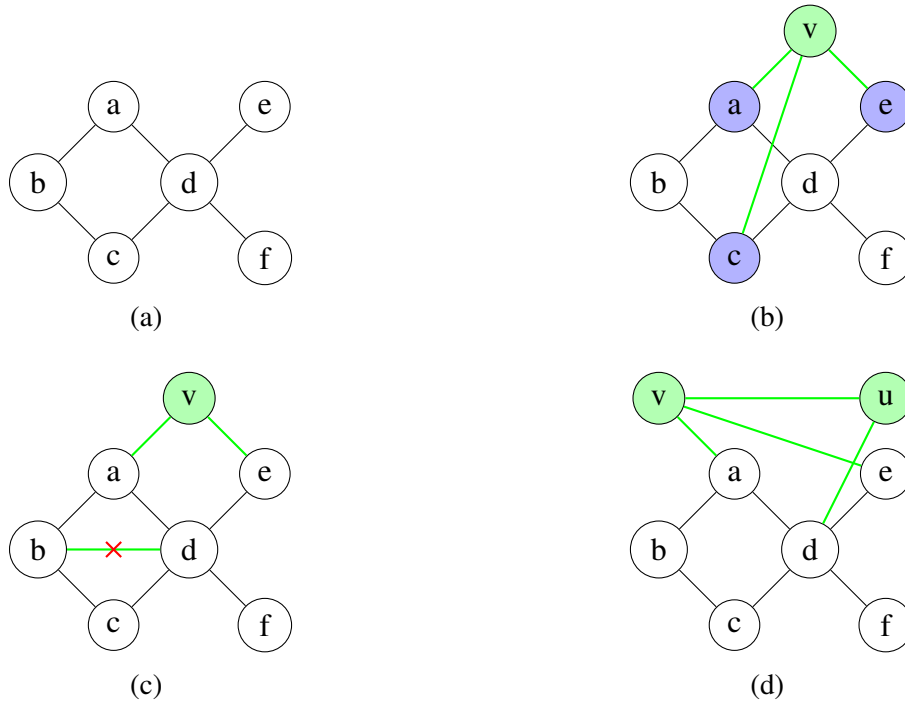


Figure 1.2: (a) A graph G (b) an extension H of G of cost three (c) a non-extension of G (d) a 2-extension of G

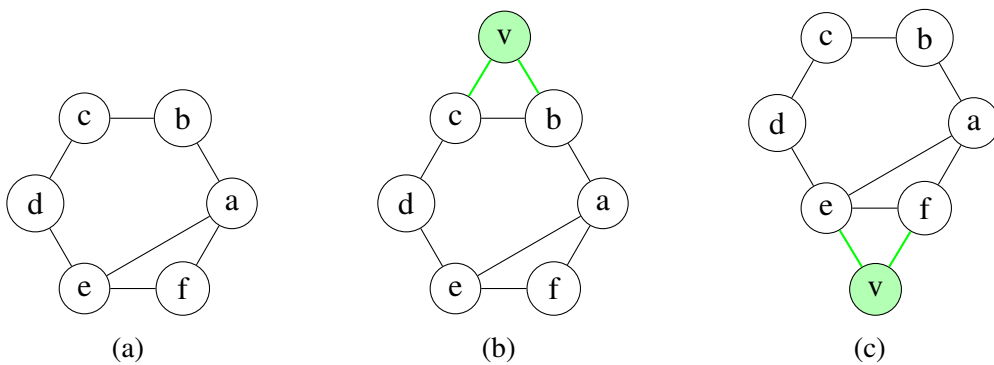


Figure 1.3: (a) a UNN G . (b) a UNN extension of G with an extension vertex v . (c) An extension of G that is not a UNN extension.

1.6 The UNN Completion Problem

The UNN Completion Problem seeks to model the task of transforming a general graph into a UNN through the insertion of edges. This type of problem falls within a broader class of graph-theoretic questions known as minimum edge fill problems, or more generally, as Π -Completion problems. Such problems are central to the study of graph classes, with notable examples including chordal graphs [11, 12], interval graphs [11, 13, 14], Hamiltonian graphs, split graphs [15], co-chordal graphs [15], and perfect graphs [15]. For each of these classes, it has been established that solving the corresponding minimum edge fill problem is NP-hard. A comprehensive overview of various edge completion problems is provided in [15].

Consider a scenario in which a UNN-based protocol has been operating for some time, but due to faults or failures, a significant number of network links become severed. To restore normal operation, it becomes necessary to repair the network by inserting new edges so that the UNN property is reestablished. In such cases, it is desirable to achieve this restoration with minimal cost, specifically. The UNN Completion Problem is formulated precisely to address this challenge.

Formally, given a graph G , we define a graph H to be a *UNN completion* of G if H is obtained from G by adding zero or more edges, and H satisfies the UNN property. The *cost of a UNN completion* is defined as the number of edge insertions required to obtain the UNN completion from the input graph, i.e., $|E(H)| - |E(G)|$. Given any graph G , the *UNN Completion Problem* seeks to find a UNN completion of G with minimum cost.

1.7 Our Contribution

In this thesis, we study the problem of generating UNNs that are extensible and repairable by studying the UNN extension and the UNN completion problem.

We have studied both the decision and optimization variants of the problem and have shown both problems to be $\mathcal{NP}$ -complete. To cope with hardness, we have tried to tackle the problems via approximations and parameterization.

We have shown that neither of the problems can be approximated to a factor better than $\mathcal{O}(\lg n)$ in polytime unless $\mathcal{P} = \mathcal{NP}$. For the UNN extension problem, we have come up with a $\mathcal{O}(\lg n)$ polynomial time approximation algorithm, i.e., a polytime approximation algorithm with asymptotically tight approximation ratio (under the assumption that $\mathcal{P} \neq \mathcal{NP}$) and for the UNN completion problem, we have established a polynomial time linear approximation algorithm.

When studying the problems from the lens of parameterized algorithms, we have shown that both problems are $W[2]$ -hard when parameterized by their cost. However, the UNN extension problem can be solved in linear time when parameterized by its treewidth.

1.8 Thesis Organization

This thesis is divided into nine separate chapters. Chapter 2 introduces basic concepts and ideas used throughout this thesis. Chapter 3 provides us with a polynomial time $\mathcal{O}(\lg n)$ approximation of the UNN extension problem. On the other hand, Chapter 4 shows that this approximation ratio is tight, i.e., unless $\mathbf{P} = \mathbf{NP}$, there is no polynomial time approximation algorithm for the UNN extension problem with an approximation ratio in $o(\lg n)$.

Chapters 5 and 6 shift their focus to the UNN completion problem. Chapter 5 shows that the UNN completion problem is $\log\text{-}\mathbf{APX}$ -hard. On the other hand, Chapter 6 provides us with a polynomial time $\mathcal{O}(|V| + |E|)$ approximation for the UNN completion problem.

Chapter 7 studies the UNN extension and the UNN completion problem from the perspective of parameterized algorithms and shows that while both the UNN extension and the UNN completion problem are $\mathbf{W}[2]$ -hard when parameterized by solution size, the UNN extension problem becomes easy when parameterized by treewidth.

Chapter 8 shows some miscellaneous results about UNNs.

Finally, Chapter 9 concludes this thesis by summarizing our results and exploring some open problems.

Chapter 2

Preliminaries

This chapter goes over common notations and introduces ideas that will be used throughout this thesis.

2.1 Decision And Optimization Problems

If the output of every instance of a problem can be expressed as a single bit (i.e., either 0 or 1), the problem is called a *decision problem*. Formally, a decision problem is a language $L \subseteq \Sigma^*$, where an instance $x \in \Sigma^*$ is a *yes*-instance if and only if $x \in L$.

Given x , we call the task of minimizing (or maximizing) some function $f(x, \cdot)$ an *optimization problem*. We call the function f the objective function for the optimization problem and x the instance of the problem.

A very closely variant of the optimization problem is the *optimizing search problem*, where instead of finding the minimum value of $f(x, y)$ over all feasible y , we try to find a y such that $f(x, y)$ attains the minimum value. It is not difficult to see that the regular variant of the optimization problem (optimizing the objective function) reduces to the optimizing search problem.

Furthermore, optimizing search problems may in fact be harder. Consider the optimization problem that seeks to optimize the following objective function:

$$g(x, y) = \begin{cases} 0 & \text{if } x \text{ is composite and } y \text{ is the largest non trivial prime factor of } x \\ 1 & \text{otherwise} \end{cases}$$

Then the regular variant of the optimization problem reduces to primality testing of integers, which is known to be in $\mathcal{P}$, whereas the search variant of this problem is as hard as integer factorization, for which there is no known polynomial time algorithm.

Throughout this thesis, when we mention an optimization problem, we will be talking about the

Name	Abbreviation
UNN Extension Problem	UE
Dominating Set Problem	DS
Total Dominating Set Problem	TDS
Good Dominating Set Problem	GDS

Table 2.1: Abbreviation Of Optimization Problem Names

search variant of the problem unless mentioned otherwise. However, the hardness results shown in this thesis will also apply to regular optimization variants of the problems as well.

2.2 Approximating Optimization Problems

Sometimes it is difficult to find a solution to an optimization problem that minimizes (or maximizes) the objective function, and thus we need to be content with finding a good enough solution for which the objective function yields a small enough (or large enough) value. When trying to find such approximate solutions, we want to have mathematical guarantees, which is what the field of *Approximation Algorithms* does. Approximation algorithms provide us with approximate solutions to optimization problems with guaranteed bounds on the ratio between the optimal value of the objective function and the value of the objective function when evaluated at the solution.

Let A be an optimization problem. We will denote the value of the objective function for a solution s to an instance x of problem A as $\text{cost}_A(x, s)$ and call it *cost of the solution s* . We will denote the cost of an optimal solution to an instance x of problem A as $\text{OPT}_A(x)$. We define the *approximation ratio* of a solution s to an input instance x of the problem A as the ratio $\frac{\text{cost}_A(x, s)}{\text{OPT}_A(x)}$ for minimization problems and $\frac{\text{OPT}_A(x)}{\text{cost}_A(x, s)}$ for maximization problems. In both cases, we will denote the approximation ratio with $R_A(x, s)$. In either case, $R_A(x, s) \geq 1$ by definition.

Throughout this thesis, we will consider multiple optimization problems, most of them with lengthy names. Thus, we will use the abbreviation of the problem names as the subscript to distinguish between the OPT , score, and R functions for different problems.

Table 2.1 lists the names of the problems we consider and their abbreviations.

Also note that throughout this thesis, we will only consider minimization problems and will often use the terms cost and score interchangeably.

2.3 Reductions and Approximating Preserving Reductions

When studying computational problems, we are often interested in whether a problem A can be solved given access to a black-box (or oracle) for another problem B . An algorithm that solves A using an oracle for B is called a *reduction* from A to B .

Given an instance x of A , a reduction may query an oracle on any number of instances of B , and use the returned answers to produce a solution for x . However, in this thesis, all reductions we present query the provided oracle exactly once.

When reductions have this additional structure, they can equivalently be represented by a pair of computable functions:

- An *instance mapping* f that maps an instance x of A to an instance $y = f(x)$ of B .
- A *solution mapping* g that maps a solution s_y of $f(x)$ to a solution $s_x = g(x, s_y)$ of x .

This alternate view is often convenient for algorithm design and analysis. In this thesis, reductions are presented as algorithms, but proofs will frequently refer to the instance and solution mappings, as their properties are central to correctness and approximation guarantees.

Not all reductions are useful for all purposes. To be meaningful, a reduction typically satisfies additional properties, such as f and g being computable in polynomial time with respect to their inputs.

When considering optimization problems, we are particularly interested in reductions that *preserve approximation*. In this thesis, we make use of a type of *approximation-preserving reduction* called a *Strict Reduction*, defined as follows:

Definition 2.3.1. Let A and B be two minimization problems. A reduction from A to B with instance mapping f and solution mapping g is a *Strict Reduction* if the following conditions hold:

- Polynomial Time Computability:** f is polynomial-time computable with respect to the size of its input, and g is polynomial-time computable with respect to the size of the instance of A it takes as input.
- Preservation of Approximation Ratio:** For any instance x of A and any solution s_y of $f(x)$, we have

$$R_A(x, g(x, s_y)) \leq R_B(f(x), s_y).$$

It follows that if there exists a Strict Reduction from A to B , then inapproximability results for B carry over to A : if no polynomial-time algorithm can approximate B within a factor of $o(f(n))$, then no polynomial-time algorithm can achieve the same approximation ratio for A .

2.4 Dominating Set and Total Dominating Set

The *dominating set* of a graph $G = (V, E)$ is a set of vertices $D \subseteq V$ such that for every vertex $v \in V$, either v or a neighbor of v belongs to D . The size of a minimum dominating set of G

is referred to as its *domination number* $\gamma(G)$. The *total dominating set* of G is a set of vertices $D \subseteq V$ such that every vertex $v \in V$ is adjacent to some vertex in D . Note that a dominating set may contain a vertex that is not adjacent to any other vertices in the set, but a total dominating set cannot contain such a vertex. It is known that every graph without any isolated vertex has a total dominating set. The size of a minimum total dominating set of a graph G is referred to as its *total domination number* $\gamma_t(G)$. For graphs with isolated vertices, their total domination number is defined as infinity.

The *minimum dominating set problem* and the *minimum total dominating set problem* focus on finding a dominating set and a total dominating set of minimum size, respectively. Both these problems are known to be $\mathcal{NP}$ -hard [11, 16].

There exists a polynomial-time $O(\log n)$ -approximation algorithm for the minimum dominating set problem [17] and no polynomial-time algorithm can have an approximation ratio of $o(\log n)$ unless $\mathcal{P} = \mathcal{NP}$ [18, 19]. Since the total domination number of a graph is at most twice its domination number in graphs without any isolated vertices [20], the inapproximability result applies also to the minimum total dominating set.

2.5 Parameterizing Hard Problems And Fixed Parameter Tractability

In the field of computational complexity, a problem is considered hard if there are no polynomial time algorithms to find exact solutions to the problem. One way to cope with hard problems is to go beyond conventional analysis, where the complexity is represented only as a function of the input size, and instead represent it as a function of the input size and some other reasonable parameters of the input instance. Such analysis is known as parameterized analysis and is studied in the field of *parameterized algorithms*.

The idea behind parameterized analysis is that a problem may be intractable in polynomial time with respect to its input size, but its hardness may be an artifact of some parameter of the input being large instead of the input size. Doing parameterized analysis will allow us to find algorithms that are efficient when certain parameters of the input are small.

We can parameterize a decision problem L with some d dimensional parameter p by defining the problem of deciding whether some $x \in L$ given a pair $(x, k) \in \{0, 1\}^* \times \mathbb{N}^d$ where k is a value of the parameter p for x . Similarly, we can define a parameterized version of an optimization problem as finding the optimal value given a function $f(x, y)$ over y given x and a value k for the parameter.

However, it is important to notice that the parameter for our problem can be anything we decide it to be. For example, if we wanted, we could parameterize the vertex cover problem with any other parameter of our choice, for example: the maximum degree of the graph, the treewidth [21]

of the graph, or the pathwidth [21] of the graph. For example, when parameterizing the decision version of the vertex cover problem as an input, we will be provided a graph G and integer k and the treewidth t of G and we have to decide whether G contains a vertex cover of size at most k . For the optimization variant of the problem, we will be provided only the graph G and its treewidth t , and we will have to find a minimum vertex cover of G . The choice of parameter is completely up to the person who is studying the problem, and different parameters lead to different parameterized problems.

Though one can choose any parameter to parameterize a problem, parameterized analysis is only useful when being done with reasonable and meaningful parameters. For example, every $\mathcal{NP}$ -hard problem can be solved in polynomial time of the size of the input if we parameterize the problem on the size of the certificate to the problem. But this choice of parameter is pathological and does not offer any new insight.

2.5.1 Fixed Parameter Tractability

Just as polynomial time is regarded as the standard measure of efficiency in classical algorithmic analysis, fixed-parameter tractability serves as the corresponding notion of efficiency in the study of parameterized algorithms.

Formally, let (x, k) be an input instance to some parameterized problem where k is the parameter, and let $\mathcal{A}$ be an algorithm for this parameterized problem. If $\mathcal{A}$ can solve the instance (x, k) in a running time bounded by $f(k)p(|x|)$ where $f(k)$ is any computable function and p is a polynomial, we call the algorithm $\mathcal{A}$ to be *fixed parameter tractable* or *FPT* for short.

For example, the algorithm mentioned earlier for the vertex cover problem that runs in $\mathcal{O}^*(1.25284^k)$ is fixed parameter tractable when considering the size of the vertex cover as the parameter, while the algorithm that ran in $\mathcal{O}^*(1.2125^n)$ is not.

Not all parameterized problems have an FPT algorithm. For example, the clique problem parameterized by clique number is believed to have no FPT algorithms [22, 23]. However, parameterizing the same problem with maximum vertex degree provides us with an FPT algorithm running in $\mathcal{O}(2^\Delta \Delta^{2n})$ time [21].

2.6 FPT reduction and W-Hierarchy

Similar to how $\mathcal{NP}$ -hardness provides us with a notion of computational intractability, in parameterized algorithms, there is also a notion of computational hardness. However, instead of having only one complexity class of hard problems like in the case of polytime tractability, we have an infinite hierarchy of classes, known as the W -hierarchy.

Continuing our analogy with $\mathcal{NP}$ -completeness, the W -hierarchy is defined with the help of a special type of reduction known as an *FPT reduction*.

Definition 2.6.1. Let A and B be two parameterized decision problems. We call an algorithm $\mathcal{A}$ an FPT reduction from A to B if it satisfies the following conditions:

- (a) **Completeness:** For any input instance (x, k) for problem A , if $(x, k) \in A$ then, $\mathcal{A}(x, k) \in B$.
- (b) **Soundness:** For any input instance (x, k) for problem A , if $\mathcal{A}(x, k) \in B$ then, $(x, k) \in A$.
- (c) **FPT Time Computability:** For any input instance (x, k) for problem A , $\mathcal{A}$ runs in FPT time i.e., there is a computable function f and a polynomial p such that the running time of $\mathcal{A}$ is bounded above by $f(k)p(|x|)$.
- (d) **Independence of Parameter Size:** If $(x', k') = \mathcal{A}(x, k)$ for some input (x, k) for the problem A , then the value (or norm in the case where k' is multi-dimensional) of k' must be bounded above by some $g(k)$ for some computable function g .

It is not hard to see from the definition that if there is an FPT reduction from problem A to problem B and if problem B has an FPT algorithm, then so does problem A .

We call the class of all parameterized decision problems that can be solved in FPT time **W[0]** (also known as **FPT**). **W[0]** is closed under FPT reduction. **W[0]** represents the class of “easy” problems in parameterized complexity (similar to the role of **P** in classical complexity theory). The complexity classes **W[1]**, **W[2]**, $\dots$ is defined using reductive closure from special types of min-weight circuit satisfiability problem.

Formally, A *boolean circuit* is defined as a directed acyclic graph with exactly one sink and one or more sources. The source vertices of the graph are called inputs of the boolean circuit. Every other vertex other than the source vertices are known as boolean gates and can be of three types: an OR-gate, an AND-gate, or a NOT-gate. Vertices of type NOT-gate must have an in-degree of one, whereas the other gates can have an in-degree larger than one.

An assignment for a boolean circuit G is a function $f : X \rightarrow \{T, F\}$ where X is the set of inputs in G . We define the value $g(v, f)$ of a vertex $v \in V(G)$ of a boolean circuit G and assignment f as follows:

$$g(v, f) = \begin{cases} f(v) & \text{if } v \text{ is an input vertex,} \\ \neg g(u, f) & \text{if } v \text{ is a NOT-gate and } u \text{ is its in-neighbor,} \\ \bigvee_{u \in V(G) \wedge (u,v) \in E(G)} g(u, f) & \text{if } v \text{ is an OR-gate,} \\ \bigwedge_{u \in V(G) \wedge (u,v) \in E(G)} g(u, f) & \text{if } v \text{ is an AND-gate,} \end{cases}$$

The value of the circuit for the assignment f is defined as the value of the sink of the circuit of the assignment. An assignment is called satisfying for a circuit G if the value of G for the assignment evaluates to true.

The min-weight circuit satisfiability problem aims to find the minimum number of variables required to be set to true in a satisfying assignment. Formally, the input to a weighted circuit satisfiability problem is a circuit G and an integer k , and we have to decide whether the circuit is satisfiable with an assignment with at most k inputs set to true. Weighted circuit satisfiability problem when parameterized by k is known to be k -weighted-circuit-SAT and believed to be not in FPT.

The classes $\mathbf{W}[1]$, $\mathbf{W}[2]$, $\dots$ are defined as closures under FPT reduction from k -weighted-circuit-SAT on restricted classes of graphs.

We define the *depth of a boolean circuit* as the longest path in the DAG and *weft of boolean circuit* as the maximum number of vertices with in-degree higher than two in a path. We denote the problem of solving the k -weighted-circuit-SAT problem when restricted to circuits with depth at most d and weft at most t as $WCS[\mathcal{C}_{t,d}]$.

The complexity class $\mathbf{W}[t]$ is defined as the set of all parameterized decision problems that FPT reduces to $WCS[\mathcal{C}_{t,d}]$ for some $d \geq 1$. It is not difficult to see that:

$$\mathbf{W}[0] \subseteq \mathbf{W}[1] \subseteq \mathbf{W}[2] \subseteq \dots$$

However, it is not known whether there exists some $i \neq j$ such that $\mathbf{W}[i] = \mathbf{W}[j]$.

We call a problem $\mathbf{W}[t]$ -hard if the problem FPT reduces to every other problem in $\mathbf{W}[t]$. If a problem is both in $\mathbf{W}[t]$ and $\mathbf{W}[t]$, then it is said to be $\mathbf{W}[t]$ -complete.

In this thesis, we will show that both the UNN extension problem and the UNN Completion problem are $\mathbf{W}[t]$ -complete.

2.7 Treewidth and Courcelle's Theorem

A very well studied parameter for parameterized graph problems is the *treewidth* of a graph, which measures how tree-like a graph is.

Formally, we call a *tree decomposition* $\mathcal{T}$ of a graph G as the pair $(T, \{X_t\}_{t \in V(T)})$ where T is a tree that has a set $X_t \subseteq V(G)$ associated with every vertex $t \in V(T)$, called the bag of t , and that satisfies the following conditions:

1. $\bigcup_{t \in V(T)} X_t = V(G)$
2. For every edge $(u, v) \in E(G)$, there exists a vertex t in T such that $u, v \in X_t$
3. For every $u \in V(G)$, the subgraph induced by the vertices $\{t \in V(T) \mid u \in X_t\}$ is connected

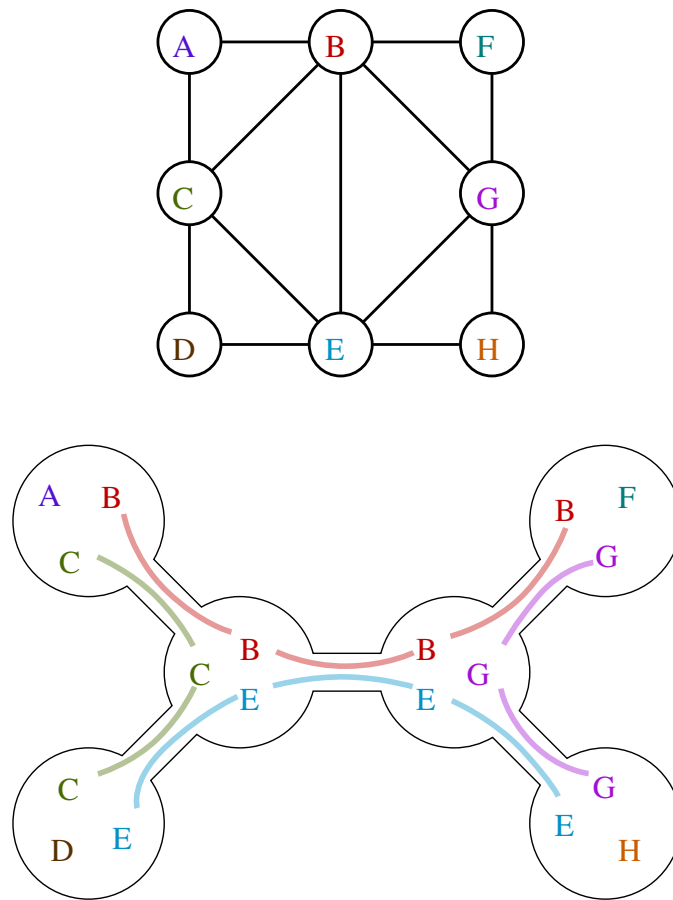


Figure 2.1: A graph with eight vertices, and a tree decomposition of it onto a tree with six nodes. Each graph edge connects two vertices that are listed together at some tree node, and each graph vertex is listed at the nodes of a contiguous subtree of the tree. Each tree node lists at most three vertices, so the width of this decomposition is two.

We define the width of a tree decomposition $\mathcal{T} = (T, \{X_t\}_{t \in V(T)})$ as $\max_{t \in T} |X_t| - 1$. The treewidth of a graph G is defined as the width of a tree decomposition of G with minimum width. In Figure 2.1, we see a graph G and a tree decomposition of G with width two.

The reason why treewidth is such a well-regarded parameter for graph problems is because of a powerful result known as *Courcelle's theorem*. Roughly speaking, it states that any property of graphs that can be expressed in a sufficiently rich logical language (monadic second-order logic) can be decided efficiently on graphs of bounded treewidth.

2.7.1 Monadic Second Order Logic

To understand Courcelle's theorem, we need to understand the logic in which graph properties are expressed. *Monadic second-order logic* is a class of formal language. It is a special type of second-order language. Zeroth-order logic, also known as propositional logic, is the simplest type of logic that allows combining logical variables with logical operators.

First order logic is a more powerful class of logical languages that allows variables to get value from a special set instead of having just the values *True* and *False*. Furthermore, first order logic allows for universal quantification of variables over the said set. In addition, first-order languages have access to domain-specific function and relation symbols to express complex mathematical ideas.

For example, in a first-order language for arithmetic where the $+$, $\cdot$, and $=$ symbols are allowed, we may construct a formula as follows to express the statement that z is a prime number:

$$\forall x \forall y \ x \cdot y = z \implies x = 1 \wedge y \neq 1 \vee x \neq 1 \wedge y = 1$$

While first-order logic allows quantification over values over a set, it does not allow for quantification over all relations of some fixed arity of the domain.

Second order logic is an extension of first-order logic that has additional variable symbols to encode relations of different arity over the domain of discourse and allows universal quantification over such relations.

For example, the following first-order statement is true for all relations P of arity 1:

$$\exists x (Px \rightarrow \forall x Px)$$

But the statement $\forall P \exists x (Px \rightarrow \forall x Px)$ is not a valid first-order statement because in first-order logic, there is no predicate variable P nor can you use universal quantification over them. But the statement “all arity 1 relation P satisfies the statement $\exists x (Px \rightarrow \forall x Px)$ ” can be expressed in second-order logic:

$$\forall P_1^1 \exists x (P_1^1 x \rightarrow \forall x P_1^1 x)$$

Here, P_1^1 is a predicate variable of arity 1.

Monadic second-order logic is a restriction of second-order logic where universal quantification can only be done over predicates or relations of arity 1 or monadic predicates.

In the context of Courcelle's theorem, we use a monadic second-order language to express statements about graphs using the following rules:

- There are two kinds of variables: vertex variables and vertex set variables. Universal quantification is allowed over all types of variables.
- There are relation symbols: $\text{adj}(\cdot, \cdot)$ and $=$. $=$ encodes equality whereas $\text{adj}(\cdot, \cdot)$ encodes the vertex adjacency relation.

This language is also known as MSO_1 for short. For example, we may use the following MSO_1 formula to encode the statement that D is a dominating set:

$$\forall_{v \in V} v \in D \vee \exists_{u \in D} \text{adj}(u, v)$$

However, there is a stronger variant of Courcelle's theorem that uses a stronger variant of monadic second-order logic known as MSO_2 . In MSO_2 , we can also use edge variables and edge set variables, as well as quantify over them. Furthermore, MSO_2 introduces the relation symbol $\text{inc}(\cdot, \cdot)$ which encodes whether a vertex is incident on an edge. For example, the following formula, which states that G is hamiltonian, is expressible in MSO_2 but not MSO_1

$$\begin{aligned} \exists_{F \subseteq E} \big(& \forall_{v \in V} \exists_{e_1, e_2 \in F} \left(e_1 \neq e_2 \wedge \text{inc}(v, e_1) \wedge \text{inc}(v, e_2) \right. \\ & \wedge \forall_{e \in F} \left(\text{inc}(v, e) \rightarrow (e = e_1 \vee e = e_2) \right) \big) \\ & \wedge \forall_{U \subseteq V} \left(U \neq \emptyset \wedge U \neq V \rightarrow \exists_{u \in U} \exists_{v \in V \setminus U} \exists_{e \in F} \right. \\ & \quad \left. \left(\text{inc}(u, e) \wedge \text{inc}(v, e) \right) \right) \big) \end{aligned}$$

2.7.2 Courcelle's Theorem

The classical version of Courcelle's theorem is for parameterized decision problems of graphs. It states that graph properties that can be expressed as MSO_2 formulas can be tested in linear time when parameterized by treewidth and the formula length.

Lemma 1 (Courcelle's theorem). *Let ϕ be a formula in MSO_2 and G be an n vertex graph equipped with evaluation of all the free variables of ϕ . Suppose, moreover, that a tree decomposition of G of width t is provided. Then there exists an algorithm that verifies whether ϕ is satisfied in G in time $f(|\phi|, t)n$, for some computable function f . [24]*

However, there is a stronger variant of Courcelle's theorem that applies to optimization problems. It states that any optimization problem relating to graphs that aims to maximize or minimize some affine function relating to the sizes of some sets of vertices that satisfies some MSO_2 formula can also be solved in linear time when parameterized by the treewidth and the size of the formula.

Lemma 2 (Optimization variant of Courcelle's Theorem). *Let ϕ be an MSO_2 formula with p free monadic variables $X_1, X_2, \dots, X_p$, and let $\alpha(x_1, x_2, \dots, x_p)$ be an affine function. Assume that we are given an n -vertex graph G together with its tree decomposition of width t , and suppose G is equipped with evaluation of all the free variables of ϕ apart from $X_1, X_2, \dots, X_p$. Then there exists an algorithm that in $f(\|\phi\|, t)n$ finds the minimum and maximum value of $\alpha(|X_1|, |X_2|, \dots, |X_p|)$ for sets $X_1, X_2, \dots, X_p$ for which $\phi(X_1, X_2, \dots, X_p)$ is true, where f is some computable function. [25]*

In this thesis, we will use Courcelle's theorem to establish a linear time algorithm for the UNN extension problem for bounded treewidth graphs.

Chapter 3

An Approximation Algorithm for UNN Extension Problem

In this chapter, we provide an approximation algorithm for the UNN Extension Problem. Before we go on to show the algorithm, we will first introduce the idea of suppressors and suppressed vertices in a graph and then use those ideas to show how finding a UNN extension of a UNN is equivalent to finding a special type of dominating set in the complement graph. Finally, we will provide a $\mathcal{O}(\lg n)$ approximation algorithm for the UNN Extension Problem that constructs an extension by finding such a dominating set of a small enough size using approximate solutions of the dominating set problem. This result provides an upper bound on the approximability of the UNN Extension problem, which will later be shown to be essentially tight in Chapter 4.

3.1 Concept of Suppressors and Suppressed Nodes

Let v, w be two vertices in a graph G . We say that v *suppresses* w (equivalently, w is suppressed by v) if $N_G(w) \subseteq N_G(v)$, and denote it by $w \preceq_G v$ or $v \succeq_G w$. Note that every vertex v , both suppresses and is suppressed by itself at the same time.

We will use the notation $w \not\preceq_G v$ or $v \not\succeq_G w$ to denote that w is not suppressed by v . For any two vertices w and v if $w \not\preceq_G v$ and $v \not\succeq_G w$, we say that w and v are *incomparable* in G and will be denoted by $w|_G v$.

We call a vertex v in G a *suppressor* in G if v suppresses some vertex in G other than itself. Similarly, v is called a *suppressed vertex* in G if v is suppressed in G by some other vertex other than itself. In Figure 1.1b, c suppresses a, b and d suppress each other, both b and d suppress e and every vertex suppresses itself and f . Every vertex other than e and f is a suppressor in Figure 1.1b and every vertex other than c is a suppressed vertex.

The proof of the following lemma follows from the definition.

Lemma 3. *The following statements are equivalent.*

- (a) G is a UNN.
- (b) No two distinct vertices in G are comparable.
- (c) No vertex v in $V(G)$ is a suppressor of any other vertex in G .
- (d) No vertex v in $V(G)$ is suppressed by any other vertex in G .

We now observe some properties of suppressors and suppressed vertices.

remark 1. *Let u, v be two adjacent vertices in a graph G . Then $u|_G v$.*

Proof. Since $v \in N_G(u)$ but $v \notin N_G(v)$, $N_G(u) \not\subseteq N_G(v)$. Similarly, $N_G(v) \not\subseteq N_G(u)$. Thus, $u|_G v$. $\square$

remark 2. *Let G be an arbitrary graph and let H be an induced subgraph of G . If $u \not\leq_H v$ for some vertices $u, v \in V(H)$, then $u \not\leq_G v$. Similarly, if $u|_H v$ holds, then we must have $u|_G v$.*

Proof. If $u \not\leq_H v$, then there exists a vertex $u' \in V(H)$ such that u' is adjacent to u but not v in H . Since H is an induced subgraph, u' and v are not adjacent in G as well. Therefore, u cannot be suppressed by v in G . Similarly, if $u|_H v$ holds, then we have $u \not\leq_G v$ and $v \not\leq_G u$ and thus $u|_G v$. $\square$

By $G - \{v\}$ we denote a graph obtained from G by deleting the vertex v from G .

Lemma 4. *Let G be a graph and let v be a vertex in G . If $N_G(v)$ is a dominating set in $\overline{G - \{v\}}$ then v is not suppressed by any other vertex in G , and vice versa.*

Proof. Assume first that v is not suppressed by any other vertex in G . We now show that $N_G(v)$ must be a dominating set in $\overline{G - \{v\}}$. Consider an arbitrary vertex $u \in V(G - \{v\})$. If u and v are adjacent in G , then u belongs to $N_G(v)$. If u and v are not adjacent in G , then since v is not suppressed by u , there exists a vertex $v' \in N_G(v)$ that is not adjacent to u in G . Therefore, v' is adjacent to u in $\overline{G - \{v\}}$. Thus, $N_G(v)$ is a dominating set of $\overline{G - \{v\}}$.

We now show that if $N_G(v)$ is a dominating set in $\overline{G - \{v\}}$, then v cannot be suppressed by any other vertex in G . Suppose for a contradiction that a vertex $u (\neq v)$ suppresses v in G . We know that either $u \in N_G(v)$ or a neighbor u' of u in $\overline{G - \{v\}}$ is in $N_G(v)$. If $u \in N_G(v)$ then, u cannot suppress v in G (Remark 1). Similarly, if $u' \in N_G(v)$, then u cannot suppress v in G because u' is adjacent to v but not to u in G . $\square$

Lemma 5. *Let G be a graph and let v be a vertex in G . If every vertex $u \in (V(G) - N_G[v])$ has a neighbor $u' \notin N_G(v)$, then v is not a suppressor in G and vice versa.*

Proof. If every vertex $u \in (V(G) - N_G[v])$ has a neighbor that does not belong to $N_G(v)$, then by definition v cannot suppress any other vertex in G . Assume now that v is not a suppressor and consider an arbitrary vertex $u \in V(G) - N_G[v]$. Since v is not a suppressor, again by definition u has a neighbor $u' \in V(G)$ that is not a neighbor of v . $\square$

3.2 Relating Dominating Set to the UNN Extensions

In this section we show that an extension of a UNN G is a UNN if and only if the neighborhood of the extension vertex forms a special type of dominating set (Lemma 8). This will later be used in subsequent sections to obtain the approximation and hardness results.

Lemma 6. *Let G' be an extension of a UNN G with the extension vertex v^* . Then G' is a UNN extension if and only if the following conditions hold:*

- (a) v^* is not suppressed by any other vertex in G' .
- (b) v^* does not suppress any other vertex in G' .

Proof. If G' is a UNN extension of G , then G' is a UNN and by the definition of UNN, G' cannot contain any suppressor or suppressed vertex. Therefore, v^* neither suppresses nor is suppressed by another vertex in G' .

Assume now that v^* neither suppresses nor is suppressed by another vertex in G' , i.e., for every vertex $u (\neq v^*)$ in G' , u and v^* are incomparable. Every pair of vertices $p (\neq v^*)$ and $q (\neq v^*)$ is incomparable in G and remains so in G' even when we add v^* (Remark 2). Therefore, by Lemma 3, G' is a UNN. $\square$

We can combine Lemmas 4 to 6 to obtain the following lemma.

Lemma 7. *Let G' be an extension of a UNN G with the extension vertex v^* . Then G' is a UNN extension if and only if the following conditions hold:*

- (a) $N_{G'}(v^*)$ is a dominating set in $\overline{G}$.
- (b) Every vertex $u \in V(G) - N_{G'}(v^*)$ has a neighbor $u' \notin N_{G'}(v^*)$ in G .

We refer to a dominating set as a *good dominating set* if it satisfies condition (b) of Lemma 7, which is formalized as follows.

Definition 3.2.1 (Good Dominating Set). A dominating set D of a graph G is good if and only if for every vertex $u \in V(G) - D$, u has at least one neighbor outside of D in $\overline{G}$.

By $\gamma_g(G)$ we refer to the *good domination number*, i.e., the size of a minimum-cardinality good dominating set. Given a dominating set D in G , we call a vertex $v \in V(G) - D$ *bad* if it has no neighbor in $\overline{G}$ outside of D (equivalently $V(G) - D \subseteq N_G[v]$). It is now straightforward to observe that a dominating set in G is good if and only if G does not contain any bad vertex.

In Figure 3.1a and Figure 3.1c we show examples of a good and a bad dominating set, respectively (highlighted in blue) for the same graph G . Using the definition of a good dominating set, we can reword Lemma 7 as follows.

Lemma 8. *Let G' be an extension of a UNN G with the extension vertex v^* . Then G' is a UNN extension if and only if $N_{G'}(v^*)$ is a good dominating set of $\overline{G}$.*

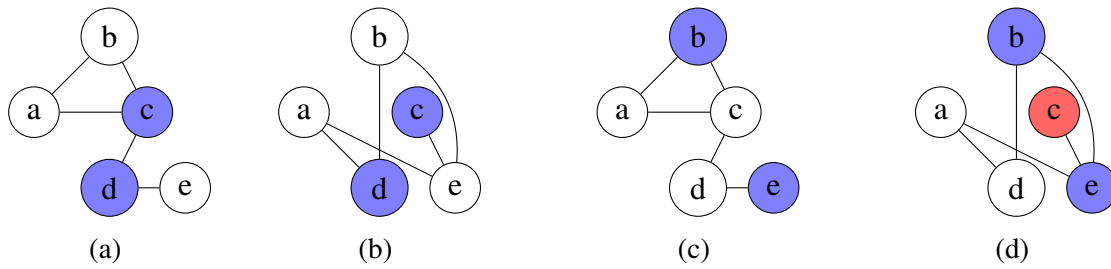


Figure 3.1: (a) Example of a good dominating set D (in blue) in a graph G . (b) $\overline{G}$, where each vertex not in D (non-blue) has a non-blue neighbor. (c) A bad dominating set D' in the same graph G shown in blue. (d) D' shown in $\overline{G}$ with the bad vertices in red (all neighbors of a bad vertex in $\overline{G}$ are blue).

3.3 An Approximation Algorithm

In this section, we construct a polynomial-time approximation algorithm for the UNN extension problem by showing a strict reduction from the UNN extension problem to the dominating set problem in complement graphs of UNNs. Since there is a polynomial-time $\mathcal{O}(\log n)$ -approximation algorithm for the minimum dominating set problem [17], our approximation algorithm will also have the same approximation ratio.

We provide the outline of our approximation algorithm in Algorithm 1.

Algorithm 1 Approximation Algorithm for the UNN Extension Problem

Require: a UNN graph G

Ensure: a UNN extension G' of G

- 1: Compute the complement graph $\overline{G}$
 - 2: Find a dominating set D of $\overline{G}$ using a black-box algorithm
 - 3: Improve D to obtain a dominating set D' of $\overline{G}$ such that $|D'| \leq |D|$
 - 4: Add a new vertex v^* to G and connect v^* to every vertex in D'
 - 5: Let G' be the resulting graph
 - 6: **return** G'
-

Figure 3.2 summarizes our algorithm in a block diagram.

It is obvious that Algorithm 1 will provide us with a UNN extension. If we can construct a good dominating set D' from D in polynomial time that is not larger than D (step 3), then Algorithm 1 will run in polynomial time. In the following lemma, we prove just that:

Lemma 9. *Let G be a UNN and let D be a dominating set in $\overline{G}$. Then there exists a good dominating set D' of $\overline{G}$ that is no larger than D and can be computed in polynomial time.*

Proof. We consider some cases depending on $\gamma(\overline{G})$.

Case 1: $\gamma(\overline{G}) = 1$. We find a vertex v in $\overline{G}$ that is adjacent to every other vertex in $\overline{G}$. We set D' to be $\{v\}$. Thus $|D'| \leq |D|$. We now show that D' is good. Note that v must be isolated in G . But since G is a UNN, this can only be true if v is the only vertex in G . In that case D' is good.

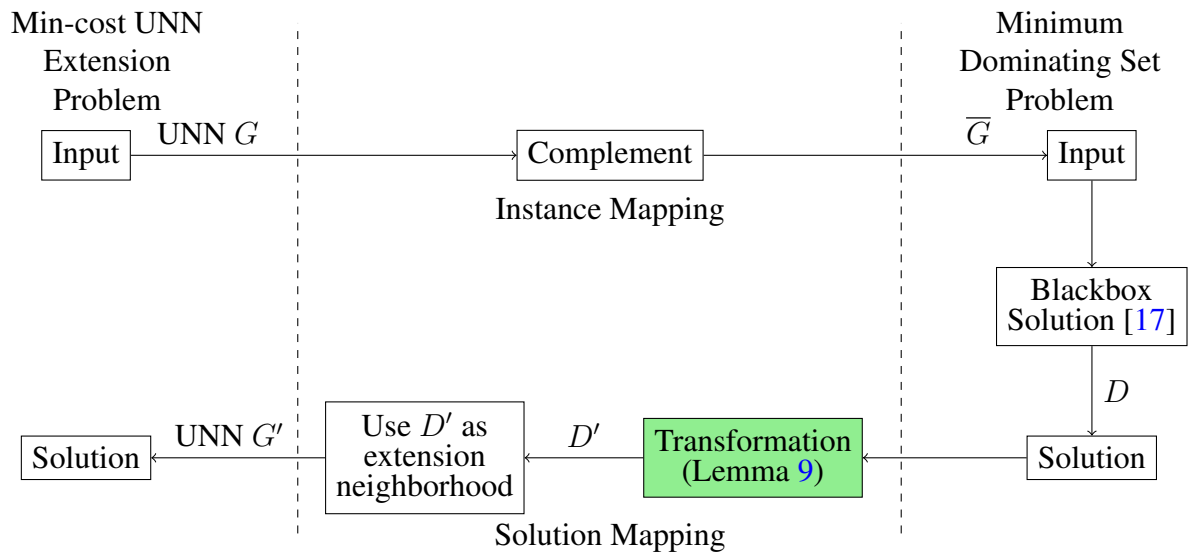


Figure 3.2: Reduction from the Minimum Dominating Set Problem to the Minimum UNN Extension Problem

Case 2: $\gamma(\overline{G}) = 2$. Here $|D| \geq 2$. We find a good dominating set D' by searching over all $\binom{V(G)}{2}$ pairs of vertices and checking if the choice of the pair is a good dominating set of $\overline{G}$. Note that such a good dominating set always exists. We prove this claim via contradiction.

Assume now that no such good dominating set exists. Since $\gamma(\overline{G}) = 2$, there exists a dominating set of size two. Let $S = \{x, y\}$ be such a dominating set. Then by our assumption there exists a vertex u that is bad with respect to S in $\overline{G}$. Since u is bad, every neighbor of u in G is in S . If u has no neighbors in G , then $\gamma(\overline{G}) = 1$, which is a contradiction. Thus, u must have either one or two neighbors in G . Note that u cannot have more than two neighbors in G since that would entail that u has neighbors outside of S .

If u has two neighbors in G , then both x and y are adjacent to u in G and thus S cannot dominate u in $\overline{G}$. Therefore u can have only one neighbor in G and that neighbor is also present in S . Without loss of generality assume that the neighbor is x . If x had any neighbor w in G other than u , then w would suppress u in G which cannot happen. Therefore, x only has one neighbor in G and that is u . Now we claim that $S' = \{x, u\}$ is a good dominating set of $\overline{G}$.

Since every vertex $v \in V(G) - \{u, x\}$ is adjacent to x in $\overline{G}$, S' is a dominating set of $\overline{G}$. Furthermore, no vertex in $V(G) - S'$ has any neighbors in S' in G . Since $\gamma(\overline{G}) > 1$, G has no isolated vertices. Therefore, every vertex in $V(G) - S'$ has at least one neighbor. Hence S' is also good, which contradicts our assumption.

Case 3: $\gamma(\overline{G}) > 2$. Without loss of generality, we may assume that D is minimal, i.e., $D - \{p\}$ is not a dominating set for $\overline{G}$ for any $p \in D$. If D is a good dominating set in $\overline{G}$, we may set $D' = D$. Otherwise, we consider two subcases.

Case 3.1: $\overline{G}$ has no isolated vertices. Let u be a bad vertex with respect to D in $\overline{G}$. Since $\overline{G}$ has no isolated vertices and D is minimal, every vertex in D must have a neighbor outside D in

$\overline{G}$. Using this observation we can construct a new set D' as follows (e.g., Figure 3.3):

- Initialize D' to an empty set and add u to D' . Since D is a dominating set in $\overline{G}$, we know that u is adjacent to at least one vertex $q \in D$ in $\overline{G}$.
- Iterate over each vertex $v(\neq q)$ in D . Note that v has a neighbor u' outside D in $\overline{G}$. If v does not have a neighbor in D' already, then add u' to D' .

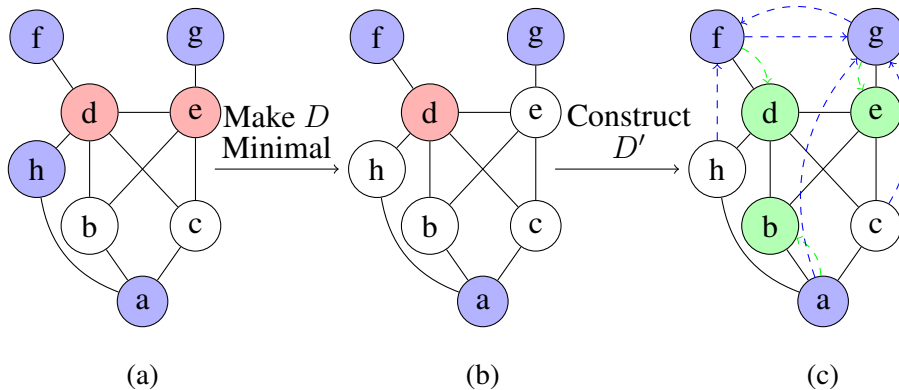


Figure 3.3: Construction of a good dominating set D' from a dominating set D , of a graph $\overline{G}$ where $\gamma(\overline{G}) > 2$. (a) A Dominating set D (in blue) of a graph $\overline{G}$. Bad vertices in $\overline{G}$ with respect to D are marked in red. (b) A minimal dominating set of $\overline{G}$ generated from the dominating set D . Vertices that are bad with respect to this new dominating set are shown in red. (c) A good dominating set D' (in green) of $\overline{G}$ generated from the minimal dominating set. Vertices in the minimal dominating set are still marked in blue. The neighbor of each blue vertex that is present in D' is marked with the green dashed lines. Note that D' is good because every non-blue vertex has a non-green neighbor in G (shown with the blue dashed lines).

We added the vertex u for the vertex $q \in D$ and for each remaining vertex in D , we added at most one vertex to D' . Therefore, $|D'|$ cannot be larger than $|D|$. In the following, we first show that D' is a dominating set of $\overline{G}$ and then show that D' is a good dominating set.

D' is a dominating set of $\overline{G}$: Let v be a vertex in $\overline{G}$. If $v \in D$, then a neighbor v' of v is in D' . If $v \notin D$, then by the definition of a bad vertex, v is dominated by the bad vertex u since $V(G) - D \subseteq N_{\overline{G}}[u]$.

D' is good in $\overline{G}$: Suppose for a contradiction that there exists a vertex v that is bad with respect to D' in $\overline{G}$. Then, v dominates every vertex in $V(G) - D'$ in $\overline{G}$. Since u is a bad vertex with respect to D in $\overline{G}$, u dominates all the vertices in $V(G) - D$ in $\overline{G}$. Since $D' \subseteq (V(G) - D)$, u dominates all the vertices in D' . Consequently, $\{u, v\}$ is a dominating set of $\overline{G}$, which contradicts that $\gamma(\overline{G}) > 2$.

Case 3.2: $\overline{G}$ has one or more isolated vertices. Let S be the set of vertices that are isolated in $\overline{G}$. Therefore, any dominating set of $\overline{G}$ must include all vertices in S .

If $\overline{G}$ only contains isolated vertices (i.e., if G is a complete graph), then $D(= S)$ is the minimum dominating set of $\overline{G}$, which is also good. We thus set D' to be D .

Otherwise, $G - S$ is a UNN such that $\overline{G} - S$ has no isolated vertices and $D - S$ is a dominating set in $\overline{G} - S$. Thus one of the cases among Cases 1, 2 and Case 3.1 must hold for $\overline{G} - S$ and thus we can find a good dominating set S' that is no larger than $|D - S|$. Therefore, $|S \cup S'| \leq |D|$. It is now straightforward to verify that $S' \cup S$ is a good dominating set in $\overline{G}$ and hence we can choose D' to be $S' \cup S$. $\square$

A direct consequence of Lemma 9 is that the minimum domination number and the minimum good domination number in complement graphs of UNNs coincide:

Lemma 10. *If G is a UNN, then $\gamma(\overline{G}) = \gamma_g(\overline{G})$.*

Proof. Let D_m be a minimum dominating set of $\overline{G}$. Then using Lemma 9 we can find a good dominating set D' from D_m such that $|D'| \leq |D_m| = \gamma(\overline{G})$. Since every good dominating set is also a dominating set, we have $\gamma(\overline{G}) \leq \gamma_g(\overline{G}) \leq |D'|$. Consequently, $\gamma_g(\overline{G}) = \gamma(\overline{G})$. $\square$

Finally, to prove that approximation algorithm can yield a $\mathcal{O}(\lg n)$ approximation ratio when using the greedy approximation algorithm as the dominating set problem black box, we will need to show that Algorithm 1 constitutes as a strict reduction i.e., the approximation ratio of the UNN extension solution is not larger than the approximation ratio of the dominating set black box solution.

Lemma 11. *Let G be a UNN and G' be the UNN extension of G constructed from Algorithm 1. If D is the dominating set of $\overline{G}$ generated in step 2 of the algorithm. Then $R_{UE}(G, G') \leq R_{DS}(\overline{G}, D)$.*

Proof. Observe that the cost of the extension is $|D'|$. Thus, the approximation ratio for the UNN extension problem solution is $R_{UE}(G, G') = \frac{|D'|}{\gamma_g(\overline{G})} = \frac{|D'|}{\gamma(\overline{G})}$. Since, $|D| \leq |D'|$ due to Lemma 9, $R_{UE}(G, G') \leq \frac{|D|}{\gamma(\overline{G})}$.

But the approximation ratio of the black box solution of the dominating set instance is $R_{DS}(\overline{G}, D)$. Thus, $R_{UE}(G, G') \leq R_{DS}(\overline{G}, D)$ and Algorithm 1 constitutes as a strict reduction from the UNN extension problem to the dominating set problem. $\square$

Thus, when using the greedy approximation for the dominating set problem, Algorithm 1 yields a $\mathcal{O}(\lg n)$ approximation ratio.

Theorem 1. *Given a UNN G with n vertices, we can compute a UNN extension G' of G in polynomial time such that the cost is at most $\mathcal{O}(\log n)$ times the cost of a minimum UNN extension of G .*

Chapter 4

Hardness of Approximation for the UNN Extension Problem

In this chapter, we establish hardness of approximation results for the UNN Extension problem. To this end, we introduce the notion of UNNification, which enables the construction of a UNN from an arbitrary graph. We then analyze structural properties of the complement of the UNNification of the complement graph for arbitrary graphs.

Using these properties, we develop a strict reduction from the Total Dominating Set problem to the UNN Extension problem. This reduction proceeds in two steps. First, we focus on a special class of dominating sets in the complement of the UNNification of the complement graph, referred to as blue dominating sets. We show that these sets are in one-to-one correspondence with total dominating sets in the original graph (Section 4.3).

Using this correspondence, we then establish a strict reduction from the Total Dominating Set problem to the minimum-cost UNN Extension problem (Section 4.4). Since the Total Dominating Set problem is $\mathcal{NP}$ -hard to approximate within any factor in $o(\lg n)$, it follows that the UNN Extension problem is also $\mathcal{NP}$ -hard to approximate within the same bound.

4.1 UNNification of a Graph

We will use the concept of *UNNification* that, given a graph G , constructs a UNN U_G as follows.

Definition 4.1.1. UNNification of a graph G is another graph U_G such that:

- For every vertex $v \in V(G)$, U_G has two copies of v , one red (denoted as $r(v)$) and one blue (denoted as $b(v)$), and $r(v)$ and $b(v)$ are adjacent in U_G .
- For distinct vertices $u, v \in V(G)$, their corresponding blue vertices $b(u)$ and $b(v)$ are adjacent in U_G if and only if u and v are adjacent in G .

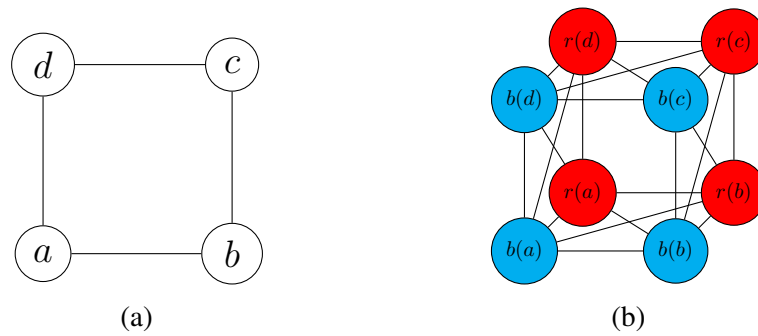


Figure 4.1: (a) A graph G . (b) The UNNification of G .

- For distinct vertices $u, v \in V(G)$, $b(u)$ and $r(v)$ are adjacent in U_G if and only if u and v are adjacent in G .
- For distinct vertices $u, v \in V(G)$, the corresponding red vertices $r(u)$ and $r(v)$ are adjacent in U_G if and only if u and v are incomparable in G .

Figure 4.1 shows an example of a graph G (on the left) and its UNNification U_G on the right. A more detailed step-by-step construction of a different graph is shown in Figure 4.2.

Let p and q be a red and blue vertex in U_G , respectively. Then $r^{-1}(p)$ and $b^{-1}(q)$ return their corresponding vertices in G . For notational convenience, we will extend the functions $b(v)$ and $r(v)$ to sets of vertices over G . For a set $S \subseteq V(G)$, $b(S)$ (similarly, $r(S)$) denotes the blue (red) vertices in U_G corresponding to the vertices in S . Similarly, we also extend the functions b^{-1} and r^{-1} .

We now have the following theorem:

Theorem 2. *Let G be a graph. Then its UNNification U_G is a UNN.*

Proof. By Lemma 3, it suffices to show that no vertex in U_G is suppressed by another vertex.

- $\mathbf{b(u) \not\preceq b(v)}$: We consider two scenarios to show that no blue vertex $b(u)$ can be suppressed by another blue vertex $b(v)$. First, if u and v are adjacent in G , then by construction $b(u)$ and $b(v)$ are adjacent in U_G , and hence they are incomparable. Second, if u and v are not adjacent in G , then by construction $b(v)$ is not adjacent to $r(u)$, whereas $r(u)$ is a neighbor of $b(u)$. Therefore, $b(u) \not\preceq b(v)$.
- $\mathbf{b(u) \not\preceq r(v)}$: We now show that no blue vertex $b(u)$ can be suppressed by a red vertex $r(v)$. Note that u and v are not necessarily distinct. We consider three cases:
 - **Case 1** ($u = v$): Since $b(u)$ and $r(u)$ are adjacent in U_G , they are incomparable. Since $r(u) = r(v)$, $b(u) \not\preceq r(v)$.
 - **Case 2** ($u \neq v$ and $u \preceq_G v$): Since v suppresses u in G , by construction $r(u)$ and $r(v)$ are not adjacent in U_G . Since $r(u)$ is a neighbor of $b(u)$ but not of $r(v)$, $b(u) \not\preceq r(v)$.

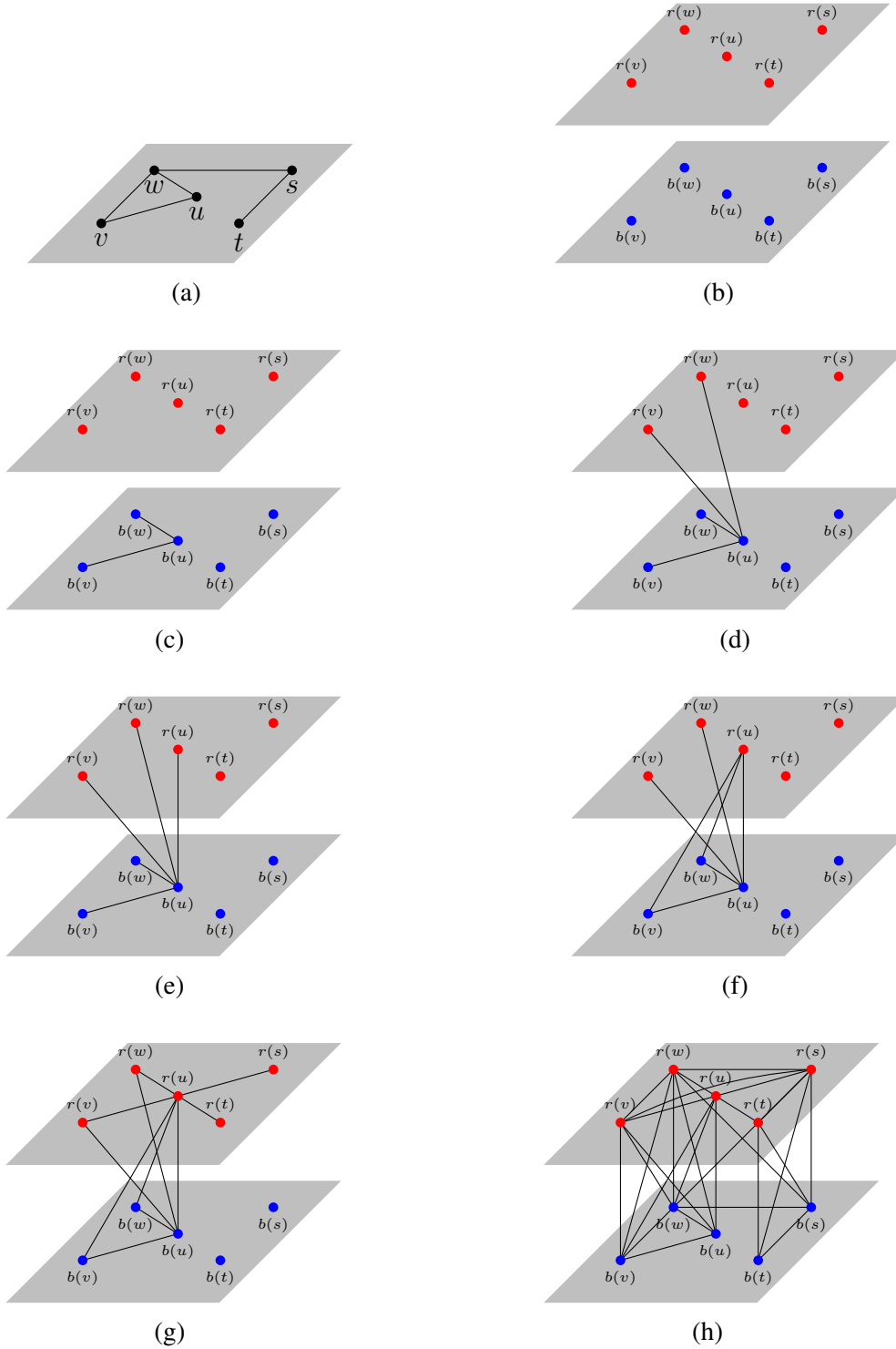


Figure 4.2: A graph G and its UNNification graph U_G shown step by step: In (a) we show the original graph G . (b) shows the vertices of U_G . (c)-(g) picks the vertex u and adds the edges between the blue and red neighbors of $b(u)$ (in (c), (d), (e)) and $r(u)$ (in (f) and (g)). (h) shows U_G after the construction is complete.

- **Case 3 ($u \neq v$ and $u \not\sim_G v$):** Let $u' \in N_G(u)$ be a vertex that is not adjacent to v in G . If $u' = v$, then $b(u)$ and $r(v)$ are adjacent in U_G , which makes them incomparable. Therefore, we may assume that $u' \neq v$. Since u and u' are adjacent in G , $b(u')$ and $b(u)$ are adjacent in U_G . Since $v \neq u'$ and since v and u' are not adjacent in G , $b(u')$ is not a neighbor of $r(v)$. Hence $b(u) \not\sim r(v)$.
- **$r(u) \not\sim b(v)$:** We now show that no red vertex $r(u)$ can be suppressed by some blue vertex $b(v)$. Note that u and v are not necessarily distinct. If $u = v$, then $r(u)$ and $b(v)$ are adjacent in U_G , which makes them incomparable. If $u \neq v$ and u and v are adjacent in G , then again $r(u)$ and $b(v)$ are adjacent in U_G , and hence they are incomparable. Otherwise, $u \neq v$ and u and v are not adjacent in G . Here $b(u)$ and $b(v)$ are not adjacent in U_G . Since $b(u)$ and $r(u)$ are adjacent in U_G , $r(u) \not\sim b(v)$.
- **$r(u) \not\sim r(v)$:** We now show that red vertex $r(u)$ can be suppressed by another red vertex $r(v)$. If u and v are incomparable in G , then by construction $r(u)$ and $r(v)$ are adjacent in U_G , which makes them incomparable. Otherwise, u and v are comparable in G , and hence u and v cannot be adjacent in G . Consequently, $b(u)$ and $r(v)$ cannot be adjacent. Since $b(u)$ and $r(u)$ are adjacent but $b(u)$ is not a neighbor of $r(v)$, $r(u) \not\sim r(v)$.

□

4.2 Properties of $\overline{U_G}$

In this section, we will study properties of the graph $\overline{U_G}$. More precisely, we will study how the edges of $\overline{U_G}$ relate to the edges of G . We will use these properties later when we construct our strict reduction.

Lemma 12. *Let G be an arbitrary graph. Let G' be the complement graph of the UNNification of $\overline{G}$. Let $u, v \in V(G')$ be adjacent in G' . Then the corresponding vertices in G are adjacent.*

Proof. Since u and v are adjacent in G' , they are non-adjacent in $\overline{G'} = U_{\overline{G}}$. Note that in a UNNification, each vertex is colored either blue or red. We now consider two cases.

Case 1 (At least one of u and v are blue): Without loss of generality assume u is blue. Let u' and v' respectively be the vertices corresponding to u and v in G . Since u is not adjacent to v in $U_{\overline{G}}$, u' and v' must be distinct.

Now if u' is not adjacent to v' in G , then u' is adjacent to v' in $\overline{G}$ and thus $u = b(u')$ is adjacent to $b(v')$ and $r(v')$ in $U_{\overline{G}}$. Based on the color of v , v is either $b(v')$ or $r(v')$. In either case, u is adjacent to v in $U_{\overline{G}}$, which is a contradiction. Thus u' must be adjacent to v' in G .

Case 2 (Both u and v are red): By Definition of UNNification, the corresponding vertices $u' = r^{-1}(u)$ and $v' = r^{-1}(v)$ are comparable in $\overline{G}$. Note that since u and v are not the same,

$u' \neq v'$. If u' and v' were adjacent in $\overline{G}$, they would be incomparable in $\overline{G}$ and thus u' and v' are not adjacent in $\overline{G}$. Thus, they are adjacent in G . $\square$

Lemma 13. *Let G be an arbitrary graph. Let G' be the complement graph of the UNNification of $\overline{G}$. Let $u, v \in V(G)$ be adjacent in G . Then $b(u)$ must be adjacent to both $b(v)$ and $r(v)$ in G' .*

Proof. Since u and v are adjacent in G , they are non-adjacent in $\overline{G}$. Thus, by the definition of UNNification, $b(u)$ is not adjacent to $b(v)$ or $r(v)$ in the UNNification of $\overline{G}$. Therefore, $b(u)$ is adjacent to both $b(v)$ and $r(v)$ in G' . $\square$

Corollary 1. *Let G be an arbitrary graph. Let G' be the complement graph of the UNNification of $\overline{G}$. Let v be a red vertex in G' and u be its blue sibling. Then $N_{G'}(v) \subseteq N_{G'}(u)$.*

Proof. It suffices to show that for every $w \in N_{G'}(v)$, w is adjacent to u . By Lemma 12, the vertices corresponding to v and w in G are adjacent. Since u and v are siblings, they correspond to the same vertex in G . Therefore, the vertices corresponding to u and w in G are adjacent. By Lemma 13, u is adjacent to w in G' . $\square$

4.3 Blue Dominating Set Problem and its Relation with the Total Dominating Set Problem

In this section, we introduce the idea of a blue dominating set in bi-colored graphs where the vertices are either blue or red. We will use this idea to show that the task of finding a total dominating set in any graph G is equivalent to finding a blue dominating set in $\overline{U_{\overline{G}}}$. This fact will later be used to show the reduction from the Total Dominating Set Problem to the UNN extension problem.

Let H be a graph where every vertex is either colored blue or red. A *blue dominating set* in H is a dominating set of H that only contains blue vertices. The size of the minimum blue dominating set is the *blue domination number* of H , which we denote as $\gamma^b(H)$.

In this section, we will show that for any total dominating set D in an arbitrary graph G , we can find a blue dominating set D' of $G' = \overline{U_{\overline{G}}}$ such that $|D'| = |D|$ in polynomial time (Lemma 14). Furthermore, given a blue dominating set D' in G' , we can find a total dominating set D in G such that $|D| = |D'|$ (Lemma 15).

Lemma 14. *Let D' be a dominating set in $G' = \overline{U_{\overline{G}}}$ where all the vertices are blue. Then the set $D = b^{-1}(D') = \{u \in V(G) \mid b(u) \in D'\}$ is a total dominating set in G .*

Proof. It suffices to show that every vertex $u \in V(G)$ is adjacent to a vertex in D . Consider the vertex $r(u)$ in G' . We know that $r(u)$ is not in D' since it is red. Since D' is a dominating set in G' , D' must contain a blue vertex $b(v)$ that is adjacent to $r(u)$ in G' . By construction of D , we have $v \in D$. Finally, by Lemma 12, u is adjacent to v in G . $\square$

Lemma 15. *Let D be a total dominating set of G . Then $b(D) = \{b(u) \in V(G') \mid u \in D\}$ is a dominating set in $G' = \overline{U_{\overline{G}}}$.*

Proof. It suffices to show that $b(D)$ dominates every vertex $v' \in V(G')$. Let v be the vertex corresponding to v' in G and $u \in D$ be a vertex that is adjacent to v . Note that u must exist since D is a total dominating set. Since $u \in D$, $b(u) \in b(D)$. Again, since u and v are adjacent in G , by Lemma 13, $b(u)$ is adjacent to both $b(v)$ and $r(v)$ in G' . Hence, $b(u)$ dominates v' . $\square$

Lemmas 14 and 15 imply a one-to-one mapping from the total dominating set of G to the blue dominating sets of $\overline{U_{\overline{G}}}$, a consequence of which is the following:

Lemma 16. *Let G be an arbitrary graph. Let $G' = \overline{U_{\overline{G}}}$ be the complement of the UNNification of $\overline{G}$. Then $\gamma_t(G) = \gamma^b(G')$.*

4.4 Reduction from Total Dominating Set Problem to the Minimum-cost UNN Extension Problem

In this section, we provide a strict reduction from the total dominating set problem in graphs without any isolated vertices to the UNN extension problem.

The outline of our reduction is shown in Algorithm 2.

Algorithm 2 Algorithm for solving Total Dominating set problem using a black box solution to the UNN Extension Problem

Require: a UNN graph G without any isolated vertices

Ensure: A total dominating set of G

- 1: Compute $U_{\overline{G}}$. By Theorem 2 $U_{\overline{G}}$ is a UNN.
 - 2: Find a UNN extension G'' of $U_{\overline{G}}$ using a black-box algorithm for UNN extension.
 - 3: Find the extension neighborhood D of G'' . By Lemma 8 D is a dominating set of $G' = \overline{U_{\overline{G}}}$.
 - 4: Using D construct a blue dominating set D' of $\overline{U_{\overline{G}}}$ such that $|D'| \leq |D|$.
 - 5: Compute $S = b^{-1}(D')$. According to Lemma 14, S is a total dominating set of G .
 - 6: **return** S
-

Figure 4.3 shows the outline of the reduction using a block diagram.

It is not difficult to see that Algorithm 2 returns a total dominating set of G . However, it is not clear how exactly we can construct the blue dominating set D from D' (step 4) and if such a construction can be done in polynomial time. Here, we will show exactly that.

Lemma 17. *Let G be a graph without any isolated vertices and let $G' = \overline{U_{\overline{G}}}$. Given any UNN extension of $U_{\overline{G}}$, a blue dominating set D' of G' can be computed in polynomial time such that D' is not larger than the cost of the UNN extension.*

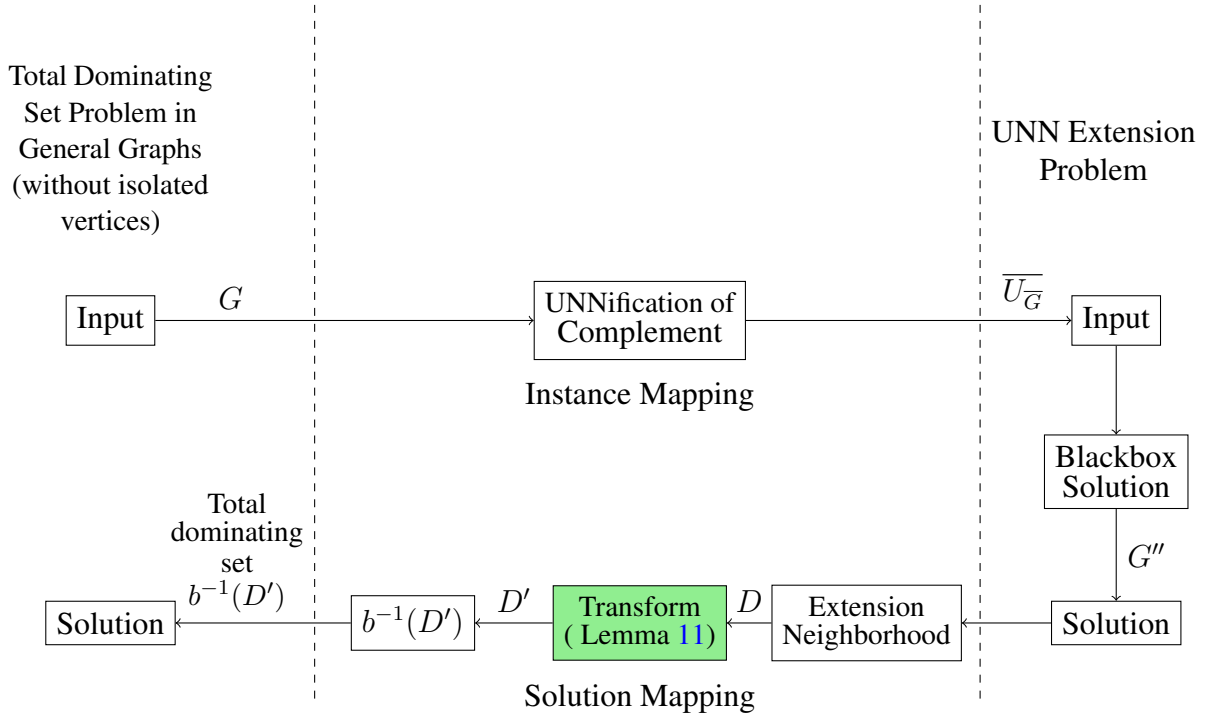


Figure 4.3: Reduction from Total Dominating Set Problem in General Graphs to Minimum Dominating Set Problem in Complement Graphs of UNNs

Proof. Let G'' be the UNN extension of $\overline{U_G}$ and let D be the neighborhood of the extension vertex of G'' . Then, the cost of G'' is $|D|$. Therefore, we will have to find a blue dominating set D' of G' such that $|D'|$ is not larger than $|D|$.

To construct the set D' , we introduce the concept of a “loneliness-free” set. Let M be a subset of $V(G')$. We say that a vertex $v' \in M$ is *lonely* in M if v' is red and its blue sibling is not present in M . We call M *loneliness-free* if no vertex $v' \in M$ is lonely. We construct D' in two steps. If D is not loneliness-free, then we first create a loneliness-free dominating set L of G' such that $|L| = |D|$. We then construct the required blue dominating set D' from L . **Step 1 (Creating L from D):** Let S be the set of lonely vertices in D and $T = b(r^{-1}(S))$ be the set of the blue siblings of all the lonely vertices in D . We will construct the set L by deleting all lonely vertices from D and adding their blue siblings to it, i.e., $L = (D \setminus S) \cup T$. We claim that L is the required loneliness-free dominating set of G' . It is evident that $|L| = |D|$ and L can be computed in polynomial time. We thus need to show that L is indeed loneliness-free and L is a dominating set in G' .

Suppose for a contradiction that v' is a lonely vertex in L . By definition of a lonely vertex, v' must be red. By the construction of L , all red vertices of L are also in D . Thus, v' must also belong to D . If v' was lonely in D , then v' cannot be in L . Thus, the blue sibling of v' must be present in D . Since all blue vertices of D are also present in L , v' cannot be lonely in L .

We now show that L is a dominating set of G' . Let u' be an arbitrary vertex in G' . Due to Lemma

8, D is a dominating set in G' , and therefore there exists a vertex $v' \in D$ that dominates u' in G' . We now show that u' remains dominated by a vertex in L .

Case 1 (v' is not lonely in D): Since we only delete lonely vertices from D in our construction of L , $v' \in L$ and thus u' is dominated by L .

Case 2 (v' is lonely in D and $u' = v'$): Let v be the vertex corresponding to v' in G and let $w' = b(v)$ be the blue sibling of v' . Since v' is lonely in D , $w' \notin D$. Hence, there exists a vertex x' in D that is adjacent to w' . Let x be the vertex in G corresponding to x' . Then, regardless of the color of x' , $b(x)$ is present in L . Since x' and v' are adjacent in G' , by Lemma 12 x and v are adjacent in G and thus due to Lemma 13 $b(x)$ is adjacent to v' .

Case 3 (v' is lonely in D and $u' \neq v'$): Since u', v' are adjacent in G' , by Lemma 12 the corresponding vertices u and v in G are adjacent. By Lemma 13 $b(v)$, which is the blue sibling of v' , is adjacent to both $b(u)$ and $r(u)$. Here one of $b(u)$ and $r(u)$ is u' . Since v' is lonely in D , we have $b(v) \in L$, and hence u' remains dominated in L .

Step 2 (Creating D' from L): We create D' from L in polynomial time as follows. Add every blue vertex $u' \in L$ to D' . For every red vertex $v' \in L$, pick an arbitrary blue neighbour of v' in G' and add it to D' . Note that since G has no isolated vertex, such a blue neighbor of v' must exist in G' .

It is straightforward to observe that $|D'| \leq |L|$. It thus remains to show that D' is a dominating set of G' . Let v' be a vertex of G' . Since L is a dominating set of G' , either v' belongs to L , or there exists a vertex $u' \in L$ that is adjacent to v' . If v' belongs to L , then by construction of D' , either v' or a neighbor of v' belongs to D' . We thus consider the scenario when $v' \notin L$.

If u' is blue, by our construction, u' remains in D' and thus v' remains dominated by D' . If u' is red, let w' be the blue sibling of u' . By Corollary 1, we know that $N_{G'}(u') \subseteq N_{G'}(w')$. Therefore, w' must be adjacent to v' as well. Notice that since L is loneliness-free, $w' \in L$. Since w' is blue, it remains in D' as well. Thus v' remains dominated by $w' \in D'$. $\square$

Lemma 17 implies that $\gamma^b(G') \leq \gamma_g(G') = \gamma(G')$. Since a blue dominating set is also a dominating set, $\gamma^b(G') = \gamma(G') = \gamma_g(G')$.

Lemma 18. *Let G be a graph without any isolated vertices. Let $G' = \overline{U_{\overline{G}}}$ be the complement of the UNNification of $\overline{G}$. Then $\gamma^b(G') = \gamma_g(G') = \gamma(G')$.*

So far, we have proven that Algorithm 2 is actually correct and runs in polynomial time as long as G has no isolated vertices. To prove that the UNN extension problem is hard to approximate in polynomial time, we will also have to show that our reduction is approximation preserving, which is what we do in the following lemma.

Lemma 19. *Let G be a graph without any isolated vertices and let S be the total dominating set of G constructed by Algorithm 2. If G'' is the UNN extension of $U_{\overline{G}}$ returned from step 1 in Algorithm 2. Then, $R_{TDS}(G, S) \leq R_{UE}(U_{\overline{G}}, G'')$.*

Proof. Let $G' = \overline{U_{\overline{G}}}$. Then, $R_{UE}(U_{\overline{G}}, G'') = \frac{|D|}{\gamma_g(G')}$. But according to Lemma 18, $\gamma^b(G') = \gamma(G') = \gamma_g(G')$. So, $R_{UE} = \frac{|S|}{\gamma^b(G')}$. But according to Lemma 16, $\gamma_t(G) = \gamma^b(\overline{G})$. Therefore, $R_{UE} = \frac{|S|}{\gamma^t(G)}$.

But, $R_{TDS}(G, S) = \frac{|S|}{\gamma^t(G)}$. Since $|S| \leq |D|$ by construction, $R_{TDS}(G, S) \leq R_{UE}(U_{\overline{G}}, G'')$. $\square$

Lemma 19 implies that the minimum-cost UNN extension problem cannot be approximated in polynomial time within a factor of $o(\log n)$ unless $\mathcal{P} = \mathcal{NP}$. If it were not the case, then there would exist an algorithm $\mathcal{A}$ that produces a UNN extension with approximation ratio in $o(\lg n)$ in polynomial time. If we used such an algorithm $\mathcal{A}$ in step 2 in Algorithm 2 as the black box for the UNN extension problem, then the approximation ratio of Algorithm 2 would be in $o(\lg n)$. But, since the total dominating set problem is hard to approximate within a factor of $c \log n$, for some constant $c > 0$, no such algorithm exists.

Theorem 3. *The minimum-cost UNN extension problem cannot be approximated in polynomial time within a factor of $o(\log n)$ unless $\mathcal{P} = \mathcal{NP}$.*

4.5 On Hardness of Decision Variant and Regular Optimization Variant of the UNN Extension Problem

Since our reduction can be easily modified to solve an approximation for the regular (non-search) optimization variant of the UNN extension problem, we can conclude that approximating the cost of the optimal UNN extension is also hard.

Since approximating the cost of the optimal UNN extension is strictly easier than the decision variant of the problem, the decision variant also remains $\mathcal{NP}$ -hard.

Chapter 5

Hardness of Approximation for the UNN Completion Problem

In this chapter, we prove that it is also $\mathcal{NP}$ -hard to approximate the UNN Completion Problem. We prove this via showing a strict reduction from the UNN Extension Problem to the UNN Completion Problem.

5.1 A Reduction from the UNN Extension Problem to the UNN Completion Problem

Let G be a UNN and G' be a UNN extension G with the extension vertex v^* . Consider the graph $H = G \cup \{v^*\}$. Clearly, G' is a UNN Completion of H . Thus, $\text{OPT}_{UC}(H) \leq \text{OPT}_{UE}(G)$. However, not every UNN Completion of H will be a UNN extension of G , because a UNN completion of H is allowed to have edge insertions between two distinct vertices of $V(G)$ and all edge insertions do not need to be incident on v^* . We, however, claim that UNN completions of H that are not valid UNN extensions of G are not “minimal” and that we can find a less costly UNN completion that is also a UNN extension of G from such non-minimal UNN Completions.

Lemma 20. *Let G be a UNN and let $H = G \cup \{v^*\}$ for some $v^* \notin V(G)$. Given any UNN Completion H' of H , we can find a UNN Completion H'' of H that is also a UNN extension of G (with the extension vertex v^*) and has a smaller cost than H' in polynomial time.*

Proof. Let $E' = E(H')/E(H)$ be the set of newly added edges in H' . We can then partition E' into two sets E_1 and E_2 where E_1 is the set of all edges in E' that are incident on v^* and $E_2 = E'/E_1$ are the edges of E' that are not. Thus, the cost of the UNN extension H' is $|E_1| + |E_2|$.

Let D be the neighbors of v^* in H . Since v^* is not suppressed by any other vertices in $V(G)$, D

is a dominating set of $\overline{G - \{v^*\}} = \overline{G} - E_2$. Since, D is a dominating set of $\overline{G} - E_2$, D is also a dominating set of $\overline{G}$.

Since G is a UNN and D is a dominating set of $\overline{G}$, we can construct a new good dominating set D' of $\overline{G}$ such that $|D'| \leq |D| = |E_1|$ using Lemma 9. Let H'' be the graph obtained by inserting edges between v^* and the vertices of D' . Then, according to Lemma 8, H'' is a UNN extension of G with the extension vertex v^* and thus is a UNN completion of H . Furthermore, the cost of $H = |D| \leq |E_1| \leq |E'|$.

Thus H'' is such a UNN completion. □

A consequence of Lemma 20 is the following:

Lemma 21. *Let G be a UNN and let $H = G \cup \{v^*\}$ for some $v^* \notin V(G)$. Then $\text{OPT}_{UE}(G) = \text{OPT}_{UC}(H)$.*

Proof. Let H' be an optimal UNN extension of H . Using Lemma 20, we can find a new UNN completion H'' in polynomial time such that H'' is a UNN extension of G and $\text{cost}(H, H'') \leq \text{OPT}_{UC}(H)$. Thus, H'' is an optimal UNN completion of H as well. Since H'' is a UNN extension of G , $\text{cost}_{UE}(G, H'') = \text{cost}_{UC}(H, H'') = \text{OPT}_{UC}(H) \geq \text{OPT}_{UE}(G)$. But, $\text{OPT}_{UC}(H) \leq \text{OPT}_{UE}(G)$. Thus, $\text{OPT}_{UE}(G) = \text{OPT}_{UC}(H)$, as required. □

Furthermore, using Lemma 20, we can construct a strict reduction from the UNN extension problem to the UNN completion problem. We show our reduction in Algorithm 3

Algorithm 3 Approximation Algorithm for the UNN Extension Problem

Require: a UNN graph G

Ensure: a UNN extension H'' of G

- 1: Compute $H = G \cup \{v^*\}$ for some $v^* \notin V(G)$.
 - 2: Find a UNN completion H' of H using a black-box algorithm for the UNN completion problem
 - 3: Improve H' to obtain a UNN completion set H'' of H using Lemma 20. **return** H''
-

In Algorithm 3, the approximation ratio of the UNN extension H'' is $R_{UE}(G, H'') = \frac{\text{cost}_{UE}(G, H'')}{\text{OPT}_{UE}(G)} = \frac{\text{cost}_{UE}(G, H'')}{\text{OPT}_{UC}(G)} \leq \frac{\text{cost}_{UC}(H, H')}{\text{OPT}_{UC}(G)} = R_{UC}(H, H')$. Thus, Algorithm 3 is a strict reduction.

Lemma 22. *Algorithm 3 is a strict reduction.*

If there exists any polynomial-time algorithm for the UNN completion problem with approximation ratio in $o(\lg n)$, then using such an algorithm as a black box in Algorithm 3 would yield a polynomial-time approximation algorithm for the UNN extension problem with a better ratio, which contradicts Theorem 3 (unless $\mathcal{P} = \mathcal{NP}$).

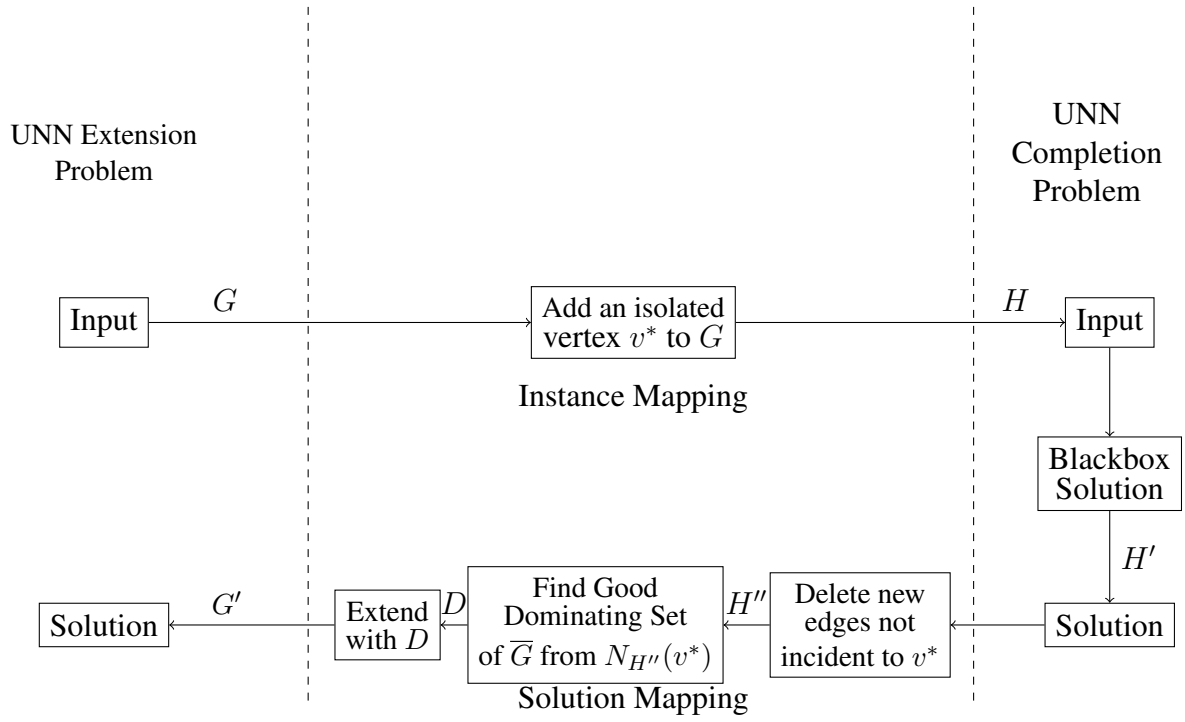


Figure 5.1: Reduction from the UNN Extension Problem to the UNN Completion Problem

Theorem 4. *The minimum-cost UNN completion problem cannot be approximated in polynomial time within a factor of $o(\log n)$ unless $\mathcal{P} = \mathcal{NP}$.*

Similar to the hardness result mentioned in Chapter 4, this result also translates for the regular variant of the optimization problem and the decision variant of the problem.

Chapter 6

An Approximation Algorithm for the UNN Completion Problem

In this chapter, we will show a polynomial time linear approximation algorithm for the UNN completion problem. On the other hand, Theorem 4 showed that the UNN completion problem is $\lg$ -APX-hard, meaning that it cannot be approximated within a factor of $\epsilon \lg n$ for small enough ϵ . The algorithm we show will take an arbitrary graph G and first transform it into a special type of graph called *partitioned graph* using edge insertions, and then we will solve the UNN completion problem on that partitioned graphs.

6.1 Notation

In this section, we will introduce some basic terminology. We call a pair of vertices u, v *twins* in G when both u and v suppress each other in G . If u suppresses v and $u \neq v$, then u is said to be a *proper suppressor* of v in G and v is said to be *properly suppressed* in G . We call a pair of vertices (u, v) a *suppression pair* of G if u is a proper suppressor of v . We call the set of all suppression pairs in G , the *suppression set* of G and denote it as $\text{Supp}(G)$

Let G be a graph and $G' = G \cup \{(u, v)\}$ be the graph obtained from G via an edge insertion between u and v . We denote the suppression pairs that are in G' but not in G by $N_{\text{supp}}(G, u, v)$ i.e., $N_{\text{supp}}(G, u, v) = \text{Supp}(G') / \text{Supp}(G)$.

6.2 Effects of Edge Insertions

To find a UNN Completion of a graph G , we need to repeatedly insert edges between two non-adjacent vertices of G until all suppression pairs are removed. Thus, it is important to consider the effects of edge insertions on the suppression set of G .

If we add an edge between two distinct non-adjacent vertices in G , it may remove some suppression pairs while introducing others. Lemma 23 tells us which suppression pairs can be removed from G due to an edge insertion while Lemma 24 describes which suppression pairs may be introduced to G after an edge insertion.

Lemma 23. *Let G be a graph and u, v be two distinct non-adjacent vertices in G . Let G' be the graph resulting from adding the edge (u, v) to G . If (p, q) is a suppression pair in G but not in G' , then either $q = u$ or $q = v$.*

Proof. If q is neither u nor v , then $N_{G'}(q) = N_G(q) \subseteq N_G(p) \subseteq N_{G'}(p)$. Thus, p continues to suppress q in G' , a contradiction.

Thus, q must be either u or v . □

Lemma 24. *Let G be a graph and u, v be two distinct non-adjacent vertices in G . Let $(p, q) \in N_{\text{supp}}(G, u, v)$. Then, either $p = v$ and q is a neighbor of u in G , or $p = u$ and q is a neighbor of v in G .*

Proof. Let $G' = G \cup (u, v)$. If p is neither u nor v , then $N_{G'}(p) = N_G(p)$. Now, if p properly suppressed q in G' , then $N_G(q) \subseteq N_{G'}(q) \subseteq N_{G'}(p) = N_G(p)$. Therefore, p properly suppressed q in G as well. But, this is impossible, since $(p, q) \in N_{\text{supp}}(G)$. Thus, p must be either u or v .

If $p = u$, then $q \neq u$ and $q \neq v$. Hence, the neighborhood of q does not change under the edge insertion:

$$N_{G'}(q) = N_G(q).$$

Since $(p, q) \in N_{\text{supp}}(G, u, v)$, q must have

$$N_G(q) \subseteq N_G(u) \cup \{v\}.$$

If q were not adjacent to v in G , then $N_G(q) \subseteq N_G(u)$, which would imply that $p = u$ properly suppresses q in G , contradicting the fact that $(p, q) \in N_{\text{supp}}(G, u, v)$.

Therefore, q must be adjacent to v in G .

By similar argument, we can also show that if $p = v$, then q must be adjacent to u in G . □

While edge insertions between arbitrary non-adjacent vertices are interesting, we will focus on the case where the two vertices have a suppressor–suppressed relationship.

Corollary 2. *Let G be a graph, and let (u, v) be a suppression pair in G . If $(p, q) \in N_{\text{supp}}(G, u, v)$, then $p = v$ and q is a neighbor of u in G .*

Proof. From Lemma 24, we know that either $p = u$ and q is a neighbor of v in G or $p = v$ and q is a neighbor of u in G .

Let us assume that $p = u$. Then, q is a neighbor of v . But u suppresses v in G , thus, q must be adjacent to p in G as well. But, then p and q are adjacent in $G \cup \{(u, v)\}$ which can not happen. Therefore, $p = v$ and q is a neighbor of u in G . $\square$

Another interesting case of edge insertion is when we add an edge between two twin vertices.

Lemma 25. *Let u and v be twins in G . Then, $N_{\text{supp}}(G, u, v) = \emptyset$.*

Proof. Let us assume that there exists pairs of vertices $(p, q) \in N_{\text{supp}}(G, u, v)$. Then, according to Corollary 2, $p = v$ since $u \succeq_G v$.

But again, $v \succeq_G u$. So, $p = u$ (Corollary 2). A contradiction. So, $N_{\text{supp}}(G, u, v) = \emptyset$. $\square$

If a suppressor v has exactly one vertex that it properly suppresses, then adding an edge between them may introduce new suppression pairs. However, when a vertex v properly suppresses more than one vertex, it is possible to remove all such suppression pairs without introducing any new suppression pairs into the graph. We show how to achieve this using Lemmas 26 and 27.

Lemma 26. *Let u, v, w be three distinct vertices in a graph G where u suppresses both v and w . Then v and w are not adjacent in G and $\text{Supp}(G \cup \{(v, w)\}) \subseteq \text{Supp}(G) / \{(u, v), (u, w)\}$.*

Proof. Since u suppresses both v and w , u is adjacent to neither v nor w . If v and w were adjacent in G , that would mean that u is adjacent to them as well, since u suppresses them. But that can not happen. So, v and w are not adjacent in G .

Now, consider the graph $G' = G \cup \{(v, w)\}$. Here $N_{G'}(u) = N_G(u)$, $N_{G'}(v) = N_G(v) \cup \{w\}$ and $N_{G'}(w) = N_G(w) \cup \{v\}$. Since $w \in N_{G'}(v)$ and $w \notin N_G(u)$, u no longer suppresses v in G' . Similarly, u no longer suppresses w in G' .

Finally, suppose that the addition of (v, w) introduces some new suppression pair (p, q) in G' that was not present in G . Then, by Lemma 24, either $p = v$ and q is adjacent to w in G , or $p = w$ and q is adjacent to v in G . Since, u suppresses both v and w in G , u is adjacent to q in G , and thus, in G' as well. But u is adjacent to neither v nor w in G' (Remark 1). Thus, p cannot suppress q in G' . $\square$

Lemma 27. *Let v be a suppressor in a graph G , and let $S = \{u_1, u_2, \dots, u_p\}$ be the set of vertices that v properly suppresses in G . Let G' be a graph resulting from G by arbitrarily adding edges between the vertices in S in such a manner that every vertex $s \in S$ has a neighbor in S . Then, $\text{Supp}(G') \subseteq \text{Supp}(G) / \{(v, u_1), (v, u_2), (v, u_3), \dots, (v, u_p)\}$.*

Proof. Consider an arbitrary vertex $u \in S$. By construction, u has at least one neighbor $u' \in S$ in the graph G' . Now, v and u' are not adjacent in G (Remark 1). Since G' only adds edges among vertices in S , v is still not adjacent to u' in G' . Therefore, u is no longer properly suppressed by

v in G' , and the suppression pairs $(v, u_1), (v, u_2), \dots, (v, u_p)$ are removed from the suppression set in G' .

It remains to show that G' introduces no new suppression pairs; that is, there exists no pair (p, q) that is present in G' but not in G .

Let $e_1, e_2, \dots, e_\ell$ be the edges present in G' but not in G , listed in an arbitrary order. Define $G_0 = G$, and for $1 \leq i \leq \ell$, let

$$G_i = G_{i-1} \cup \{e_i\}.$$

Consider adding the edge $e_i = (u, w)$ to G_{i-1} . Suppose that this insertion introduces some new suppression pair (p, q) in G_i that was not present in G_{i-1} . Then, by Lemma 24, either $p = u$ and q is adjacent to w in G_{i-1} , or $p = w$ and q is adjacent to u in G_{i-1} . Without loss of generality, assume that $p = u$.

Observe that

$$N_{G_i}(u) = N_G(u) \cup S', \quad \text{and} \quad N_{G_i}(q) = N_G(q) \cup S'',$$

where $S', S'' \subseteq S$. Moreover, $N_G(u)$ and $N_G(q)$ are disjoint from S . Therefore, if

$$N_{G_i}(q) \subseteq N_{G_i}(u),$$

then it must be the case that

$$N_G(q) \subseteq N_G(u).$$

This implies that (p, q) was already a suppression pair in G .

Since this argument holds for every edge e_i added in order, it follows that no genuinely new suppression pairs are introduced in G' . $\square$

Using Lemmas 23 and 27, we can remove a suppression pair from G without introducing any new suppression pairs using Algorithm 4.

Let Algorithm 4 run for k iterations of the loop in Line 3. Define $S_0 = \{(u, v)\}$ and, for $i \geq 1$, let

$$S_i = N_{\text{supp}}(G_{i-1}, v_{i-1}, v_i)$$

denote the set of suppression pairs present in G_i but not in G_{i-1} —i.e., the suppression pairs introduced after the i th iteration.

At iteration i of the loop, the algorithm removes the suppression pairs in S_{i-1} and introduces the suppression pairs in S_i . More formally,

$$\text{Supp}(G_i) \subseteq \text{Supp}(G_{i-1})/S_{i-1} \cup S_i \subseteq \text{Supp}(G) \cup S_i, \quad \text{for } 1 \leq i \leq k.$$

Finally, after the k th iteration, the algorithm removes all remaining suppression pairs introduced in the last step, i.e., the suppression pairs in S_k , as described in Line 10, using Lemma 27.

Algorithm 4 Remove Suppression Pair from a Graph (version 1)**Require:** A non-UNN G and a suppression pair (u, v) of G **Ensure:** A graph H such that $\text{Supp}(H) \subseteq \text{Supp}(G) \setminus \{(u, v)\}$

```

1: procedure REMOVESUPPRESSIONPAIR( $G, u, v$ )
2:   Initialize  $G_0$  with  $G$ ,  $v_0$  with  $u$  and  $i$  with 0.
3:   while  $v_i$  properly suppresses more than one vertex in  $G_i$  do
4:     Let  $v_{i+1}$  be the vertex that  $v_i$  properly suppresses in  $G_i$ .
5:     Initialize  $G_{i+1}$  as  $G_i \cup \{(v_i, v_{i+1})\}$ .
6:     Increment  $i$ .
7:   end while
8:   Let  $S$  be the set of proper suppressors of  $v_i$  in  $G_i$ .
9:   if  $v_i$  has more than one proper suppressor then
10:    Let  $H$  be a graph that resulted from arbitrarily inserting edges between vertices in  $S$ 
    in  $G_i$  in such a manner that every vertex in  $S$  has at least one neighbor in  $S$ .
11:   end if
12:   return  $H$ 
13: end procedure

```

Thus, Algorithm 4 removes the suppression pair (u, v) from G without introducing any new suppression pairs.

6.3 An Algorithm for the UNN Completion Problem

Algorithm 4 provides a general framework for solving the UNN Completion problem. By repeatedly applying Algorithm 4, we can eliminate all suppression pairs from a graph, thereby obtaining a UNN completion, as formalized in Algorithm 5.

Algorithm 5 Find a UNN Completion of a Graph (version 1)**Require:** A graph G **Ensure:** a UNN Completion G' of G

```

1: Initialize  $H_0$  with  $G$  and  $i$  with 0.
2: while  $\text{Supp}(H_i)$  is not empty do
3:   Pick an arbitrary suppression pair  $(u, v)$  in  $H_i$ .
4:   Initialize  $H_{i+1}$  as REMOVESUPPRESSIONPAIR( $H_i, u, v$ ).
5:   Increment  $i$ .
6: end while
7: Let  $G' = H_i$ 
8: return  $G'$ 

```

It is important to note that Algorithm 5 does not constitute a complete algorithm on its own; it serves only as a framework. Applying it directly can yield a UNN completion, but the resulting approximation ratio may be poor. To construct a more effective approximation algorithm, we will introduce the concept of a *suppressor-partitioned graph*.

Furthermore, to analyze and justify the performance of our final algorithm, we introduce the notion of a *recovery patch*. This concept will play a key role in demonstrating the correctness and quality of the refined algorithm.

6.3.1 Recovery Patch

To derive an upper bound on the cost of Algorithm 5, we analyze the behavior of the REMOVE SUPPRESSION PAIR procedure from Algorithm 4.

Consider executing this procedure on a graph G with a suppression pair (u, v) , where u properly suppresses only v . If the procedure performs k iterations of its main loop, it generates a sequence of vertices $v_0, v_1, \dots, v_k$, which we call a *recovery sequence*.

During execution, let S denote the set maintained by the algorithm at Line 10. We refer to this set as the *recovery tail* corresponding to the recovery sequence $v_0, v_1, \dots, v_k$. The pair consisting of a recovery sequence and its corresponding recovery tail is called a *recovery patch*.

Formally, let G be a graph. Let $S_1 = (v_0, v_1, \dots, v_k)$ be a sequence of vertices of G , and let $S_2 = \{u_1, \dots, u_p\}$ be a set of vertices.

Then consider the following sequence of graphs:

$$\begin{aligned} G_0 &= G, \\ G_i &= G_{i-1} \cup \{(v_{i-1}, v_i)\}, \quad 1 \leq i \leq k. \end{aligned}$$

We say that (S_1, S_2) is a *recovery patch* of G if the following conditions hold:

1. $k \geq 1$.
2. v_0 properly suppresses v_1 in G_0 , i.e., $v_0 \succeq_{G_0} v_1$ and $v_0 \neq v_1$.
3. For all $0 \leq i \leq k - 2$, either

$$N_{\text{supp}}(G_i, v_i, v_{i+1}) = \{(v_{i+1}, v_{i+2})\},$$

or G_{i+1} is not a simple graph.

4. Either

$$N_{\text{supp}}(G_k, v_{k-1}, v_k) = \{(v_k, u_1), \dots, (v_k, u_p)\},$$

or G_k is not a simple graph.

5. $p \neq 1$.
6. For all $v \in V(G) \setminus \{v_0, v_1\}$, v_0 does not suppress v in G_0 .

In this case, S_1 is called a *recovery sequence*, and S_2 is called the *recovery tail* corresponding to S_1 .

Although the definition allows intermediate graphs G_i to be non-simple, this situation does not arise in our setting. This will be established in Lemma 30, using Lemmas 28 and 29.

Lemma 28. *Let G be a graph and let $u, v, w \in V(G)$ such that u properly suppresses v in G , but v does not properly suppress w in G . If v properly suppresses w in $G' = G \cup \{(u, v)\}$, then u and w are adjacent in G .*

Proof. Since u properly suppresses v in G , we have $u \neq v$ and $(u, v) \notin E(G)$. If $v \succeq_{G'} w$, then $v \neq w$ and $(v, w) \notin E(G')$. Since $(u, v) \in E(G')$, we must have $u \neq w$.

Since $v \succeq_{G'} w$, we have

$$N_{G'}(w) = N_G(w) \subseteq N_{G'}(v) = N_G(v) \cup \{u\}.$$

However, since v does not properly suppress w in G , we have

$$N_G(w) \not\subseteq N_G(v).$$

Thus, the only way the inclusion can hold in G' is if $u \in N_G(w)$. Therefore, u and w are adjacent in G . $\square$

Lemma 29. *Let G be a graph and let $S_1 = (v_0, v_1, \dots, v_j)$ be a sequence of vertices of G .*

Define

$$\begin{aligned} G_0 &= G, \\ G_i &= G_{i-1} \cup \{(v_{i-1}, v_i)\}, \quad 1 \leq i \leq j-1. \end{aligned}$$

If the following conditions hold, then $v_j \notin \{v_0, v_1, \dots, v_{j-1}\}$:

1. *The vertices $v_0, v_1, \dots, v_{j-1}$ are pairwise distinct.*
2. *The graphs $G_0, G_1, \dots, G_{j-1}$ are simple.*
3. *v_0 properly suppresses v_1 in G_0 .*
4. *$(v_{i+1}, v_{i+2}) \in N_{\text{supp}}(G_i, v_i, v_{i+1})$ for all $0 \leq i \leq j-2$.*

Proof. Assume for contradiction that $v_j = v_i$ for some $i < j$. Consider the subsequence $(v_i, v_{i+1}, \dots, v_j)$.

If $j - i \geq 4$, then $v_{i+1} \neq v_{j-2}$. From condition (4), applied at index $j-2$, we have

$$v_{j-1} \succeq_{G_{j-1}} v_j \quad \text{and} \quad v_j = v_i.$$

Hence,

$$v_{j-1} \succeq_{G_{j-1}} v_i.$$

Therefore,

$$N_{G_{j-1}}(v_i) \subseteq N_{G_{j-1}}(v_{j-1}).$$

We now expand both sides.

Between G_i and G_{j-1} , the only edge added incident to v_i is (v_i, v_{i+1}) . Thus,

$$N_{G_{j-1}}(v_i) = N_{G_i}(v_i) \cup \{v_{i+1}\}.$$

Similarly, between G_i and G_{j-1} , the only edge added incident to v_{j-1} is (v_{j-2}, v_{j-1}) . Hence,

$$N_{G_{j-1}}(v_{j-1}) = N_{G_i}(v_{j-1}) \cup \{v_{j-2}\}.$$

Combining the above, we obtain

$$N_{G_i}(v_i) \cup \{v_{i+1}\} \subseteq N_{G_i}(v_{j-1}) \cup \{v_{j-2}\}.$$

If $j - i \geq 4$, then $v_{i+1} \neq v_{j-2}$. Therefore,

$$v_{i+1} \in N_{G_i}(v_{j-1}).$$

Since v_i properly suppresses v_{i+1} in G_i , it follows that v_i is adjacent to v_{j-1} in G_i , and hence in G_{j-1} . This contradicts Remark 1, because v_{j-1} cannot suppress $v_j = v_i$ in G_j .

We now consider the remaining cases:

- If $i = 0$ and $j = 1$, then $v_0 \neq v_1$ since v_0 properly suppresses v_1 .
- If $i \neq 0$ and $j = i + 1$, then $v_i \neq v_{i+1}$. This must be the case because $N_{\text{supp}}(G_{i-1}, v_{i-1}, v_i) \in \{(v_i, v_{i+1})\}$.
- If $j = i + 2$, then by Lemma 28, v_i is adjacent to v_{i+2} in G_i , contradicting simplicity if $v_i = v_{i+2}$.
- If $j = i + 3$, then by Lemma 28, v_i is adjacent to v_{i+2} . Since v_{i+2} suppresses $v_{i+3} = v_i$ in G_{i+2} , this contradicts Remark 1.

In all cases, we obtain a contradiction. Therefore, v_j is distinct from $v_0, \dots, v_{j-1}$. □

Lemma 30. *Let G be a graph and (S_1, S_2) be a recovery patch of G where $S_1 = v_0v_1 \dots v_k$. Define*

$$\begin{aligned} G_0 &= G, \\ G_i &= G_{i-1} \cup \{(v_{i-1}, v_i)\}, \quad 1 \leq i \leq k. \end{aligned}$$

Then the following holds:

1. *The vertices of the recovery sequence S_1 are pairwise distinct.*
2. *Every graph G_i , $0 \leq i \leq k$, is a simple graph with no self-loops.*

Proof. We prove the lemma by induction on i , $0 \leq i \leq k$.

Base case ($i = 1$): Since v_0 properly suppresses v_1 in G , we have $v_0 \neq v_1$ and $(v_0, v_1) \notin E(G)$. Therefore, G_1 is a simple graph with no self-loops.

Inductive hypothesis: Assume that for some $l < k$, the vertices $v_0, v_1, \dots, v_l$ are pairwise distinct and the graphs $G_0, G_1, \dots, G_l$ are simple with no self-loops.

Inductive step: By Lemma 29, $v_{l+1} \notin \{v_0, v_1, \dots, v_l\}$. Since G_l is a simple graph with no self-loops and v_l properly suppresses v_{l+1} in G_l , we have $v_l \neq v_{l+1}$ and $(v_l, v_{l+1}) \notin E(G_l)$. Hence, $G_{l+1} = G_l \cup \{(v_l, v_{l+1})\}$ is a simple graph with no self-loops.

Conclusion: By induction, the vertices $v_0, v_1, \dots, v_k$ are pairwise distinct, and the graphs $G_0, G_1, \dots, G_k$ are all simple with no self-loops. $\square$

From Lemma 30, it follows that the informal and formal definitions of a recovery patch coincide. We can use this to construct an algorithm to find a recovery patch, as shown in Algorithm 6.

Algorithm 6 Find Recovery Patch in a non-UNN from Suppression Pair

Require: A non-UNN G and a suppression pair (u, v) of G such that u only properly suppresses v in G

Ensure: A recovery patch (S_1, S_2) of G that such that S_1 starts with uv

```

1: procedure FINDRECOVERYPATCH( $G, u, v$ )
2:   Initialize  $G_0$  with  $G$ ,  $v_0$  with  $u$  and  $i$  with 0
3:   while  $v_i$  suppresses more than one vertex in  $G_i$  do
4:     Let  $v_{i+1}$  be the vertex that  $v_i$  suppresses in  $G_i$ 
5:     Let  $G_{i+1}$  be the graph resulting from adding a link between  $v_i$  and  $v_{i+1}$  in  $G_i$ .
6:     increment  $i$ 
7:   end while
8:   Let  $S_1$  be  $v_0v_1 \dots v_k$ 
9:   Let  $S_2$  be the set of vertices  $v_k$  suppress in  $G_k$ 
10:  return  $(S_1, S_2)$ 
11: end procedure

```

Studying the structure and properties of recovery patches is crucial to analyzing the cost of our approximation algorithm.

6.3.2 Structure of a Recovery Patch

In this subsection, we study the structural properties of a recovery patch. In particular, we show:

- A recovery sequence is densely connected: each vertex is adjacent to all other vertices in the sequence except itself and its immediate neighbors (Lemma 31).
- The recovery sequence and its corresponding recovery tail are disjoint (Lemma 32) and densely connected to each other (Lemma 33).
- The recovery sequence exhibits a clear inductive structure (Lemma 34).
- The recovery tail contains exactly the vertices that the last vertex of the recovery sequence, v_k , properly suppresses in G_k (Lemma 35).

Lemma 31. *Let G be a graph and (S_1, S_2) a recovery patch of G , with recovery sequence $S_1 = v_0 v_1 \dots v_k$. Let v_i be an arbitrary vertex in the recovery sequence. Then, for every $j \geq i + 2$, v_i is adjacent to v_j in G .*

Proof. We prove the claim by induction on i , counting backwards along the sequence.

Base case: If $i \geq k - 1$, there is no $j \geq i + 2$, so there is nothing to prove.

Inductive step: Assume that for some $i > 0$, the vertex v_i is adjacent to every v_j with $j \geq i + 2$. We show that v_{i-1} is adjacent to every v_j with $j \geq i + 1$.

Let

$$G_0 = G, \quad G_i = G_{i-1} \cup \{(v_{i-1}, v_i)\} \quad \text{for } 1 \leq i \leq k.$$

By definition of the recovery sequence, v_{i-1} properly suppresses v_i in G_{i-1} . Therefore, any neighbor of v_i in G_{i-1} is also a neighbor of v_{i-1} in G_{i-1} .

From the inductive hypothesis, each v_j with $j \geq i + 2$ is adjacent to v_i in G , and hence also in G_{i-1} . By Lemma 28, v_{i-1} is adjacent to v_{i+1} in G_{i-1} .

Finally, by Lemma 30, all vertices $v_0, \dots, v_k$ are distinct. Therefore, $N_{G_{i-1}}(v_{i-1}) = N_G(v_{i-1}) \cup \{v_{i-2}\}$ if $i - 1 > 0$, and $N_{G_{i-1}}(v_{i-1}) = N_G(v_{i-1})$ otherwise. Thus, even if $i - 1 > 0$, adding v_{i-2} to the neighborhood does not affect adjacency with v_j for $j \geq i + 1$. Hence, v_{i-1} is adjacent to all v_j with $j \geq i + 1$ in G .

By induction, the claim holds for all v_i in the recovery sequence. □

A consequence of Lemma 31 is that the recovery sequence and the recovery tail of any recovery patch are disjoint.

Lemma 32. *Let G be a graph and (S_1, S_2) be a recovery patch of G . Then no vertex of the recovery tail appears in the recovery sequence, i.e.,*

$$S_1 \cap S_2 = \emptyset.$$

Proof. Let $S_1 = v_0v_1 \dots v_k$ be the recovery sequence, and define

$$G_0 = G, \quad G_i = G_{i-1} \cup \{(v_{i-1}, v_i)\}, \quad 1 \leq i \leq k.$$

From the definition of a recovery patch, for every $u \in S_2$, the vertex v_k properly suppresses u in G_k .

By Lemma 31, every vertex v_j with $0 \leq j \leq k-2$ is adjacent to v_k in G , and hence also in G_k . Furthermore, v_{k-1} is adjacent to v_k in G_k by construction.

Thus, every vertex in the recovery sequence S_1 is adjacent to v_k in G_k . Since proper suppression requires non-adjacency, v_k cannot properly suppress any vertex in S_1 .

Therefore, no vertex $u \in S_2$ can belong to S_1 , and hence

$$S_1 \cap S_2 = \emptyset.$$

□

Lemma 33. *Let G be a graph and let (S_1, S_2) be a recovery patch of G , where $S_1 = v_0v_1 \dots v_k$ and $S_2 = \{u_1, u_2, \dots, u_p\}$. Let $v_i \neq v_k$ be an arbitrary vertex in the recovery sequence. Then v_i is adjacent to every vertex $u \in S_2$ in G .*

Proof. We prove this by induction on i .

Base case: Let $i = k-1$. By Lemma 28, v_{k-1} is adjacent to every vertex $u \in S_2$ in G_{k-1} .

Now observe that

$$N_{G_{k-1}}(v_{k-1}) = \begin{cases} N_G(v_{k-1}) \cup \{v_{k-2}\}, & \text{if } k-2 \geq 0, \\ N_G(v_{k-1}), & \text{otherwise.} \end{cases}$$

If $k-2 < 0$, then $G_{k-1} = G$, and hence v_{k-1} is adjacent to every $u \in S_2$ in G .

If $k-2 \geq 0$, then by Lemma 32, we have $u \neq v_{k-2}$ for all $u \in S_2$. Therefore, all adjacencies between v_{k-1} and u already exist in G . Hence, v_{k-1} is adjacent to every $u \in S_2$ in G .

Inductive step: Assume that for some $i < k$, the vertex v_i is adjacent to every vertex $u \in S_2$ in G . We show that v_{i-1} is also adjacent to every vertex $u \in S_2$ in G .

Since $v_{i-1} \succeq_{G_{i-1}} v_i$, every neighbor of v_i in G_{i-1} is also a neighbor of v_{i-1} in G_{i-1} . By the inductive hypothesis, every $u \in S_2$ is adjacent to v_i in G , and hence also in G_{i-1} . Therefore, v_{i-1} is adjacent to every $u \in S_2$ in G_{i-1} .

Finally, since S_1 and S_2 are disjoint, none of these adjacencies involve edges added during the construction of the sequence. Hence, these adjacencies already exist in G , and v_{i-1} is adjacent to every $u \in S_2$ in G .

This completes the proof. □

Lemma 34. *Let G be a graph and let (S_1, S_2) be a recovery patch of G , where $S_1 = v_0v_1 \dots v_k$ and $S_2 = \{u_1, u_2, \dots, u_p\}$. Define*

$$\begin{aligned} G_0 &= G, \\ G_i &= G_{i-1} \cup \{(v_{i-1}, v_i)\}, \quad 1 \leq i \leq k. \end{aligned}$$

Then for any $i < k$, the pair $(v_iv_{i+1} \dots v_k, S_2)$ is a recovery patch of G_i .

Proof. We prove the claim by induction on i .

Base case: For $i = 0$, the statement holds trivially since (S_1, S_2) is a recovery patch of $G = G_0$.

Inductive step: Assume that for some $l < k$, the pair $(v_lv_{l+1} \dots v_k, S_2)$ is a recovery patch of G_l . We show that $(v_{l+1}v_{l+2} \dots v_k, S_2)$ is a recovery patch of G_{l+1} , provided $l + 1 < k$.

We verify that all six conditions of a recovery patch hold:

1. **Length:** The sequence $v_{l+1} \dots v_k$ has length $k - (l + 1) + 1 = k - l$, which is greater than 1 since $l + 1 < k$.
2. **Initial suppression:** From the inductive hypothesis,

$$N_{\text{supp}}(G_l, v_l, v_{l+1}) = \{(v_{l+1}, v_{l+2})\}.$$

Since $G_{l+1} = G_l \cup \{(v_l, v_{l+1})\}$, the suppression relation (v_{l+1}, v_{l+2}) holds in G_{l+1} , and hence v_{l+1} properly suppresses v_{l+2} in G_{l+1} .

3. **Intermediate suppression structure:** Since (S_1, S_2) is a recovery patch of G , we have

$$N_{\text{supp}}(G_i, v_i, v_{i+1}) = \{(v_{i+1}, v_{i+2})\}$$

for all relevant indices. In particular, this holds for all $l + 1 \leq i \leq k - 2$.

4. **Final suppression:** By definition of a recovery patch,

$$N_{\text{supp}}(G_{k-1}, v_{k-1}, v_k) = \{(v_k, u_1), \dots, (v_k, u_p)\}.$$

This remains unchanged in the suffix.

5. **Tail size:** Since (S_1, S_2) is a recovery patch, we have $|S_2| \neq 1$. This condition is unchanged.
6. **Uniqueness of suppression:** We show that v_{l+1} does not suppress any vertex other than v_{l+2} in G_{l+1} .

From the inductive hypothesis,

$$v_l \succeq_{G_l} v_{l+1}, \quad \text{and} \quad N_{\text{supp}}(G_l, v_l, v_{l+1}) = \{(v_{l+1}, v_{l+2})\}.$$

Suppose for contradiction that $v_{l+1} \succeq_{G_{l+1}} u$ for some vertex $u \neq v_{l+1}$. Then

$$N_{G_{l+1}}(u) \subseteq N_{G_{l+1}}(v_{l+1}) = N_{G_l}(v_{l+1}) \cup \{v_l\}.$$

Note that, since v_l and v_{l+1} are adjacent in G_{l+1} , $u \neq v_l$. Therefore,

$$N_{G_{l+1}}(u) = N_{G_l}(u).$$

Case 1: $v_l \notin N_{G_l}(u)$. Then

$$N_{G_{l+1}}(u) = N_{G_l}(u) \subseteq N_{G_l}(v_{l+1}),$$

so $v_{l+1} \succeq_{G_l} u$. Since $v_l \succeq_{G_l} v_{l+1}$, we obtain $v_l \succeq_{G_l} u$. By the inductive hypothesis, this is only possible if $u \in \{v_l, v_{l+1}\}$. However, u is neither, a contradiction.

Case 2: $v_l \in N_{G_l}(u)$. Then $v_l \in N_{G_{l+1}}(u)$, but $v_l \notin N_{G_l}(v_{l+1})$, since $v_l \succeq_{G_l} v_{l+1}$. Thus, (v_{l+1}, u) cannot be a suppression pair in G_l , and must arise in G_{l+1} .

Therefore,

$$(v_{l+1}, u) \in N_{\text{supp}}(G_l, v_l, v_{l+1}).$$

By the inductive hypothesis,

$$N_{\text{supp}}(G_l, v_l, v_{l+1}) = \{(v_{l+1}, v_{l+2})\},$$

so $u = v_{l+2}$.

Thus, v_{l+1} only suppresses v_{l+2} in G_{l+1} .

Hence, $(v_{l+1} \dots v_k, S_2)$ is a recovery patch of G_{l+1} .

This completes the proof. □

Lemma 35. Let G be a graph and let (S_1, S_2) be a recovery patch of G , where $S_1 = v_0 v_1 \dots v_k$ and $S_2 = \{u_1, u_2, \dots, u_p\}$. Let

$$\begin{aligned} G_0 &= G, \\ G_i &= G_{i-1} \cup \{(v_{i-1}, v_i)\}, \quad 1 \leq i \leq k. \end{aligned}$$

Then v_k properly suppresses a vertex v in G_k if and only if $v \in S_2$.

Proof. From the definition of a recovery patch, we have

$$N_{\text{supp}}(G_{k-1}, v_{k-1}, v_k) = \{(v_k, v) \mid v \in S_2\}.$$

Hence, if $v \in S_2$, then v_k properly suppresses v in G_k .

It remains to show that v_k does not properly suppress any vertex outside S_2 in G_k . Suppose, for contradiction, that there exists a vertex $u \notin S_2$ such that $v_k \succeq_{G_k} u$.

Since $(v_k, u) \notin N_{\text{supp}}(G_{k-1}, v_{k-1}, v_k)$, it follows that $v_k \succeq_{G_{k-1}} u$. Because $v_{k-1} \succeq_{G_{k-1}} v_k$, we obtain

$$v_{k-1} \succeq_{G_{k-1}} u.$$

By Lemma 34, the pair $(v_{k-1}v_k, S_2)$ is a recovery patch of G_{k-1} . Therefore, v_{k-1} can only properly suppress vertices in $\{v_k\}$ within this structure.

Thus, $u = v_k$. However, a vertex cannot properly suppress itself, which is a contradiction.

Therefore, v_k does not properly suppress any vertex outside S_2 in G_k . $\square$

6.3.3 Relating Suppression Pairs To Recovery Patches

In this subsection, we study how a suppression pair can interact with a recovery patch. Namely, we describe in what ways the members of a suppression pair can participate in a recovery patch.

Let (s, t) be a suppression pair of a graph G , and let (S_1, S_2) be a recovery patch of G with $S_1 = v_0v_1 \dots v_k$. Define $G_0 = G$ and, for $1 \leq i \leq k$, let $G_i = G_{i-1} \cup \{(v_{i-1}, v_i)\}$. Then we show that:

1. If s is a member of the recovery sequence S_1 , then there are only two possibilities. Either $s = v_0$ and $t = v_1$, or $s = v_k$, $t = v_{k-1}$, s and t are twins in G_{k-1} , and S_2 is empty (Lemma 36).
2. If s is a member of the recovery tail S_2 , then either $t = v_k$ and s and t are twins in G_k , or t is also a member of S_2 (Lemma 37).

Lemma 36. *Let G be a graph and let (S_1, S_2) be a recovery patch of G with $S_1 = v_0v_1 \dots v_k$. Define $G_0 = G$ and, for $1 \leq i \leq k$, let*

$$G_i = G_{i-1} \cup \{(v_{i-1}, v_i)\}.$$

Let (s, t) be a suppression pair in G . If s is in S_1 , then either:

- (a) $s = v_0$ and $t = v_1$.
- (b) $s = v_k$, $t = v_{k-1}$, s and t are twins in G_{k-1} , and S_2 is empty.

Proof. If $s = v_0$, then by the definition of a recovery patch, s can only properly suppress v_1 in G . Therefore, t must be v_1 .

On the other hand, if $s \neq v_0$, then $s = v_i$ for some $i > 0$.

By the definition of a recovery sequence, $v_{i-1} \succeq_{G_{i-1}} v_i$. Now, if $t \neq v_j$ for any $j < i$, then

$$N_{G_{i-1}}(t) = N_G(t) \subseteq N_G(s) = N_G(v_i) = N_{G_{i-1}}(v_i) \subseteq N_{G_{i-1}}(v_{i-1}).$$

By Lemma 34, $(v_{i-1}v_i \dots v_k, S_2)$ is a recovery patch of G_{i-1} . Hence, v_{i-1} can only properly suppress v_i in G_{i-1} , which is a contradiction. Therefore, $t = v_j$ for some $j < i$.

Now, by Lemma 31, if $j < i - 1$, then t and s would be adjacent, which is not possible. Hence, $t = v_{i-1}$.

Analyzing neighborhoods, we have

$$N_G(s) = N_{G_{i-1}}(s) \subseteq N_{G_{i-1}}(t) = N_G(t) \cup \{v_{i-2}\},$$

which implies that s and t are twins in G_{i-1} . Then, by Lemma 25, it follows that $s = v_k$, $t = v_{k-1}$, and S_2 is empty. $\square$

Lemma 37. *Let G be a graph and let (S_1, S_2) be a recovery patch of G with $S_1 = v_0v_1 \dots v_k$. Let (s, t) be a suppression pair in G . Define $G_0 = G$ and, for $1 \leq i \leq k$, let*

$$G_i = G_{i-1} \cup \{(v_{i-1}, v_i)\}.$$

If $s \in S_2$, then either $t \in S_2$, or $t = v_k$ and s and t are twins in G_k .

Proof. We prove the claim by considering two cases.

Case 1 (t is a member of the recovery sequence): Let $t = v_j$ for some j . Since $s \in S_2$ and s suppresses t in G , they cannot be adjacent in G (Remark 1). However, if $j < k$, then by Lemma 33, $t = v_j$ is adjacent to s , which is a contradiction. Thus, $t = v_k$.

Furthermore, we know that $v_k \succeq_{G_k} s$. Hence,

$$N_{G_k}(s) = N_G(s) \subseteq N_{G_k}(v_k) = N_G(v_k) \cup \{v_{k-1}\}.$$

By Lemma 33, $v_{k-1} \in N_G(s)$. We also know that $N_G(v_k) \subseteq N_G(s)$. Therefore,

$$N_{G_k}(s) = N_G(s) = N_G(v_k) \cup \{v_{k-1}\} = N_{G_k}(v_k),$$

and hence s and t are twins in G_k .

Case 2 (t is not a member of the recovery sequence): Since $t \notin S_1$, we have $N_{G_k}(t) = N_G(t)$. Similarly, since $s \in S_2$, we have $N_{G_k}(s) = N_G(s)$. Thus, s continues to properly suppress t in G_k .

This implies that v_k suppresses t in G_k . Since $t \notin S_1$, we have $t \neq v_k$, and hence v_k properly suppresses t . By Lemma 35, this can only happen if $t \in S_2$. $\square$

6.3.4 Suppressor Partitioned Graphs

Lemma 31 tells us that the number of iterations of the loop in algorithm 4 on an input graph G and suppression pair (u, v) is at most $\deg_G(u) + 1$. However, analyzing the number of edge insertions in Algorithm 4 is more subtle. Specifically, line 10 does not specify how edges are added between vertices in S , making it difficult to count the exact number of insertions. Nevertheless, at least $\lceil |S|/2 \rceil$ edges must be inserted.

The analysis becomes more challenging because adding edges to remove a suppression pair increases the degrees of some vertices. If there are a linear number of suppressors, this increase can cause the number of edge insertions to grow quadratically in the number of vertices.

To simplify the analysis, we introduce the notion of a *suppressor partitioned graph*.

Definition 6.3.1 (Suppressor Partitioned Graph). A graph G is called a suppressor partitioned graph if it satisfies the following conditions:

1. Each suppressor in G suppresses exactly one vertex other than itself.
2. G contains no twins.

We immediately observe the following property:

remark 3. *Let G be a suppressor partitioned graph. Then no two suppressed vertices in G share the same proper suppressor.*

We will also need the following property:

Lemma 38. *Let u be a suppressed vertex in a suppressor partitioned graph G . Then u suppresses no vertex other than itself.*

Proof. Suppose, for contradiction, that there exists a vertex $v \neq u$ such that $u \succeq_G v$. Since u is suppressed, there exists a vertex $w \neq u$ such that $w \succeq_G u$, and thus $w \succeq_G v$. Because G is suppressor partitioned and has no twins, we have $w \neq v$. But then w is a common proper suppressor of both u and v , contradicting Remark 3. $\square$

If a graph G has k suppressed vertices, it can be converted into a suppressor partitioned graph by adding at most $\binom{k}{2}$ vertices using Algorithm 7.

By Lemma 27, $\text{Supp}(H_{i+1}) \subseteq \text{Supp}(H_i)$ in Algorithm 7. Therefore, Algorithm 7 only adds edges between suppressed vertices in G . Consequently, if G initially has k suppressed vertices, at most $\binom{k}{2}$ edges are added.

Algorithm 7 Make Graph Suppressor Partitioned**Require:** A graph G **Ensure:** A graph H such that H is a spanning supergraph of G and is suppressor partitioned

```

1: procedure PARTITIONSUPPRESSORS( $G$ )
2:   Initialize  $H_0$  as  $G$  and  $i$  as 0
3:   Let  $S_0$  denote the set of suppressors in  $H_0$  that properly suppresses more than one vertex
4:   while  $S_i$  is not empty do
5:     Let  $v$  be an arbitrary suppressor in  $S_i$  and let  $u_1, u_2, \dots, u_p$  be the vertices that  $v$ 
       properly suppresses in  $H_i$ .
6:     Let  $H_{i+1} = H_i \cup \{(u_1, u_2), (u_2, u_3), \dots, (u_{p-1}, u_p)\}$ .
7:     Let  $S_{i+1}$  denote the set of suppressors in  $H_{i+1}$  that properly suppresses more than
       one vertex.
8:     Increment  $i$ .
9:   end while
10:  Let  $H$  be  $H_i$ .
11:  return  $H$ 
12: end procedure

```

The main advantage of working with a suppressor partitioned graph is that once a suppressor appears in a recovery patch (either in the recovery sequence or the recovery tail) and has been applied, it no longer acts as a suppressor. This property greatly simplifies our analysis.

6.3.5 Resolving Recovery Tail Suppression Pairs

One issue with our algorithm for removing a suppression pair (Algorithm 4) is that it does not specify how edges are added between vertices in the recovery tail. If there are other suppressors in the recovery tail and edges are added carelessly, their degrees may increase, which complicates the analysis. To address this, we divide the vertices of a recovery tail in a suppressor partitioned graph into two categories and add edges between them carefully to maintain a good approximation ratio.

Let (S_1, S_2) be a recovery patch in a suppressor partitioned graph, where $S_1 = v_0v_1\dots v_k$. Define

$$G_0 = G, \quad G_i = G_{i-1} \cup (v_{i-1}, v_i), \quad 1 \leq i \leq k.$$

We partition the vertices of S_2 as follows:

- **Type A:** A vertex $u \in S_2$ is of type A if it properly suppresses some vertex $u' \in S_2$ in G_k . Furthermore, for a type A vertex u , the vertex u' is unique; no other vertex in S_2 is properly suppressed by u in G_k (Lemma 40).
- **Type B:** A vertex $u \in S_2$ is of type B if it does not properly suppress any vertex in G_k . Any vertex in S_2 that is not of type A must therefore be of type B .

To ensure that this classification is well-defined, we must show that each vertex in S_2 belongs to exactly one type. This is established in Lemma 41, using Lemmas 37 and 39.

Lemma 39. *Let G be a suppressor partitioned graph, and let (S_1, S_2) be a recovery patch where $S_1 = v_0 v_1 \dots v_k$. Let*

$$G_0 = G, \quad G_i = G_{i-1} \cup \{(v_{i-1}, v_i)\}, \quad 1 \leq i \leq k.$$

Then there exists no vertex $u \in V(G)$ such that u is a twin of v_k in G_k .

Proof. Suppose, for contradiction, that there exists a vertex $u \in V(G)$ such that u is a twin of v_k in G_k .

By definition of twins in G_k , we have

$$N_{G_k}(u) = N_{G_k}(v_k).$$

Therefore, v_k suppresses u in G_k .

By Lemma 35, any vertex properly suppressed by v_k in G_k must be in the recovery tail S_2 . Therefore, $u \in S_2$.

Observe that

$$N_{G_k}(v_k) = N_G(v_k) \cup \{v_{k-1}\} = N_{G_k}(u).$$

By Lemma 32, $N_{G_k}(u) = N_G(u)$. Therefore, u properly suppresses v_k in G .

Since $u \in S_2$ and $|S_2| \geq 2$, there exists a vertex $u' \in S_2$ with $u' \neq u$ such that $v_k \succeq_{G_k} u'$. Then

$$N_G(u') = N_{G_k}(u') \subseteq N_{G_k}(v_k) = N_{G_k}(u) = N_G(u).$$

Hence, u is a common proper suppressor of v_k and u' in G , which contradicts the fact that G is suppressor partitioned.

Therefore, no such u can exist. □

Lemma 40. *Let G be a suppressor partitioned graph and let (S_1, S_2) be a recovery patch of G where $S_1 = v_0 v_1 \dots v_k$. Define $G_0 = G$ and, for $1 \leq i \leq k$, let*

$$G_i = G_{i-1} \cup \{(v_{i-1}, v_i)\}.$$

Let u be a vertex in S_2 . If u suppresses some vertex u' in G_k , then $u' \in S_2$, and u suppresses no other vertex in G_k besides u' .

Proof. **Claim 1:** $u' \in S_2$. By Lemma 33, every vertex of the recovery sequence other than v_k is adjacent to u . Thus, if u' is in S_1 , then $u' = v_k$. In that case, u and v_k would be twins in G_k , which contradicts Lemma 39. Hence, u' is not in the recovery sequence.

Since u suppresses u' in G_k and $u' \neq v_k$, it follows that v_k also suppresses u' in G_k . By Lemma 35, this implies that $u' \in S_2$.

Claim 2: u suppresses no other vertex in G_k . Observe that $N_{G_k}(u) = N_G(u)$ and $N_{G_k}(u') = N_G(u')$, so u also properly suppresses u' in G .

Suppose, for contradiction, that u suppresses another vertex $v \neq u'$ in G_k . Then

$$N_{G_k}(v) \subseteq N_{G_k}(u) = N_G(u).$$

Since $N_G(v) \subseteq N_{G_k}(v)$, it follows that u properly suppresses v in G as well.

But this contradicts the fact that G is suppressor partitioned, since u would properly suppress both u' and v in G . Therefore, no such vertex v exists. $\square$

Lemma 41. *Let G be a suppressor partitioned graph and let (S_1, S_2) be a recovery patch of G where $S_1 = v_0v_1 \dots v_k$. Let $G_0 = G$ and, for $1 \leq i \leq k$, let*

$$G_i = G_{i-1} \cup \{(v_{i-1}, v_i)\}.$$

Then for any vertex $u \in S_2$, exactly one of the following holds:

- (a) u properly suppresses exactly one vertex u' in G_k , and $u' \in S_2$.
- (b) u does not properly suppress any vertex in G_k .

Proof. This follows directly from Lemma 40. If $u \in S_2$ suppresses some vertex u' in G_k , then by that lemma, $u' \in S_2$ and u suppresses no other vertex in G_k , which gives (a). Otherwise, u suppresses no vertex in G_k , which gives (b). $\square$

We are now ready to specify how we add edges between the tail vertices of a recovery patch. Consider a recovery patch (S_1, S_2) of a graph G , where $S_1 = v_0v_1 \dots v_k$, and let

$$G_k = G \cup \{(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)\}.$$

Algorithm 8 describes how to remove the suppression pairs introduced in G_k that were not present in G .

Let H be the graph obtained by adding edges between vertices of S_2 such that every vertex in S_2 has at least one neighbor in S_2 . By Lemma 27, we have

$$\text{Supp}(H) \subseteq \text{Supp}(G_k) \setminus \{(v_k, u) \mid u \in S_2\}.$$

Since we add at most one edge per iteration and iterate over each vertex in S_2 only once, the total number of added edges is at most $|S_2|$.

Algorithm 8 Remove Suppression Pairs from Tail Vertices

Require: A graph G_k generated from a suppressor partition graph G by joining adjacent vertices of some recovery sequence with edges, the last vertex of the recovery sequence v_k , and the corresponding recovery tail S_2

Ensure: The graph H such that $\text{Supp}(H) \subseteq \text{Supp}(G_k) / \{(v_k, u) | u \in S_2\}$

```

1: procedure REMOVESUPPRESSIONPAIRSFROMRECOVERYTAIL( $G_k, v_k, S_2$ )
2:   Initialize  $H_0$  with  $G$  and  $i$  with 0.
3:   Let  $A$  the set of all vertices in  $S_2$  of type  $A$ .
4:   for  $v \in A$  do
5:     if  $v$  is not already marked black then
6:       Find the unique vertex  $u \in S_2$  that  $v$  properly suppresses in  $G_k$ .
7:       Let  $H_{i+1} = H_i \cup (u, v)$ .
8:       Mark both  $u$  and  $v$  as black.
9:       Increment  $i$ .
10:    end if
11:  end for
12:  Initialize  $H'_0$  with  $H_i$  and  $j$  with 0
13:  Let  $B$  be the set of all vertices in  $S_2$  of type  $B$ 
14:  for  $v \in B$  do
15:    if  $v$  has not been marked as black yet then
16:      Arbitrarily select a vertex  $u \in S_2$  such that  $u \neq v$ .
17:      Let  $H'_{j+1} = H'_j \cup (u, v)$ 
18:      Mark both  $u$  and  $v$  as black.
19:      Increment  $j$ 
20:    end if
21:  end for
22:  Let  $H = H'_j$ .
23:  return  $H$ 
24: end procedure

```

6.3.6 Removing a Single Suppression Pair

We are now ready to describe how to remove a single suppression pair. The algorithm is simple and is presented in Algorithm 9.

Algorithm 9 Remove Suppression Pair from a Graph (final version)

Require: A suppressor partitioned non-UNN G and a suppression pair (u, v) of G

Ensure: A graph H such that $\text{Supp}(H) \subseteq \text{Supp}(G) \setminus \{(u, v)\}$

```

1: procedure REMOVESUPPRESSIONPAIR( $G, u, v$ )
2:   Let  $(S_1, S_2) = \text{FINDRECOVERYPATCH}(G, u, v)$ 
3:   Let  $S_1 = v_0 v_1 \dots v_k$ 
4:   Initialize  $G_0$  with  $G$ .
5:   for  $i = 1$  to  $k$  do
6:      $G_i = G_{i-1} \cup \{(v_{i-1}, v_i)\}$ 
7:   end for
8:   Let  $H = \text{REMOVESUPPRESSIONPAIRSFROMRECOVERYTAIL}(G_k, v_k, S_2)$ 
9:   return  $H$ 
10: end procedure

```

It follows by induction that

$$\text{Supp}(G_k) = (\text{Supp}(G) \setminus \{(u, v)\}) \cup \{(v_k, u) \mid u \in S_2\}.$$

Moreover, the procedure `REMOVESUPPRESSIONPAIRSFROMRECOVERYTAIL` removes these newly introduced suppression pairs from G_k . Hence,

$$\text{Supp}(H) \subseteq \text{Supp}(G) \setminus \{(u, v)\}.$$

During this process, we add at most $k + |S_2|$ edges. Since v_0 is adjacent in G to every vertex in S_2 and to the vertices $v_2, v_3, \dots, v_k$, the total number of inserted edges is bounded by $\deg_G(u) + 1$. Furthermore, Algorithm 9 preserves the property that the graph is suppressor partitioned. Suppose, for contradiction, that H is not suppressor partitioned. Then one of the following must hold.

First, there exist two distinct vertices x and y that are twins in H . In this case,

$$\{(x, y), (y, x)\} \subseteq \text{Supp}(H) \subseteq \text{Supp}(G),$$

which implies that x and y are also twins in G . This contradicts the assumption that G is suppressor partitioned.

Second, there exist two distinct vertices x and y that share a common proper suppressor w in H . Then,

$$\{(w, x), (w, y)\} \subseteq \text{Supp}(H) \subseteq \text{Supp}(G),$$

which implies that x and y also share a common proper suppressor in G . This again contradicts the assumption that G is suppressor partitioned.

In either case, we obtain a contradiction. Therefore, H is suppressor partitioned.

6.3.7 Removing All Suppression Pairs from a Suppressor Partitioned Graph

By repeatedly identifying suppression pairs and removing them using Algorithm 9, we obtain an algorithm that eliminates all suppression pairs from G , thereby solving the UNN Completion problem. This iterative procedure is described in Algorithm 10.

Algorithm 10 Remove All Suppression Pair from a Suppressor Partitioned Graph

Require: A suppressor partitioned graph G

Ensure: a UNN Completion H of G

```

1: procedure REMOVEALLSUPPRESSIONPAIRS( $G$ )
2:   Initialize  $i$  with 0 and  $H_0$  with  $G$ 
3:   while  $\text{Supp}(H_i)$  is non-empty do
4:     Let  $(u_i, u'_i)$  be arbitrary suppression pair from  $\text{Supp}(H_i)$ 
5:     Let  $H_{i+1} = \text{REMOVESUPPRESSIONPAIR}(H_i, u_i, u'_i)$ 
6:     Increment  $i$  by 1
7:   end while
8:   return  $H_i$ 
9: end procedure

```

At iteration i , the procedure $\text{REMOVESUPPRESSIONPAIR}(H_i, u_i, u'_i)$ inserts at most $\deg_{H_i}(u_i) + 1$ edges. Thus, to bound the total number of inserted edges, it suffices to understand how $\deg_{H_i}(u_i)$ relates to $\deg_G(u_i)$.

The key observation is that, although the graph evolves over iterations, the degree of the vertex selected as a suppressor does not change. This is formalized in the following lemma, whose proof relies on Lemmas 36 and 37.

Lemma 42. *Let G be a suppressor partitioned graph, and let $H_0, H_1, \dots, H_t$ be the sequence of graphs generated during the execution of Algorithm 10 on input G . Let $(u_0, u'_0), (u_1, u'_1), \dots, (u_{t-1}, u'_{t-1})$ be the sequence of suppression pairs processed during the execution. Then, for all $0 \leq i \leq t - 1$,*

$$\deg_{H_i}(u_i) = \deg_G(u_i).$$

Proof. First observe that all graphs $H_0, H_1, \dots, H_t$ are suppressor partitioned.

Suppose, for contradiction, that there exists an index i such that

$$\deg_{H_i}(u_i) \neq \deg_G(u_i).$$

Since the algorithm only inserts edges, degrees can only increase, and hence

$$\deg_{H_i}(u_i) > \deg_G(u_i).$$

Therefore, there must exist some iteration $j < i$ at which an edge incident to u_i was inserted. This can only happen if u_i appears in the recovery patch constructed at iteration j . Let (S_1, S_2) be this recovery patch, where $S_1 = v_0 v_1 \dots v_k$.

We consider two cases.

Case 1: $u_i \in S_1$. By Lemma 36, since (u_i, u'_i) is a suppression pair in H_i , it must be that either $u_i = v_0$ and $u'_i = v_1$, or $u_i = v_k$ and $u'_i = v_{k-1}$. In both cases, u_i and u'_i are consecutive vertices in S_1 , and hence are made adjacent during iteration j . Therefore, they remain adjacent in H_i , contradicting the assumption that (u_i, u'_i) is a suppression pair in H_i .

Case 2: $u_i \in S_2$. By Lemma 40, it follows that $u'_i \in S_2$, and moreover, the edge $\{u_i, u'_i\}$ is added during iteration j by the procedure `REMOVESUPPRESSIONPAIRSFROMRECOVERYTAIL`. Hence, u_i and u'_i are adjacent in H_{j+1} , and therefore also in H_i . This again contradicts the assumption that (u_i, u'_i) is a suppression pair in H_i .

In both cases, we obtain a contradiction. Therefore, no edge incident to u_i is inserted in any iteration $j < i$, and hence

$$\deg_{H_i}(u_i) = \deg_G(u_i).$$

□

Using Lemma 42, we can come to the conclusion that Algorithm 10 inserts at most $\sum_{u \in S} \deg_G(u) + 1$ where S is the set of suppressors in G .

6.3.8 Putting It All Together

We can use Algorithm 7 and Algorithm 10 to finally construct an algorithm for the UNN Completion problem with a provable approximation guarantee. The key idea is to first transform the graph into a suppressor-partitioned graph using Algorithm 7, and then systematically eliminate all suppression pairs using Algorithm 10.

We present the full procedure in Algorithm 11.

Algorithm 11 Find a UNN Completion of a Graph

Require: A graph G

Ensure: a UNN Completion G' of G

- 1: Let $H = \text{PARTITIONSUPPRESSORS}(G)$.
 - 2: Let $G' = \text{REMOVEALLSUPPRESSIONPAIRS}(H)$.
 - 3: **return** G'
-

Algorithm 11 inserts edges in two phases: first in the procedure PARTITIONSUPPRESSORS, and then in the procedure REMOVEALLSUPPRESSIONPAIRS.

The procedure PARTITIONSUPPRESSORS inserts at most $\binom{k}{2}$ edges, where k is the number of suppressed vertices in G .

On the other hand, REMOVEALLSUPPRESSIONPAIRS inserts at most

$$\sum_{u \in S} (\deg_H(u) + 1),$$

where S is the set of suppressor vertices in H .

We bound this quantity as follows. Since $S \subseteq V(H)$,

$$\sum_{u \in S} \deg_H(u) \leq \sum_{u \in V(H)} \deg_H(u) = 2|E(H)|.$$

Hence,

$$\sum_{u \in S} (\deg_H(u) + 1) \leq 2|E(H)| + |S| \leq 2|E(H)| + |V(H)|.$$

Now, since H is obtained from G by adding at most $\binom{k}{2}$ edges in PARTITIONSUPPRESSORS, we have

$$|E(H)| \leq |E(G)| + \binom{k}{2}.$$

Thus,

$$\sum_{u \in S} (\deg_H(u) + 1) \leq 2|E(G)| + k(k-1) + |V(G)|.$$

Combining both phases, the total number of edges inserted by the algorithm is at most

$$\binom{k}{2} + 2|E(G)| + k(k-1) + |V(G)| = \frac{3}{2}k(k-1) + 2|E(G)| + |V(G)|.$$

We also know that any UNN completion of a graph with k suppressed vertices requires at least $\frac{k}{2}$ edge insertions.

This gives Algorithm 11 an approximation ratio of

$$3(k-1) + \frac{2|V(G)| + 4|E(G)|}{k}.$$

If $k = 1$, Algorithm 11 inserts at most $|V(G)| + |E(G)|$ edges. If $k \geq 2$, then

$$3(k-1) + \frac{2|V(G)| + 4|E(G)|}{k} \leq 4(|V(G)| + |E(G)|).$$

Thus, we obtain the following.

Theorem 5. *Algorithm 11 has an approximation ratio of $4(|V(G)| + |E(G)|)$.*

Chapter 7

UNN Extension and Completion Problems from a Parameterized Perspective

In this chapter, we shift our focus to the parameterized versions of the UNN Extension and Completion problems. First, we show that the decision versions of both problems are $\mathbf{W}[2]$ -hard when parameterized by the solution size. We then show that the UNN Extension problem can be solved in linear time when parameterized by the treewidth, given a tree decomposition, using Courcelle's theorem.

7.1 Parameterized Hardness of the UNN Extension and Completion Problems

To show that a problem is $\mathbf{W}[2]$ -hard, we must provide an FPT reduction from a problem already known to be $\mathbf{W}[2]$ -hard. In our case, we have already constructed a reduction from the Total Dominating Set problem to the UNN Extension problem and a reduction from the UNN Extension problem to the UNN Completion problem. Furthermore, it is trivial to see that both reductions also qualify as FPT reductions. Thus, to show that both problems are $\mathbf{W}[2]$ -hard, it suffices to show that the Total Dominating Set problem, parameterized by the total domination number, is $\mathbf{W}[2]$ -hard.

This can easily be established via an FPT reduction from the set cover problem parameterized by set cover size. Our reduction is shown in Algorithm [12](#).

Algorithm 12 Algorithm for solving the Set Cover problem parameterized by set cover size using black box for the total dominating set problem parameterized by total domination number

Require: A set cover instance $((\mathcal{U}, \mathcal{S}), k)$

Ensure: A single bit indicating whether or not a set cover of size at most k exists

```

1: if  $\bigcup_{s \in \mathcal{S}} s \neq \mathcal{U}$  then
2:   return False
3: else if A set cover of size 1 exists and  $k \geq 1$  then
4:   return True
5: else
6:   Let  $V = \mathcal{U} \cup \mathcal{S}$ .
7:   Let  $E_1 = \{(u, v) \in \mathcal{U} \times \mathcal{S} \mid u \in v\}$ 
8:   Let  $E_2 = \{(u, v) \in \mathcal{S} \times \mathcal{S} \mid u \neq v \wedge \min(u \setminus v \cup v \setminus u) \in u\}$ 
9:   Construct a graph  $G = (V, E_1 \cup E_2)$ .
10:  Find out whether there is a total dominating set of size at most  $k$  in  $G$  using a black-box
    algorithm for parameterized total dominating set problem.
11:  return The answer from the black box.
12: end if

```

It is obvious that Algorithm 12 runs in FPT time. Thus, it remains to prove its correctness.

Consider a set cover instance $((\mathcal{U}, \mathcal{S}), k)$. It is obvious that if no set cover exists, or if there exists a set cover of size 1 (and $k \geq 1$), then Algorithm 12 provides us with the correct answer.

Now, in the case where there is a set cover but no set cover of size 1, then we claim that there would be a set cover of size at most k if and only if there was a total dominating set of size at most k in the graph G (which was constructed in Line 9).

First, consider the case that there is a set cover S of size at most k . We claim that S is a total dominating set of G . To show this, we will need to show that every vertex $v \in V(G)$ has a neighbor in S . We will break this proof down into two cases.

Case a: $v \in \mathcal{U}$. Since $v \in \mathcal{U}$ and S is a set cover, there exists $s \in S$ such that $v \in s$ and thus v and s are adjacent.

Case b: $v \in \mathcal{S}$. Since all elements of $\mathcal{S}$ are adjacent to each other, any vertex in $s \in S - \{v\}$ is adjacent v . Since $|S| > 1$, $S - \{v\}$ is non-empty.

Thus, for every vertex $v \in V(G)$ has a neighbor in S and thus S is a total dominating set of size at most k .

Conversely, suppose there exists a total dominating set S of size at most k . Let $U = \mathcal{U} \cap S$. We can then construct a new set S' as follows: start with $S - U$, and for each $u \in U$, add a neighbor of u from $\mathcal{S}$ to S' if such a neighbor is not already present. Note that this is possible because the existence of S implies that there are no isolated vertices in G and thus every element of $\mathcal{U}$ has a neighbor in G . Since $\mathcal{U}$ is an independent set, this new set S' is a subset of $\mathcal{S}$. Furthermore, it is of size at most k . Moreover, it is straightforward to verify that S' is also a set cover.

Thus, there is a set cover of size at most k if and only if there is a total dominating set of

size at most k . Thus, Algorithm 12 outlines an FPT reduction, and the total dominating set is $\mathbf{W}[2]$ -hard. Therefore, both the UNN extension problem and the UNN completion problem, when parameterized by their solution sizes, are $\mathbf{W}[2]$ -hard.

Theorem 6. *The UNN Extension Problem, when parameterized by the extension size, is $\mathbf{W}[2]$ -hard.*

Theorem 7. *The UNN Completion Problem, when parameterized by the completion size, is $\mathbf{W}[2]$ -hard.*

7.2 The UNN Extension Problem Parameterized by Treewidth

In this section, we show that the minimum-cost UNN Extension problem can be solved in linear time on bounded-treewidth graphs using Courcelle's theorem.

Courcelle [24] showed that for bounded treewidth graphs, every decision problem definable in monadic second-order logic (MSO_2) can be solved in time linear in the number of vertices. Furthermore, if an optimization problem can be expressed as finding a set of vertices of minimum or maximum cardinality that satisfies a condition definable in MSO_2 , then it can also be solved in linear time [25].

By Lemma 8, finding the minimum-cost UNN extension is equivalent to finding a minimum good dominating set D of $\overline{G}$. We can model this problem in MSO_2 as minimizing the size of a set D that satisfies the formula $\text{DomSet}(\overline{G}, D) \wedge \text{Good}(\overline{G}, D)$. Here, $\text{DomSet}(\overline{G}, D) = \forall_{v \in V} \exists_{u \in D} \neg \text{adj}(u, v)$ verifies if D is a dominating set in $\overline{G}$, and $\text{Good}(\overline{G}, D) = \forall_{v \in V} (v \in D \vee (\exists_{u' \in V} \text{adj}(u', v) \wedge u' \notin D))$ ensures that D is good by checking if all vertices outside of D has a neighbor outside D in G .

Thus, using the optimization variant of Courcelle's Theorem, we can solve the minimum-cost UNN extension problem in linear time.

Theorem 8. *Let $G = (V, E)$ be a bounded-treewidth UNN. One can find a UNN extension of minimum cost in linear time.*

Chapter 8

Miscellaneous

In this chapter, we show two shorter results that do not fit into the broader study of this thesis. First, we show that UNNs have the property of containing no small forbidden subgraphs. Next, we explore a generalization of the UNN extension problem, named the UNN k -extension problem, and show that the problem is tractable for all $k \geq 2$.

8.1 No Small Forbidden Subgraphs in UNNs

In this section, we show that for any graph G , we can construct a unique neighborhood network G' that has G as its induced subgraph with only an additional logarithmic number of vertices. This implies that UNNs have no small forbidden induced subgraphs.

8.1.1 Constructing UNN from Arbitrary Graphs

Let G be a UNN with n vertices. We can construct a new graph $\text{UNN}(G, c)$ from G as follows:

1. Add c new vertices $s_1, s_2, \dots, s_c$ to G . We will denote the set of newly added vertices as S . For every vertex $s_i \in S$, connect s_i to every other vertex $s_j \in S - \{s_i\}$. Thus, the subgraph induced by S is the complete graph K_c .
2. For each vertex v_i of the original graph G , assign a unique subset S_i of S of size $\lfloor \frac{c}{2} \rfloor$, and add edges between v_i and the vertices of S_i . Note that for our construction to be well defined, $\binom{c}{\lfloor \frac{c}{2} \rfloor} \geq n$.
3. Add two new vertices p_1 and p_2 . We will denote the set $\{p_1, p_2\}$ as P . For every vertex $p_i \in P$, add an edge between p_i and every vertex in the original graph G . Add an edge between p_1 and p_2 .

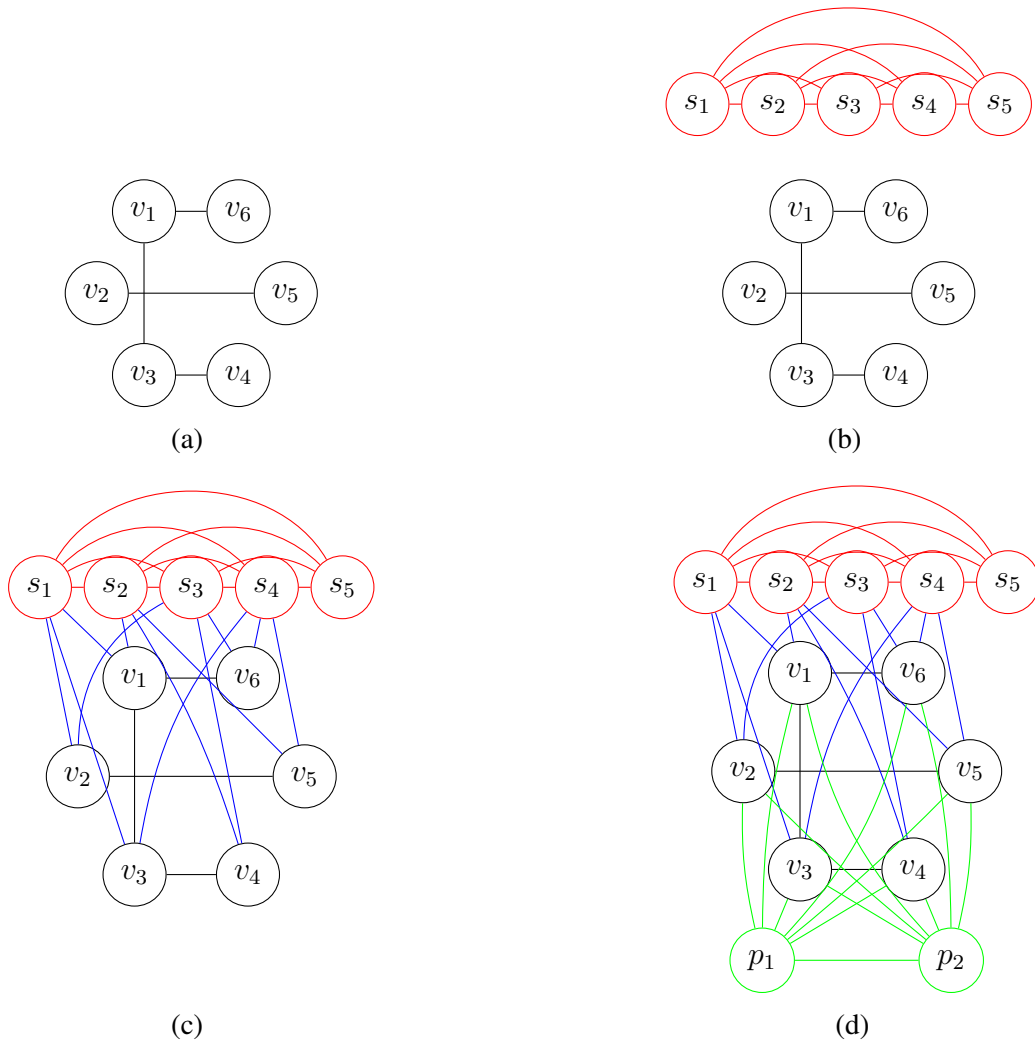


Figure 8.1: An arbitrary graph G and construction of $UNN(G, c)$ step by step

Figure 8.1a shows an arbitrary graph G with six vertices and Figures 8.1b to 8.1d show the construction of $UNN(G, c)$ step by step.

The following lemma shows that $UNN(G, c)$ is always a UNN except in boundary cases:

Lemma 43. *Let G be an arbitrary graph with $n > 1$ vertices and $c > 2$ a non-negative integer. Then the graph, $G' = UNN(G, c)$ is a UNN.*

Proof. We will show that for all ordered pairs of distinct vertices (u, v) , $N_{G'}(u) \not\subseteq N_{G'}(v)$. We break our proof into seven cases:

Case 1: $u, v \in S$. Since by construction, any two distinct vertices of S are adjacent, $N_{G'}(u) \not\subseteq N_{G'}(v)$ and $N_{G'}(v) \not\subseteq N_{G'}(u)$.

Case 2: $u \in S$ and $v \in V(G) \cup P$. Notice that v is adjacent to at most $\lfloor \frac{c}{2} \rfloor$ vertices in S but u is adjacent to $c - 1$ vertices in S . Since c is greater than two, $c - 1 \geq \lfloor \frac{c}{2} \rfloor$ and thus, $N_{G'}(u) \not\subseteq N_{G'}(v)$.

Case 3: $u \in V(G)$ and $v \in S$. Since $p_1 \in N_{G'}(u)$, but $p_1 \notin N_{G'}(v)$, $N_{G'}(u) \not\subseteq N_{G'}(v)$.

Case 4: $u, v \in V(G)$. Consider the neighbors of u and v that are in S . Both u and v are adjacent to two different subsets of S of size $\lfloor \frac{c}{2} \rfloor$. Thus, u has a neighbor in S that is not a neighbor of v . Therefore, $N_{G'}(u) \not\subseteq N_{G'}(v)$.

Case 5: $u \in V(G)$ and $v \in P$. Then, u and v are adjacent. Therefore, $N_{G'}(u) \not\subseteq N_{G'}(v)$.

Case 6: $u \in P$ and $v \in S$. Let p' be the element of P that is not u . Then, $p' \in N_{G'}(u)$, but $p' \notin N_{G'}(v)$. Thus, $N_{G'}(u) \not\subseteq N_{G'}(v)$.

Case 7: $u \in P$ and $v \in V(G) \cup P$. Then, u and v are adjacent. Therefore, $N_{G'}(u) \not\subseteq N_{G'}(v)$. Since in all seven cases $N_{G'}(u) \not\subseteq N_{G'}(v)$, we can safely conclude that G' is a UNN. $\square$

We denote the smallest integer c such that $\binom{c}{\lfloor \frac{c}{2} \rfloor} \geq n$ as $c^*(n)$. So, given any graph G with n vertices, $\text{UNN}(G, c^*(n))$ will be a UNN as long as $n > 1$ and $c^*(n) > 2$. Since $c^*(n) > 2$ for all $n \geq 3$, we can say the following:

Lemma 44. *Let G be an arbitrary graph with $n > 2$ vertices. Then the graph, $G' = \text{UNN}(G, c^*(n))$ is a UNN.*

The construction $\text{UNN}(G, c)$ adds $c + 2$ vertices to G . Furthermore, since $\text{UNN}(G, c)$ does not erase any of the edges or vertices in G or add new edges between the vertices of G , G remains as a induced subgraph in $\text{UNN}(G, c)$. This leads us to the following theorem.

Theorem 9. *For every graph G with $n > 2$ vertices, there exists a UNN G' with at most $n + c^*(n) + 2$ vertices such that G' contains G as an induced subgraph.*

8.1.2 Properties of $c^*(n)$

Since, for any graph G with $n > 2$ vertices, we can construct a UNN by adding only $c^*(n) + 2$ vertices, one may be interested in properties of the function. It is not hard to see that $c^*(n)$ can be computed in $\mathcal{O}(\lg n)$ number of arithmetic operations using binary search. Thus, we are more interested on findings bounds with closed forms.

The following Lemma provides us with bounds for $c^*(n)$.

Lemma 45. *Let $c^*(n)$ denote the smallest integer c such that $\binom{c}{\lfloor \frac{c}{2} \rfloor} \geq n$. Then,*

$$\lg n \leq c^*(n) \leq \lceil 2 \lg 2n \rceil$$

Proof. Proof of Lower Bound: Consider some integer $c < \lg n$. Then:

$$\begin{aligned} c &< \lg n \\ \Rightarrow 2^c &< n \\ \Rightarrow \binom{c}{\lfloor \frac{c}{2} \rfloor} &< n \end{aligned} \tag{8.1}$$

Since Equation 8.1 holds for all $c < \lg n$, $c^*(n)$ cannot be smaller than $\lg n$.

Proof of Upper Bound: Consider an integer $c \geq 2 \lg 2n$. Then,

$$\begin{aligned}
 c &\geq 2 \lg 2n \\
 \Rightarrow \frac{c}{2} &\geq \lg 2n \\
 \Rightarrow \left\lfloor \frac{c}{2} \right\rfloor + 1 &\geq \lg 2n \\
 \Rightarrow 2n &\leq 2 \times 2^{\left\lfloor \frac{c}{2} \right\rfloor} \\
 \Rightarrow n &\leq 2^{\left\lfloor \frac{c}{2} \right\rfloor} \\
 \Rightarrow n &\leq \left(\frac{c}{2} \right)^{\left\lfloor \frac{c}{2} \right\rfloor} \\
 \Rightarrow n &\leq \left(\frac{c}{\left\lfloor \frac{c}{2} \right\rfloor} \right)^{\left\lfloor \frac{c}{2} \right\rfloor}
 \end{aligned} \tag{8.2}$$

It is a well-known fact that,

$$\left(\frac{n}{k} \right)^k \leq \binom{n}{k} \leq \left(\frac{ne}{k} \right)^k \tag{8.3}$$

From Equation 8.3, we can say that, $\left(\frac{c}{\left\lfloor \frac{c}{2} \right\rfloor} \right)^{\left\lfloor \frac{c}{2} \right\rfloor} \leq \binom{c}{\left\lfloor \frac{c}{2} \right\rfloor}$. Thus, we can come to the conclusion from Equation 8.2 that $n \leq \binom{c}{\left\lfloor \frac{c}{2} \right\rfloor}$.

Therefore, $c^*(n) \leq \lceil 2 \lg 2n \rceil$. □

Note that Lemma 45 is true for all values of n whether large or small. However, when n is large, the bounds become much tighter, as we will show in the following lemma:

Lemma 46. *Let $c^*(n)$ denote the smallest integer c such that $\binom{c}{\left\lfloor \frac{c}{2} \right\rfloor} \geq n$. Then, $c^*(n) = (1 + o(1)) \lg n$.*

Proof. It is known that

$$\binom{n}{\left\lfloor \frac{n}{2} \right\rfloor} \sim \frac{2^n}{\sqrt{\pi n}} \tag{8.4}$$

Thus, for every $\epsilon > 0$, there exists a non-negative integer n_0 such that if $n > n_0$, then $(1 - \epsilon) \left(\frac{2^n}{\sqrt{\pi n}} \right) \leq \binom{n}{\left\lfloor \frac{n}{2} \right\rfloor} \leq (1 + \epsilon) \left(\frac{2^n}{\sqrt{\pi n}} \right)$.

Using this, consider some integer $c = \lceil (1 + \delta) \lg n \rceil$ for some $\delta > 0$. For any $0 < \epsilon < 1$, for

sufficiently large n , we have:

$$\begin{aligned}
 c &\geq (1 + \delta) \lg n \\
 \Rightarrow \binom{c}{\lfloor \frac{c}{2} \rfloor} &\geq (1 - \epsilon) \frac{2^{(1+\delta) \lg n}}{\sqrt{\frac{\pi}{2} \times (1 + \delta) \lg n}} \\
 &= (1 - \epsilon) \frac{n^{1+\delta}}{\sqrt{\frac{\pi}{2} \times (1 + \delta) \lg n}} \\
 &= n \frac{(1 - \epsilon)n^\delta}{\sqrt{\frac{\pi}{2} \times (1 + \delta) \lg n}}
 \end{aligned} \tag{8.5}$$

For large enough n , $(1 - \epsilon) \frac{n^\delta}{\sqrt{\frac{\pi}{2} (1+\delta) \lg n}}$ is greater than 1.

So, $\binom{c}{\lfloor \frac{c}{2} \rfloor} \geq n$. Therefore, for sufficiently large n , $c^*(n) \leq \lceil (1 + \delta) \lg n \rceil$ for all $\delta > 0$. Thus, $c^*(n) \leq \lceil (1 + o(1)) \lg n \rceil$.

Lemma 45 tells us that $c^*(n) \geq \lg n$. Thus, $c^*(n) = (1 + o(1)) \lg n$. $\square$

8.1.3 Optimality of Construction

A natural question is whether, under a different construction, we can generate a UNN from a graph G with n vertices by adding significantly fewer than $c^*(n) + 2$ vertices, which is required when using $\text{UNN}(G, c^*(n))$. The following theorem shows that our construction is indeed tight and very close to the optimal.

Theorem 10. *For every non-negative integer n , there exists a graph G such that no UNN with fewer than $n + c^*(n)$ vertices contains G as an induced subgraph.*

Proof. We claim that the graph G consisting of n isolated vertices is such a graph. Let G' be a UNN that contains G as an induced subgraph. Consider the neighborhoods of the vertices of G in G' . Each of their neighborhoods is a subset of $V(G')/V(G)$, and none of them is the subset of the other. That is, the set of neighborhoods of the vertices in $V(G)$ in the graph G' forms an antichain of size n in $\mathcal{P}(V(G')/V(G))$, the power set of $V(G')/V(G)$. But due to Sperner's Lemma [26], this is possible only if $\binom{|V(G')| - n}{\lfloor \frac{|V(G')| - n}{2} \rfloor} \geq n$. This holds if and only if $|V(G')| - n \geq c^*(n)$ that is, if and only if $|V(G')| \geq n + c^*(n)$. $\square$

8.1.4 Size of Forbidden Induced Subgraph

We know that a UNN of size $n + c^*(n) + 2$ has no forbidden induced subgraphs of size n . However, if the size of the UNN is n , what can we say about its forbidden induced subgraphs?

Theorem 11. *Unique Neighborhood Networks with $n > 16$ vertices have no forbidden induced subgraphs of size less than $n - 2 \lg n - 4$.*

Proof. Let $n > 16$ and let G be any graph with

$$|V(G)| = m \leq n - 2 \lg(2n) - 4.$$

We will show that G is not a forbidden induced subgraph of any UNN on n vertices.

Set

$$c = n - m - 2,$$

and consider the graph $\text{UNN}(G, c)$. By construction, this graph has exactly n vertices. It suffices to show that $c \geq c^*(m)$, so that the construction is valid and produces a UNN.

First, observe that $c^*(n)$ is non-decreasing in n , since $\binom{c}{\lfloor c/2 \rfloor}$ is increasing in c .

Next, by Lemma 45, we have

$$c^*(m) \leq \lceil 2 \lg(2m) \rceil.$$

Since $m \leq n - 2 \lg(2n) - 4$, it follows that

$$c^*(m) \leq \lceil 2 \lg(2n - 4 \lg(2n) - 8) \rceil.$$

Using the inequality $\lceil x \rceil \leq x + 1$, we obtain

$$c^*(m) \leq 2 \lg(2n - 4 \lg(2n) - 8) + 1.$$

Since $2n - 4 \lg(2n) - 8 < 2n$, we have

$$c^*(m) \leq 2 \lg(2n) + 1.$$

On the other hand,

$$c = n - m - 2 \geq n - (n - 2 \lg(2n) - 4) - 2 = 2 \lg(2n) + 2.$$

Thus,

$$c \geq 2 \lg(2n) + 2 > 2 \lg(2n) + 1 \geq c^*(m),$$

which implies that $c \geq c^*(m)$.

Finally, since $n > 16$, we have $c \geq 2$, so the construction $\text{UNN}(G, c)$ is valid. Therefore, there exists a UNN on n vertices that contains G as an induced subgraph.

It follows that no graph on at most $n - 2 \lg(2n) - 4$ vertices can be a forbidden induced subgraph of a UNN on n vertices. $\square$

8.2 The UNN k -Extension Problem

The *UNN k -extension problem* is a generalization of the UNN extension problem that seeks to find a k -extension of a UNN G that is also a UNN.

The UNN k -extension problem reduces to the UNN extension problem for $k = 1$. However, for all other values of k , the problem becomes trivial. The idea behind solving the UNN k -extension problem for $k \geq 2$ is to pair the newly added vertices arbitrarily and add an edge between each pair. If k is even, we are done.

If k is odd, one of the newly added vertices will remain unpaired. Then, we can pick an arbitrary pair of newly added vertices and connect this vertex to those two vertices.

If G as a UNN, such a k -extension will also be a UNN and only adds $\lceil \frac{k}{2} \rceil$ edges which is the minimum for $k \geq 2$.

Chapter 9

Conclusion

In this thesis, we studied the problem of generating and modifying Unique Neighborhood Networks (UNNs) in a manner suitable for practical applications, with a focus on both algorithmic and complexity-theoretic aspects. We investigated two central problems: the UNN Extension problem and the UNN Completion problem, providing approximation algorithms, hardness results, and parameterized analyses. We now summarize the contributions of each chapter.

Chapter 1. In the introduction, we first introduced the necessary graph-theoretic notation and formally defined Unique Neighborhood Networks (UNNs). We then motivated their study by highlighting their relevance in networking and cryptographic applications. We discussed the limitations of existing UNN generation techniques and emphasized the need for algorithmic approaches suitable for practical protocol design. Motivated by these limitations, we introduced two central combinatorial optimization problems studied in this thesis, namely the UNN Extension problem and the UNN Completion problem. Finally, we summarized the main contributions of the thesis and outlined its organization.

Chapter 2. This chapter introduced the notation and basic concepts used throughout the thesis. It provided the necessary background in graph theory, optimization and approximation algorithms, reductions (including approximation-preserving reductions), and parameterized complexity, which together form the foundation for the results developed in the subsequent chapters.

Chapter 3. In this chapter, we developed an approximation algorithm for the UNN Extension problem. We introduced the notions of suppressors and suppressed vertices, and showed that the problem of finding a feasible extension can be reduced to finding a special type of dominating set in the complement graph. Leveraging this connection, we designed a polynomial-time $\mathcal{O}(\lg n)$ -approximation algorithm.

Chapter 4. In this chapter, we established hardness of approximation results for the UNN Extension problem. We introduced the notion of UNNification, which allows the construction of a UNN from an arbitrary graph, and analyzed structural properties of such constructions. Using these tools, we developed a strict reduction from the Total Dominating Set problem to the UNN Extension problem. This reduction shows that the UNN Extension problem is log-**APX**-hard. This hardness result complements the $\mathcal{O}(\lg n)$ -approximation algorithm presented in Chapter 3, showing that the algorithm is essentially optimal in terms of approximation ratio up to constant factors.

Chapter 5. In this chapter we showed a strict reduction from the UNN Extension problem to the UNN Completion problem, proving that this problem is also log-**APX**-hard, like the UNN Extension problem.

Chapter 6. In this chapter, we presented a polynomial-time approximation algorithm for the UNN Completion problem. The algorithm transforms an arbitrary graph into a structured suppressor-partitioned graph through edge insertions and then constructs a solution on this transformed instance. The resulting algorithm achieves a linear-factor approximation, which provides an upper bound on the approximability of the problem.

On the other hand, Theorem 4 from Chapter 5 shows that the UNN Completion problem is lg-**APX**-hard, meaning that it cannot be approximated within a factor of $\epsilon \lg n$ for sufficiently small ϵ . Together, these results characterize the approximation landscape of the UNN Completion problem up to logarithmic factors.

Chapter 7. In this chapter, we studied the UNN Extension and Completion problems from a parameterized complexity perspective. We showed that the decision versions of both problems are **W[2]**-hard when parameterized by solution size, indicating their intractability under this parameterization. On the other hand, we obtained a positive result for the UNN Extension problem when parameterized by treewidth. In particular, we proved that the UNN Extension problem can be solved in linear time on bounded treewidth graphs using Courcelle's theorem. However, this approach does not extend to the UNN Completion problem, and no FPT algorithm for the UNN Completion problem parameterized by treewidth is known at the time of writing.

Chapter 8. In this chapter, we studied forbidden subgraphs in UNNs and showed that UNNs do not possess any small forbidden subgraphs. We also analyzed the UNN k -extension problem, a generalization of the UNN Extension problem, and showed that it is tractable for all values of $k \neq 1$.

Overall Contributions. Taken together, the results of this thesis provide a comprehensive study of algorithmic and complexity aspects of UNN modification problems. We established near-optimal approximation algorithms for both the extension and completion problems, complemented by matching hardness results. Furthermore, we explored the parameterized complexity landscape, identifying both hardness and tractable cases, and provided additional structural insights into UNNs.

Open Problems

Despite the progress made in this thesis, several questions remain open. In this section, we discuss several open problems and their status.

We hope that the techniques and results presented in this thesis contribute to a better understanding of UNNs and serve as a foundation for future work in this area.

$\mathcal{O}(\lg n)$ -Approximation for the UNN Completion Problem

Does there exist a polynomial-time $\mathcal{O}(\lg n)$ -approximation algorithm for the UNN Completion problem? This question naturally arises from the gap between the known upper and lower bounds for the problem. In Chapter 6, we presented a polynomial-time approximation algorithm achieving a linear-factor approximation. On the other hand, Chapter 5 established that the problem is $\log$ -APX-hard, implying that it cannot be approximated within a factor of $\epsilon \lg n$ for sufficiently small ϵ , unless $\mathbf{P} = \mathbf{NP}$.

Thus, there remains a significant gap between the current $\mathcal{O}(|V| + |E|)$ upper bound and the logarithmic lower bound. Closing this gap, either by improving the approximation algorithm or by strengthening the hardness result, remains an important open problem.

UNN Completion on Bounded Treewidth Graphs

Does there exist a polynomial-time algorithm for the UNN Completion problem on graphs of bounded treewidth? In Chapter 7, we showed that the UNN Extension problem becomes tractable when parameterized by treewidth using Courcelle's theorem. However, this approach does not extend to the UNN Completion problem, and no such algorithm is currently known.

The main difficulty lies in the fact that a solution to the UNN Completion problem is a subset of $E(\overline{G})$, rather than $E(G)$. As a result, it is not possible to directly express the problem in MSO_2 , since this would require quantifying over subsets of $V \times V$, which is not monadic. Moreover, quantifying over subsets of $V \times V$ is fundamentally different from quantifying over two subsets of V . Consequently, the natural formulations of the problem do not yield MSO_2 expressions.

Determining whether the UNN Completion problem admits a polynomial-time (or fixed-parameter tractable) algorithm on bounded treewidth graphs remains an open question.

Online UNN Extension

How does the UNN Extension problem behave in an online setting, and what is the best competitive ratio achievable?

In the online version of the problem, vertices arrive one at a time. Upon the arrival of each new vertex, the algorithm must make irrevocable decisions to ensure that the graph induced by the vertices seen so far forms a UNN at every step. This introduces a strong dependency between early and later decisions, as choices made for earlier vertices can significantly alter the feasible options for future ones.

It is natural to consider greedy strategies that maintain a minimal solution at each step; however, it is not known whether such approaches achieve optimal or near-optimal competitive ratios. The difficulty lies in the fact that local optimality does not necessarily preserve global feasibility, since early decisions may eliminate structures required for future extensions.

In contrast, in the offline setting, this problem corresponds to the UNN k -extension problem. For $k = 1$, a logarithmic approximation is known, while for $k \geq 2$, polynomial-time exact algorithms exist. However, these results do not directly translate to the online setting.

Determining the best achievable competitive ratio for the online UNN Extension problem remains an open question.

UNN Editing Problem

What is the minimum number of edge insertions and deletions required to transform a given graph into a Unique Neighborhood Network (UNN)?

This problem can be viewed as a variant of the general graph modification framework, where the goal is to transform an input graph into a member of a target graph class using a minimum number of edge edits. In particular, it can be seen as a variant of the Π -completion problem, where the objective is to modify a graph into a graph satisfying a given property Π . Such graph modification problems have been widely studied for various graph classes, including chordal graphs, interval graphs, and perfect graphs, and are known to be computationally challenging in general [15].

Determining the computational complexity and approximability of the UNN Editing problem remains an open question.

References

- [1] S. Rass, “Perfectly secure communication, based on graph-topological addressing in unique-neighborhood networks,” 2023. <https://arxiv.org/abs/1810.05602>.
- [2] D. P. Sumner, “Point determination in graphs,” *Discrete Mathematics*, vol. 5, no. 2, pp. 179–187, 1973.
- [3] J. J. Bull and C. M. Pease, “Combinatorics and variety of mating-type systems,” *Evolution*, pp. 667–671, 1989.
- [4] P. Erdős and A. Rényi, “On the evolution of random graphs,” *Publ. math. inst. hung. acad. sci*, vol. 5, no. 1, pp. 17–60, 1960.
- [5] E. N. Gilbert, “Random graphs,” *The Annals of Mathematical Statistics*, vol. 30, no. 4, pp. 1141–1144, 1959.
- [6] M. Penrose, *Random geometric graphs*, vol. 5. Oxford: OUP Oxford, 2003.
- [7] J. Katz and Y. Lindell, *Introduction to modern cryptography: principles and protocols*. Chapman and hall/CRC, 2007.
- [8] W. Diffie and M. E. Hellman, “New directions in cryptography,” *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976.
- [9] A. Shamir, “How to share a secret,” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [10] S. Rass, “Tell me who you are friends with and I will tell you who you are: Unique neighborhoods in random graphs,” *Theoretical Computer Science*, vol. 988, p. 114386, 2024.
- [11] M. R. Garey and D. S. Johnson, “Computers and intractability,” *A Guide to the*, 1979.
- [12] M. Yannakakis, “Computing the minimum fill-in is np-complete,” *SIAM Journal on Algebraic Discrete Methods*, vol. 2, no. 1, pp. 77–79, 1981.

- [13] P. W. Goldberg, M. C. Golumbic, H. Kaplan, and R. Shamir, “Four strikes against physical mapping of dna,” *Journal of Computational Biology*, vol. 2, no. 1, pp. 139–152, 1995.
- [14] T. Kashiwabara and T. Fujisawa, “An np-complete problem on interval graphs,” in *IEEE Symp. of Circuits and Systems*, pp. 82–83, 1979.
- [15] A. Natanzon, R. Shamir, and R. Sharan, “Complexity classification of some edge modification problems,” *Discrete Applied Mathematics*, vol. 113, no. 1, pp. 109–128, 2001.
- [16] R. Laskar, J. Pfaff, S. M. Hedetniemi, and S. T. Hedetniemi, “On the algorithmic complexity of total domination,” *SIAM Journal on Algebraic Discrete Methods*, vol. 5, no. 3, pp. 420–425, 1984.
- [17] D. S. Johnson, “Approximation algorithms for combinatorial problems,” in *Proceedings of the fifth annual ACM symposium on Theory of computing*, pp. 38–49, 1973.
- [18] N. Alon, D. Moshkovitz, and S. Safra, “Algorithmic construction of sets for k -restrictions,” *ACM Transactions on Algorithms (TALG)*, vol. 2, no. 2, pp. 153–177, 2006.
- [19] V. Kann, *On the approximability of NP-complete optimization problems*. PhD thesis, Royal Institute of Technology Stockholm, 1992.
- [20] B. Bollobás and E. J. Cockayne, “Graph-theoretic parameters concerning domination, independence, and irredundance,” *Journal of Graph Theory*, vol. 3, no. 3, pp. 241–249, 1979.
- [21] M. Cygan, F. V. Fomin, Ł. Kowalik, D. Lokshtanov, D. Marx, M. Pilipczuk, M. Pilipczuk, and S. Saurabh, *Parameterized algorithms*, vol. 5. Springer, 2015.
- [22] L. Cai, “Parameterized complexity of cardinality constrained optimization problems,” *The Computer Journal*, vol. 51, no. 1, pp. 102–121, 2008.
- [23] D. Marx, “Parameterized complexity and approximation algorithms,” *The Computer Journal*, vol. 51, no. 1, pp. 60–78, 2008.
- [24] B. Courcelle, “The monadic second-order logic of graphs. I. Recognizable sets of finite graphs,” *Information and computation*, vol. 85, no. 1, pp. 12–75, 1990.
- [25] S. Arnborg, J. Lagergren, and D. Seese, “Easy problems for tree-decomposable graphs,” *Journal of Algorithms*, vol. 12, no. 2, pp. 308–340, 1991.
- [26] E. Sperner, *Neuer Beweis für die Invarianz der Dimensionszahl und des Gebietes*. No. 7, Vandenhoeck & Ruprecht, 1928.

Index

- Π -Completion problem, 8
- W[0]**, 15
- k -extension of a graph, 6
- k -weighted-circuit-SAT, 16
- adjacent, 1
- approximation algorithms, 11
- approximation ratio, 11
- approximation-preserving reduction, 12
- blue dominating set, 31
- blue domination number, 31
- boolean circuit, 15
- closed neighborhood, 2
- complement of a graph, 2
- Courcelle's theorem, 17
- decision problem, 10
- depth of a boolean circuit, 16
- dia-graph, 1
- directed graph, 1
- dominating set, 12
- domination number, 13
- edge, 1
- edge set, 1
- extension neighborhood, 6
- extension of a graph, 6
- extension vertex, 6
- fixed parameter tractable, FPT, 14
- FPT reduction, Fixed Parameter Tractable reduction, 14
- good dominating set, 22
- graph, 1
- incident, 1
- incomparable, 20
- instance mapping, 12
- loneliness-free set, 33
- lonely vertex, 33
- mating graph, 2
- minimum edge fill problem, 8
- multi-edge, 1
- multi-graph, 1
- neighborhood, 2
- open neighborhood, 2
- optimization problem, 10
- optimizing search problem, 10
- parameterized algorithms, 13
- point determining graph, 2
- proper suppressor, 39
- properly suppressed, 39
- recovery patch, 44
- recovery sequence, 45
- recovery tail, 45
- reduction, 11

- self-loop, 1
- simple graph, 1
- solution mapping, 12
- strict reduction, 12
- suppress, 20
- suppressed vertex, 20
- suppression pair, 39
- suppression set, 39
- suppressor, 20

- total dominating set, 13
- total domination number, 13
- tree decomposition, 16
- treewidth, 16
- twin, 39

- unique neighborhood network, 2
- UNN, 2
- UNN k -extension problem, 73
- UNN completion, 8
- UNN completion problem, 8
- UNN extension, 6
- UNN extension problem, 6
- UNNification, 27

- vertex, 1
- vertex set, 1

- weak UNN, 2
- weft, 16

Generated using Postgraduate Thesis L^AT_EX Template, Version 1.03. Department of
Computer Science and Engineering, Bangladesh University of Engineering and
Technology, Dhaka, Bangladesh.

This thesis was generated on Saturday 18th April, 2026 at 11:06am.